

Contents

1	Geometry	6
1.1	二维几何基础操作	6
1.1.1	点和向量	6
1.1.2	极角排序	7
1.1.3	直线	7
1.1.4	线段	7
1.1.5	多边形	8
1.2	凸包	9
1.3	半平面交	11
1.4	整数半平面交	12
1.5	$O(n^2)$ Ear Clipping 三角剖分	13
1.6	单插入动态凸包	13
1.7	圆	14
1.7.1	基础操作	14
1.7.2	圆反演、阿波罗尼茨圆	16
1.7.3	圆面积并	17
1.7.4	多边形与圆面积交	18
1.8	球面基础, 经纬度球面距离	19
1.9	三维几何基础操作	20
1.10	三维凸包	23
1.11	相关技巧	25
1.11.1	点集的最小最大三角形	25
1.11.2	平面最近点对	26
1.11.3	判断多条线段是否有交点	26
1.11.4	多边形面积并	27
1.11.5	点、线段在简单多边形内	28
1.11.6	最小覆盖球	29
1.11.7	圆上整点	30
1.11.8	三相之力	30
1.12	相关公式	31
1.12.1	Heron's Formula	31
1.12.2	四面体内接球球心	31
1.12.3	三角形内心	31
1.12.4	三角形外心	31
1.12.5	三角形垂心	31
1.12.6	三角形偏心	31
1.12.7	三角形内接外接圆半径	31
1.12.8	Pick's Theorem 格点多边形面积	31
1.12.9	Euler's Formula 多面体与平面图的点、边、面	31
1.12.10	三角公式	32
1.12.11	超球坐标系	32
1.12.12	三维旋转公式	32
1.12.13	立体角公式	32
1.12.14	常用体积公式	32
1.12.15	扇形与圆弧重心	32
1.12.16	高维球体积	33
2	Tree & Graph	34
2.1	树的重心	34
2.2	树的直径	34
2.3	树链剖分	34

2.4	prufer 与树的互相转化	36
2.5	树哈希	37
2.6	内向基环树	40
2.7	Hopcroft-Karp $O(\sqrt{V}E)$	40
2.8	Hungarian $O(VE/w)$	41
2.9	Shuffle 一般图最大匹配 $O(VE)$	42
2.10	KM 最大权匹配 $O(V^3)$	42
2.11	有向图欧拉回路/路径	44
2.12	2-SAT, 强连通分量	47
2.13	Bitset Kosaraju	48
2.14	边双连通分量	49
2.15	点双连通分量	50
2.16	Dominator Tree 支配树	52
2.17	Dinic 最大流	52
2.18	原始对偶费用流	56
2.19	虚树	59
2.20	网络流总结	59
2.21	Gomory-Hu 无向图最小割树 $O(V^3E)$	61
2.22	Stoer-Wagner 无向图最小割 $O(VE + V^2 \log V)$	61
2.23	弦图	62
2.24	Minimum Mean Cycle 最小平均值环 $O(n^2)$	63
2.25	一般图最大匹配 - Blossom	63
2.26	图论结论	64
2.26.1	最小乘积问题原理	64
2.26.2	最小环	64
2.26.3	度序列的可图性	64
2.26.4	切比雪夫距离与曼哈顿距离转化	64
2.26.5	树链的交	64
2.26.6	带修改 MST	64
2.26.7	差分约束	65
2.26.8	Segment Tree Beats	65
2.26.9	二分图	65
2.26.10	稳定婚姻问题	65
2.26.11	竞赛图 Landau's Theorem	65
2.26.12	Ramsey Theorem $R(3,3)=6, R(4,4)=18$	65
2.26.13	树的计数 Prufer 序列	65
2.26.14	有根树计数 1,1,2,4,9,20,48,115,286,719,1842,4766	65
2.26.15	无根树计数	65
2.26.16	生成树计数 Kirchhoff's Matrix-Tree Theorem	65
2.26.17	有向图欧拉回路计数 BEST Theorem	66
2.26.18	Tutte Matrix	66
2.26.19	Edmonds Matrix	66
2.26.20	有向图无环定向, 色多项式	66
2.26.21	拟阵交问题	66
2.26.22	双极定向	67
2.26.23	图中的环	67
3	Data Structure	68
3.1	回滚并查集	68
3.2	莫队合集	69
3.2.1	普通莫队	69
3.2.2	树上莫队	69
3.2.3	带修莫队	69

3.2.4	回滚莫队	71
3.2.5	二次离线莫队	72
3.3	树状数组	75
3.4	单点修改线段树	76
3.5	懒标记线段树	78
3.6	线段树合并与分裂	81
3.7	笛卡尔树	82
3.8	点分治	82
3.9	点分树	83
3.10	树上启发式合并	83
3.11	$O(n \log n) - O(1)$ LCA	84
3.12	LCT 动态树	85
3.13	可持久化 Treap	85
3.14	PBDS 实现平衡树	86
3.15	PBDS 实现可并堆	87
3.16	手写 bitset	89
3.17	珂朵莉树	91
4	String	93
4.1	最小表示法	93
4.2	Manacher	93
4.3	KMP, exKMP	94
4.4	Border 树	94
4.5	AC 自动机	95
4.6	Lyndon Word Decomposition	97
4.7	后缀自动机	98
4.8	后缀数组	99
4.9	Suffix Balanced Tree 后缀平衡树	101
4.10	广义后缀自动机 (离线)	101
4.11	广义在线 SAM	103
4.12	回文自动机	104
4.13	回文自动机 + dp	107
4.14	Runs	108
4.15	字符串 Hash	111
4.16	String Conclusions	114
5	Math 数学	115
5.1	Long Long $O(1)$ 乘, Barrett	115
5.2	exgcd, 逆元	115
5.3	同余方程	116
5.4	CRT 中国剩余定理	116
5.5	Miller Rabin, Pollard Rho	117
5.6	线性基	118
5.7	扩展卢卡斯	119
5.8	阶乘取模	119
5.9	类欧几里得直线下格点统计	120
5.10	万能欧几里德	120
5.11	平方剩余	120
5.12	线性同余不等式	121
5.13	原根	121
5.14	多项式	122
5.15	FFT	125
5.16	NTT	125

5.17	MTT 任意模数卷积	126
5.18	多项式运算	126
5.18.1	多项式求逆 开根 $\ln \exp$	126
5.18.2	多项式除法 取模	127
5.18.3	多点求值	128
5.18.4	插值	128
5.19	线性递推	129
5.20	Berlekamp-Massey 最小多项式	130
5.21	FWT	131
5.22	K 进制 FWT	132
5.23	Simplex 单纯形	132
5.24	高斯消元最小范数解	133
5.25	Pell 方程	134
5.26	解一元三次方程	134
5.27	自适应 Simpson	134
6	Tricks	136
6.1	线性预处理左边第 k 大	136
6.2	二维支配前驱查询	136
6.3	区间众数	136
6.4	区间 mex 和 mex 区间	136
6.5	平面不相交矩形建树	138
6.6	无穷字符串匹配/比较	139
6.7	曼哈顿和切比雪夫	140
6.8	接受有效日期的最小状态自动机	140
7	Appendix	142
7.1	Formulas 公式表	142
7.1.1	Mobius Inversion	142
7.1.2	杜教筛	142
7.1.3	降幂公式	142
7.1.4	其他常用公式	142
7.1.5	单位根反演	142
7.1.6	Arithmetic Function	143
7.1.7	组合数	144
7.1.8	Fibonacci Numbers, Lucas Numbers	145
7.1.9	Sum of Powers	145
7.1.10	Catalan Numbers 1, 1, 2, 5, 14, 42, 132, 429, 1430...	146
7.1.11	Motzkin Numbers 1, 1, 2, 4, 9, 21, 51, 127, 323, 835...	146
7.1.12	Derangement 错排数 0, 1, 2, 9, 44, 265, 1854, 14833...	146
7.1.13	Bell Numbers 1, 1, 2, 5, 15, 52, 203, 877, 4140 ...	146
7.1.14	Stirling Numbers	146
7.1.15	Eulerian Numbers	147
7.1.16	Harmonic Numbers, 1, 3/2, 11/6, 25/12, 137/60...	148
7.1.17	卡迈克尔函数	148
7.1.18	求拆分数	148
7.1.19	Bernoulli Numbers 1, 1/2, 1/6, 0, -1/30, 0, 1/42 ...	148
7.1.20	k-th MAX-MIN 反演	149
7.1.21	伍德伯里矩阵等式	149
7.1.22	Sum of Squares	149
7.1.23	枚举勾股数 Pythagorean Triple	149
7.1.24	四面体体积 Tetrahedron Volume	149
7.1.25	杨氏矩阵与钩子公式	149

7.1.26	常见博弈游戏	149
7.1.27	概率相关	150
7.1.28	邻接矩阵行列式的意义	150
7.1.29	Others (某些近似数值公式在这里)	150
7.2	Calculus, Integration Table 导数积分表	151
7.3	Python Hint	153
8	Miscellany	157
8.1	初始模板	157
8.2	WIN 运行脚本	157
8.3	Zeller 日期公式	158
8.4	有理数二分: Stern-Brocot 树, Farey 序列	158
8.5	黄金三分	158
8.6	DP 优化	158
8.6.1	四边形不等式	158
8.6.2	树形背包优化	159
8.6.3	$O(n \cdot \max a_i)$ Subset Sum	159
8.7	Hash Table	160
8.8	基数排序	160
8.9	O3, 读入优化	160
8.10	builtin	161
8.11	试机赛与纪律文件	162
8.12	Constant Table 常数表	162

1. Geometry

1.1 二维几何基础操作

1.1.1 点和向量

```

1 using T = long double; // 全局数据类型
2
3 constexpr T eps = 1e-8;
4 constexpr T INF = numeric_limits<T>::max();
5 constexpr T PI = 3.14159265358979323841;
6 struct Point {
7     T x, y;
8
9     bool operator==(const Point &a) const {
10         return (abs(x - a.x) <= eps && abs(y - a.y) <= eps);
11     }
12     bool operator<(const Point &a) const {
13         if (abs(x - a.x) <= eps) return y < a.y - eps;
14         return x < a.x - eps;
15     }
16     bool operator>(const Point &a) const { return !(*this < a || *this == a); }
17     Point operator+(const Point &a) const { return {x + a.x, y + a.y}; }
18     Point operator-(const Point &a) const { return {x - a.x, y - a.y}; }
19     Point operator-() const { return {-x, -y}; }
20     Point operator*(const T k) const { return {k * x, k * y}; }
21     Point operator/(const T k) const { return {x / k, y / k}; }
22     T operator*(const Point &a) const { return x * a.x + y * a.y; } // 点积
23     T operator^(const Point &a) const {
24         return x * a.y - y * a.x;
25     } // 叉积, 注意优先级
26
27     // 1: 左侧 (逆时针) -1: 右侧 (顺时针) 0: 共线
28     int toleft(const Point &a) const {
29         const auto t = (*this) ^ a;
30         return (t > eps) - (t < -eps);
31     } // to-left 测试
32     T len2() const { return (*this) * (*this); } // 向量长度的平方
33     T dis2(const Point &a) const {
34         return (a - (*this)).len2();
35     } // 两点距离的平方
36     int quad() const // 象限判断 0: 原点 1:x 轴正 2: 第一象限 3:y 轴正 4: 第二象限
37         // 5:x 轴负 6: 第三象限 7:y 轴负 8: 第四象限
38     {
39         if (abs(x) <= eps && abs(y) <= eps) return 0;
40         if (abs(y) <= eps) return x > eps ? 1 : 5;
41         if (abs(x) <= eps) return y > eps ? 3 : 7;
42         return y > eps ? (x > eps ? 2 : 4) : (x > eps ? 8 : 6);
43     }
44
45     // 必须用浮点数
46     T len() const { return sqrtl(len2()); } // 向量长度
47     T dis(const Point &a) const { return sqrtl(dis2(a)); } // 两点距离
48     T ang(const Point &a) const {
49         return acosl(max(-1.01, min(1.01, ((*this) * a) / (len() * a.len()))));
50     } // 返回较小的向量夹角
51     Point rot(const T rad) const {
52         return {x * cos(rad) - y * sin(rad), x * sin(rad) + y * cos(rad)};
53     } // 逆时针旋转 (给定角度)

```

```
54 Point rot(const T cosr, const T sinr) const {
55     return {x * cosr - y * sinr, x * sinr + y * cosr};
56 } // 逆时针旋转 (给定角度的正弦与余弦)
57};
```

1.1.2 极角排序

```
1 struct Argcmp {
2     bool operator()(const Point &a, const Point &b) const {
3         const int qa = a.quad(), qb = b.quad();
4         if (qa != qb) return qa < qb;
5         const auto t = a ^ b;
6         // if (abs(t)<=eps) return a*a<b*b-eps; // 不同长度的向量需要分开
7         return t > eps;
8     }
9};
```

1.1.3 直线

```
1 struct Line {
2     Point p, v; // p 为直线上一点, v 为方向向量
3
4     bool operator==(const Line &a) const {
5         return v.toleft(a.v) == 0 && v.toleft(p - a.p) == 0;
6     }
7     int toleft(const Point &a) const {
8         return v.toleft(a - p);
9     } // to-left 测试
10    bool operator<(const Line &a) const // 半平面交算法定义的排序
11    {
12        if (abs(v ^ a.v) <= eps && v * a.v >= -eps) return toleft(a.p) == -1;
13        return Argcmp()(v, a.v);
14    }
15
16    // 必须用浮点数
17    Point inter(const Line &a) const {
18        return p + v * ((a.v ^ (p - a.p)) / (v ^ a.v));
19    } // 直线交点
20    ld dis(const Point &a) const {
21        return abs(v ^ (a - p)) / v.len();
22    } // 点到直线距离
23    Point proj(const Point &a) const {
24        return p + v * ((v * (a - p)) / (v * v));
25    } // 点在直线上的投影
26};
```

1.1.4 线段

```
1 struct Segment {
2     Point a, b;
3
4     bool operator<(const Segment &s) const {
5         return make_pair(a, b) < make_pair(s.a, s.b);
6     }
7
8     // 判定性函数建议在整数域使用
9
10    // 判断点是否在线段上
```

```

11 // -1 点在线段端点 | 0 点不在线段上 | 1 点严格在线段上
12 int is_on(const Point &p) const {
13     if (p == a || p == b) return -1;
14     return (p - a).toleft(p - b) == 0 && (p - a) * (p - b) < -eps;
15 }
16
17 // 判断线段直线是否相交
18 // -1 直线经过线段端点 | 0 线段和直线不相交 | 1 线段和直线严格相交
19 int is_inter(const Line &l) const {
20     if (l.toleft(a) == 0 || l.toleft(b) == 0) return -1;
21     return l.toleft(a) != l.toleft(b);
22 }
23
24 // 判断两线段是否相交
25 // -1 在某一线段端点处相交 | 0 两线段不相交 | 1 两线段严格相交
26 int is_inter(const Segment &s) const {
27     if (is_on(s.a) || is_on(s.b) || s.is_on(a) || s.is_on(b)) return -1;
28     const Line l{a, b - a}, ls{s.a, s.b - s.a};
29     return l.toleft(s.a) * l.toleft(s.b) == -1 &&
30         ls.toleft(a) * ls.toleft(b) == -1;
31 }
32
33 // 点到线段距离（必须用浮点数）
34 ld dis(const Point &p) const {
35     if ((p - a) * (b - a) < -eps || (p - b) * (a - b) < -eps)
36         return min(p.dis(a), p.dis(b));
37     const Line l{a, b - a};
38     return l.dis(p);
39 }
40
41 // 两线段间距离（必须用浮点数）
42 ld dis(const Segment &s) const {
43     if (is_inter(s)) return 0;
44     return min({dis(s.a), dis(s.b), s.dis(a), s.dis(b)});
45 }
46 };

```

1.1.5 多边形

```

1 struct Polygon {
2     vector<Point> p; // 以逆时针顺序存储
3
4     size_t nxt(const size_t i) const { return i == p.size() - 1 ? 0 : i + 1; }
5     size_t pre(const size_t i) const { return i == 0 ? p.size() - 1 : i - 1; }
6
7     // 回转数
8     // 返回值第一项表示点是否在多边形边上
9     // 对于狭义多边形，回转数为 0 表示点在多边形外，否则点在多边形内
10    pair<bool, int> winding(const Point &a) const {
11        int cnt = 0;
12        for (size_t i = 0; i < p.size(); i++) {
13            const Point u = p[i], v = p[nxt(i)];
14            if (abs((a - u) ^ (a - v)) <= eps && (a - u) * (a - v) <= eps)
15                return {true, 0};
16            if (abs(u.y - v.y) <= eps) continue;
17            const Line uv = {u, v - u};
18            if (u.y < v.y - eps && uv.toleft(a) <= 0) continue;
19            if (u.y > v.y + eps && uv.toleft(a) >= 0) continue;
20            if (u.y < a.y - eps && v.y >= a.y - eps) cnt++;

```



```

21         if (u.y >= a.y - eps && v.y < a.y - eps) cnt--;
22     }
23     return {false, cnt};
24 }
25
26 // 多边形面积的两倍
27 // 可用于判断点的存储顺序是顺时针或逆时针
28 T area() const {
29     T sum = 0;
30     for (size_t i = 0; i < p.size(); i++) sum += p[i] ^ p[nxt(i)];
31     return sum;
32 }
33
34 // 多边形的周长
35 ld circ() const {
36     T sum = 0;
37     for (size_t i = 0; i < p.size(); i++) sum += p[i].dis(p[nxt(i)]);
38     return sum;
39 }
40 };

```

1.2 凸包

```

1 // 凸多边形
2 struct Convex : Polygon {
3     // 闵可夫斯基和
4     Convex operator+(const Convex &c) const {
5         const auto &p = this->p;
6         vector<Segment> e1(p.size()), e2(c.p.size()),
7             edge(p.size() + c.p.size());
8         vector<Point> res;
9         res.reserve(p.size() + c.p.size());
10        const auto cmp = [](const Segment &u, const Segment &v) {
11            return Argcmp()(u.b - u.a, v.b - v.a);
12        };
13        for (size_t i = 0; i < p.size(); i++) e1[i] = {p[i], p[this->nxt(i)]};
14        for (size_t i = 0; i < c.p.size(); i++) e2[i] = {c.p[i], c.p[c.nxt(i)]};
15        rotate(e1.begin(), min_element(e1.begin(), e1.end(), cmp), e1.end());
16        rotate(e2.begin(), min_element(e2.begin(), e2.end(), cmp), e2.end());
17        merge(e1.begin(), e1.end(), e2.begin(), e2.end(), edge.begin(), cmp);
18        const auto check = [](const vector<Point> &res, const Point &u) {
19            const auto back1 = res.back(), back2 = *prev(res.end(), 2);
20            return (back1 - back2).toleft(u - back1) == 0 &&
21                (back1 - back2) * (u - back1) >= -eps;
22        };
23        auto u = e1[0].a + e2[0].a;
24        for (const auto &v : edge) {
25            while (res.size() > 1 && check(res, u)) res.pop_back();
26            res.push_back(u);
27            u = u + v.b - v.a;
28        }
29        if (res.size() > 1 && check(res, res[0])) res.pop_back();
30        return {res};
31    }
32
33    // 旋转卡壳
34    // 例：凸多边形的直径的平方
35    T rotcaliper() const {
36        const auto &p = this->p;

```

```

37     if (p.size() == 1) return 0;
38     if (p.size() == 2) return p[0].dis2(p[1]);
39     const auto area = [](const Point &u, const Point &v, const Point &w) {
40         return (w - u) ^ (w - v);
41     };
42     T ans = 0;
43     for (size_t i = 0, j = 1; i < p.size(); i++) {
44         const auto nxti = this->nxt(i);
45         ans = max({ans, p[j].dis2(p[i]), p[j].dis2(p[nxti])});
46         while (area(p[this->nxt(j)], p[i], p[nxti]) >=
47             area(p[j], p[i], p[nxti])) {
48             j = this->nxt(j);
49             ans = max({ans, p[j].dis2(p[i]), p[j].dis2(p[nxti])});
50         }
51     }
52     return ans;
53 }
54
55 // 判断点是否在凸多边形内
56 // 复杂度 O(logn)
57 // -1 点在多边形边上 | 0 点在多边形外 | 1 点在多边形内
58 int is_in(const Point &a) const {
59     const auto &p = this->p;
60     if (p.size() == 1) return a == p[0] ? -1 : 0;
61     if (p.size() == 2) return Segment{p[0], p[1]}.is_on(a) ? -1 : 0;
62     if (a == p[0]) return -1;
63     if ((p[1] - p[0]).toleft(a - p[0]) == -1 ||
64         (p.back() - p[0]).toleft(a - p[0]) == 1)
65         return 0;
66     const auto cmp = [&](const Point &u, const Point &v) {
67         return (u - p[0]).toleft(v - p[0]) == 1;
68     };
69     const size_t i =
70         lower_bound(p.begin() + 1, p.end(), a, cmp) - p.begin();
71     if (i == 1) return Segment{p[0], p[i]}.is_on(a) ? -1 : 0;
72     if (i == p.size() - 1 && Segment{p[0], p[i]}.is_on(a)) return -1;
73     if (Segment{p[i - 1], p[i]}.is_on(a)) return -1;
74     return (p[i] - p[i - 1]).toleft(a - p[i - 1]) > 0;
75 }
76
77 // 凸多边形关于某一方向的极点
78 // 复杂度 O(logn)
79 // 参考资料: https://codeforces.com/blog/entry/48868
80 template <typename F>
81 size_t extreme(const F &dir) const {
82     const auto &p = this->p;
83     const auto check = [&](const size_t i) {
84         return dir(p[i]).toleft(p[this->nxt(i)] - p[i]) >= 0;
85     };
86     const auto dir0 = dir(p[0]);
87     const auto check0 = check(0);
88     if (!check0 && check(p.size() - 1)) return 0;
89     const auto cmp = [&](const Point &v) {
90         const size_t vi = &v - p.data();
91         if (vi == 0) return 1;
92         const auto checkv = check(vi);
93         const auto t = dir0.toleft(v - p[0]);
94         if (vi == 1 && checkv == check0 && t == 0) return 1;
95         return checkv ^ (checkv == check0 && t <= 0);
96     };

```

```

97     return partition_point(p.begin(), p.end(), cmp) - p.begin();
98 }
99
100 // 过凸多边形外一点求凸多边形的切线, 返回切点下标
101 // 复杂度  $O(\log n)$ 
102 // 必须保证点在多边形外
103 pair<size_t, size_t> tangent(const Point &a) const {
104     const size_t i = extreme([&](const Point &u) { return u - a; });
105     const size_t j = extreme([&](const Point &u) { return a - u; });
106     return {i, j};
107 }
108
109 // 求平行于给定直线的凸多边形的切线, 返回切点下标
110 // 复杂度  $O(\log n)$ 
111 pair<size_t, size_t> tangent(const Line &a) const {
112     const size_t i = extreme([&](...) { return a.v; });
113     const size_t j = extreme([&](...) { return -a.v; });
114     return {i, j};
115 }
116 };
117
118 // 点集的凸包
119 // Andrew 算法, 复杂度  $O(n \log n)$ 
120 Convex convexhull(vector<Point> p) {
121     vector<Point> st;
122     if (p.empty()) return Convex{st};
123     sort(p.begin(), p.end());
124     const auto check = [&](const vector<Point> &st, const Point &u) {
125         const auto back1 = st.back(), back2 = *prev(st.end(), 2);
126         return (back1 - back2).toleft(u - back1) <= 0;
127     };
128     for (const Point &u : p) {
129         while (st.size() > 1 && check(st, u)) st.pop_back();
130         st.push_back(u);
131     }
132     size_t k = st.size();
133     p.pop_back();
134     reverse(p.begin(), p.end());
135     for (const Point &u : p) {
136         while (st.size() > k && check(st, u)) st.pop_back();
137         st.push_back(u);
138     }
139     st.pop_back();
140     return Convex{st};
141 }

```

1.3 半平面交

```

1 // 半平面交
2 // 排序增量法, 复杂度  $O(n \log n)$ 
3 // 输入与返回值都是用直线表示的半平面集合
4 vector<Line> halfinter(vector<Line> l, const T lim = 1e9) {
5     const auto check = [&](const Line &a, const Line &b, const Line &c) {
6         return a.toleft(b.inter(c)) < 0;
7     };
8     // 无精度误差的方法, 但注意取值范围会扩大到三次方
9     /*const auto check=[&](const Line &a,const Line &b,const Line &c)
10     {
11         const Point

```

```

12     p=a.v*(b.v^c.v),q=b.p*(b.v^c.v)+b.v*(c.v^(b.p-c.p))-a.p*(b.v^c.v); return
13     p.toleft(q)<0;
14     }*/
15     l.push_back({{-lim, 0}, {0, -1}});
16     l.push_back({{0, -lim}, {1, 0}});
17     l.push_back({{lim, 0}, {0, 1}});
18     l.push_back({{0, lim}, {-1, 0}});
19     sort(l.begin(), l.end());
20     deque<Line> q;
21     for (size_t i = 0; i < l.size(); i++) {
22         if (i > 0 && l[i - 1].v.toleft(l[i].v) == 0 &&
23             l[i - 1].v * l[i].v > eps)
24             continue;
25         while (q.size() > 1 && check(l[i], q.back(), q[q.size() - 2]))
26             q.pop_back();
27         while (q.size() > 1 && check(l[i], q[0], q[1])) q.pop_front();
28         if (!q.empty() && q.back().v.toleft(l[i].v) <= 0) return vector<Line>();
29         q.push_back(l[i]);
30     }
31     while (q.size() > 1 && check(q[0], q.back(), q[q.size() - 2])) q.pop_back();
32     while (q.size() > 1 && check(q.back(), q[0], q[1])) q.pop_front();
33     return vector<Line>(q.begin(), q.end());
34 }

```

1.4 整数半平面交

```

1 struct line : point {
2     | LD z; // ax + by + c >= 0
3     | line () {}
4     | line (LD a, LD b, LD c): point(a, b), z(c) {}
5     | line (cp a, cp b): point((b-a).rot90()), z(det(a, b)){
6     | LD operator () (cp a) const{return dot(a, *this) + z;}};
7 point line_inter (cl u, cl v) {
8     return point(det({u.z, u.y}, {v.z, v.y}),
9                 det({u.x, u.z}, {v.x, v.z}) ) / -det(u, v); }
10 LD dis (cl l, cp x = {0, 0}) { return l(x) / l.len(); }
11 bool is_para(cl x, cl y) { return !sgn(det(x, y)); }
12 LD det(cl a, cl b, cl c) {
13     | return det(a,b)*c.z + det(b,c)*a.z + det(c,a)*b.z;}
14 int check(cl a, cl b, cl c) { // sgn left(a, inter(b, c))
15     | return sgn(det(b, c, a)) * sgn(det(b, c)); }
16 bool turn_left(cl a, cl b, cl c){return check(a, b, c) >0;}
17 bool cmp (cl a, cl b) {
18     if (is_para(a, b) && dot(a, b) > 0) return dis(a) < dis(b);
19     return half(a) == half(b) ? sgn(det(a,b))>0 : half(b)>0;}
20 // 用以上函数替换 HPI 函数, 需要 half(point)
21 line perp(cl l) { return {l.y, -l.x, 0}; } // 垂直
22 line para(cl l, cp o) { // 过一点平行
23     return {l.x, l.y, l.z - l(o)}; }
24 point proj(cp x, cl l) {return x - l * (l(x)/l.len2());}
25 point refl(cp x, cl l) {return x - l * (l(x)/l.len2())*2;}
26 bool is_perp(cl x, cl y) { return !sgn(dot(x, y)); }
27 LD area(cl a, cl b, cl c) { // 0 when error
28     | LD d = det(a, b, c);
29     | return d * d / (det(a, b) * det(b, c) * det(c, a)); }
30 vector <line> cut (const vector <line>& o, line l){
31     | vector <line> ret; int n = (int) o.size();
32     | for (int i = 0; i < n; i++) {
33         | cl u = o[i], v = o[(i+1) % n], w = o[(i + 2) % n];

```

```

34 | | int va = check(l, u, v), vb = check(l, v, w);
35 | | if (va > 0 || vb > 0 || (va == 0 && vb == 0))
36 | | | ret.push_back(v);
37 | | if (va >= 0 && vb < 0) ret.push_back(l);
38 | } return ret; }

```

1.5 $O(n^2)$ Ear Clipping 三角剖分

```

1 vector <array <int, 3>> tri;
2 void solve () {
3 | list <int> l;
4 | for (int k = 0; k < n; k++) l.push_back(k);
5 | auto check = [&](auto u, auto v, auto w) {
6 | | if (turn(u, v, w) <= 0) return false;
7 | | for (auto i : l)
8 | | | if (turn(u, v, p[i]) == 1 &&
9 | | | | turn(v, w, p[i]) == 1 &&
10 | | | | turn(w, u, p[i]) == 1) return false;
11 | | return true; };
12 | for (auto it = l.begin(); l.size() > 3; ) {
13 | | auto u = (it == l.begin() ? prev(l.end()) : prev(it));
14 | | auto v = (next(it) == l.end() ? l.begin() : next(it));
15 | | if (!check(p[*u], p[*it], p[*v])) it = v;
16 | | else {
17 | | | tri.push_back({*u, *it, *v});
18 | | | l.erase(it);
19 | | | it = u; } }
20 | tri.push_back({l.front(), *next(l.begin()), l.back()}); }

```

1.6 单插入动态凸包

```

1 struct hull { // upper hull, left to right
2 set <point> a; LL tot;
3 hull () {tot = 0;}
4 LL calc(auto it) {
5 | auto u = it == a.begin() ? a.end() : prev(it);
6 | auto v = next(it);
7 | LL ret = 0;
8 | if (u != a.end()) ret += det(*u, *it);
9 | if (v != a.end()) ret += det(*it, *v);
10 | if (u != a.end() && v != a.end()) ret -= det(*u, *v);
11 | return ret; }
12 void insert (point p) {
13 | if (!a.size()) { a.insert (p); return; }
14 | auto it = a.lower_bound (p);
15 | bool out;
16 | if (it == a.begin()) out = (p < *it); // special case
17 | else if (it == a.end()) out = true;
18 | else out = turn(*prev(it), *it, p) > 0;
19 | if (!out) return;
20 | while (it != a.begin()) {
21 | | auto o = prev(it);
22 | | if (o == a.begin() || turn(*prev(o), *o, p) < 0) break;
23 | | else erase(o); }
24 | while (it != a.end()) {
25 | | auto o = next(it);
26 | | if (o == a.end() || turn(p, *it, *o) < 0) break;
27 | | else erase(it), it = o; }

```

```
28 | tot += calc(a.insert(p).first); }
29 void erase(auto it) { tot -= calc(it); a.erase(it); } };
```

1.7 圆

1.7.1 基础操作

```
1 struct Circle {
2     Point c;
3     T r; // 一般来说必须用浮点数
4
5     bool operator==(const Circle &a) const {
6         return c == a.c && abs(r - a.r) <= eps;
7     }
8     T circ() const { return 2 * PI * r; } // 周长
9     T area() const { return PI * r * r; } // 面积
10
11     // 点与圆的关系
12     // -1 圆上 | 0 圆外 | 1 圆内
13     int is_in(const Point &p) const {
14         const T d = p.dis(c);
15         return abs(d - r) <= eps ? -1 : d < r - eps;
16     }
17
18     // 直线与圆关系
19     // 0 相离 | 1 相切 | 2 相交
20     int relation(const Line &l) const {
21         const T d = l.dis(c);
22         if (d > r + eps) return 0;
23         if (abs(d - r) <= eps) return 1;
24         return 2;
25     }
26
27     // 圆与圆关系
28     // -1 相同 | 0 相离 | 1 外切 | 2 相交 | 3 内切 | 4 内含
29     int relation(const Circle &a) const {
30         if (*this == a) return -1;
31         const T d = c.dis(a.c);
32         if (d > r + a.r + eps) return 0;
33         if (abs(d - r - a.r) <= eps) return 1;
34         if (abs(d - abs(r - a.r)) <= eps) return 3;
35         if (d < abs(r - a.r) - eps) return 4;
36         return 2;
37     }
38
39     // 直线与圆的交点
40     vector<Point> inter(const Line &l) const {
41         const T d = l.dis(c);
42         const Point p = l.proj(c);
43         const int t = relation(l);
44         if (t == 0) return vector<Point>();
45         if (t == 1) return vector<Point>{p};
46         const T k = sqrt(r * r - d * d);
47         return vector<Point>{p - (l.v / l.v.len()) * k,
48                               p + (l.v / l.v.len()) * k};
49     }
50
51     // 圆与圆交点
52     vector<Point> inter(const Circle &a) const {
```

```

53     const T d = c.dis(a.c);
54     const int t = relation(a);
55     if (t == -1 || t == 0 || t == 4) return vector<Point>();
56     Point e = a.c - c;
57     e = e / e.len() * r;
58     if (t == 1 || t == 3) {
59         if (r * r + d * d - a.r * a.r >= -eps) return vector<Point>{c + e};
60         return vector<Point>{c - e};
61     }
62     const T costh = (r * r + d * d - a.r * a.r) / (2 * r * d),
63           sinth = sqrt(1 - costh * costh);
64     return vector<Point>{c + e.rot(costh, -sinth), c + e.rot(costh, sinth)};
65 }
66
67 // 圆与圆交面积
68 T inter_area(const Circle &a) const {
69     const T d = c.dis(a.c);
70     const int t = relation(a);
71     if (t == -1) return area();
72     if (t < 2) return 0;
73     if (t > 2) return min(area(), a.area());
74     const T costh1 = (r * r + d * d - a.r * a.r) / (2 * r * d),
75           costh2 = (a.r * a.r + d * d - r * r) / (2 * a.r * d);
76     const T sinth1 = sqrt(1 - costh1 * costh1),
77           sinth2 = sqrt(1 - costh2 * costh2);
78     const T th1 = acos(costh1), th2 = acos(costh2);
79     return r * r * (th1 - costh1 * sinth1) +
80           a.r * a.r * (th2 - costh2 * sinth2);
81 }
82
83 // 过圆外一点圆的切线
84 vector<Line> tangent(const Point &a) const {
85     const int t = is_in(a);
86     if (t == 1) return vector<Line>();
87     if (t == -1) {
88         const Point v = {-(a - c).y, (a - c).x};
89         return vector<Line>{{a, v}};
90     }
91     Point e = a - c;
92     e = e / e.len() * r;
93     const T costh = r / c.dis(a), sinth = sqrt(1 - costh * costh);
94     const Point t1 = c + e.rot(costh, -sinth), t2 = c + e.rot(costh, sinth);
95     return vector<Line>{{a, t1 - a}, {a, t2 - a}};
96 }
97
98 // 两圆的公切线
99 vector<Line> tangent(const Circle &a) const {
100     const int t = relation(a);
101     vector<Line> lines;
102     if (t == -1 || t == 4) return lines;
103     if (t == 1 || t == 3) {
104         const Point p = inter(a)[0], v = {-(a.c - c).y, (a.c - c).x};
105         lines.push_back({p, v});
106     }
107     const T d = c.dis(a.c);
108     const Point e = (a.c - c) / (a.c - c).len();
109     if (t <= 2) {
110         const T costh = (r - a.r) / d, sinth = sqrt(1 - costh * costh);
111         const Point d1 = e.rot(costh, -sinth), d2 = e.rot(costh, sinth);
112         const Point u1 = c + d1 * r, u2 = c + d2 * r, v1 = a.c + d1 * a.r,

```

```

113         v2 = a.c + d2 * a.r;
114         lines.push_back({u1, v1 - u1});
115         lines.push_back({u2, v2 - u2});
116     }
117     if (t == 0) {
118         const T costh = (r + a.r) / d, sinth = sqrt(1 - costh * costh);
119         const Point d1 = e.rot(costh, -sinth), d2 = e.rot(costh, sinth);
120         const Point u1 = c + d1 * r, u2 = c + d2 * r, v1 = a.c - d1 * a.r,
121             v2 = a.c - d2 * a.r;
122         lines.push_back({u1, v1 - u1});
123         lines.push_back({u2, v2 - u2});
124     }
125     return lines;
126 }
127
128 // 圆的反演
129 // auto result = circle.inverse(line);
130 // if (std::holds_alternative<Circle>(result))
131 // Circle c = std::get<Circle>(result);
132 std::variant<Circle, Line> inverse(const Line &l) const {
133     if (l.toleft(c) == 0) return l;
134     const Point v =
135         l.toleft(c) == 1 ? Point{l.v.y, -l.v.x} : Point{-l.v.y, l.v.x};
136     const T d = r * r / l.dis(c);
137     const Point p = c + v / v.len() * d;
138     return Circle{(c + p) / 2, d / 2};
139 }
140
141 std::variant<Circle, Line> inverse(const Circle &a) const {
142     const Point v = a.c - c;
143     if (a.is_in(c) == -1) {
144         const T d = r * r / (a.r + a.r);
145         const Point p = c + v / v.len() * d;
146         return Line{p, {-v.y, v.x}};
147     }
148     if (c == a.c) return Circle{c, r * r / a.r};
149     const T d1 = r * r / (c.dis(a.c) - a.r),
150         d2 = r * r / (c.dis(a.c) + a.r);
151     const Point p = c + v / v.len() * d1, q = c + v / v.len() * d2;
152     return Circle{(p + q) / 2, p.dis(q) / 2};
153 }
154 };

```

1.7.2 圆反演、阿波罗尼茨圆

所有关于两点 A, B 满足 $PA/PB = k$ 且不等于 1 的点 P 的轨迹是一个圆.

圆幂: 半径为 R 的圆 O , 任意一点 P 到 O 的幂为 $h = OP^2 - R^2$

圆幂定理: 过 P 的直线交圆在 A 和 B 两点, 则 $PA \cdot PB = |h|$

根轴: 到两圆等幂点的轨迹是一条垂直于连心线的直线

反演: 已知一圆 C , 圆心为 O , 半径为 r , 如果 P 与 P' 在过圆心 O 的直线上, 且 $OP \cdot OP' = r^2$, 则称 P 与 P' 关于 O 互为反演. 一般 C 取单位圆.

反演的性质:

不过反演中心的直线反形是过反演中心的圆, 反之亦然.

不过反演中心的圆, 它的反形是一个不过反演中心的圆.

两条直线在交点 A 的夹角, 等于它们的反形在相应点 A' 的夹角, 但方向相反.

两个相交圆周在交点 A 的夹角等于它们的反形在相应点 A' 的夹角, 但方向相反.

直线和圆周在交点 A 的夹角等于它们的反演图形在相应点 A' 的夹角, 但方向相反.

正交圆反形也正交. 相切圆反形也相切, 当切点为反演中心时, 反形为两条平行线. 两两相切的圆 r_1, r_2, r_3 , 求与他们都相切的圆 r_4 . 分母取负号, 答案再取绝对值, 为外切圆半径. 分母取正号为内切圆半径.

$$r_4^{\pm} = \frac{r_1 r_2 r_3}{r_1 r_2 + r_1 r_3 + r_2 r_3 \pm 2\sqrt{r_1 r_2 r_3 (r_1 + r_2 + r_3)}}$$

```

1 circle inv_c2c(point O, LD R, circle A) {
2   | LD OA = dis(A.c, O);
3   | LD RB = 0.5 * R * R * (1 / (OA - A.r) - 1 / (OA + A.r));
4   | LD OB = OA * RB / A.r;
5   | point B = O + (A.c - O) * (OB / OA);
6   | return {B, RB};
7 } // 点 O 在圆 A 外, 求圆 A 的反演圆 B, R 是反演半径
8 circle inv_l2c(point O, LD R, line l) {
9   | point P = proj_to_line(O, l);
10  | LD d = dis(O, P);
11  | LD RB = R * R / (2 * d);
12  | point VB = (P - O) / d * RB;
13  | return {O + VB, RB};
14 } // 不过 O 点的直线反演为过 O 点的圆, R 是反演半径
15 line inv_c2l (point O, LD R, circle A) {
16   | LD t = R * R / (2 * A.r);
17   | point p = O + (A.c - O).unit() * t;
18   | return {p, p + (O - p).rot90()};
19 } // 过 O 点的圆反演为不过 O 点的直线, R 是反演半径

```

1.7.3 圆面积并

```

1 // 圆面积并
2 // 轮廓积分, 复杂度 O(n^2 log n)
3 // ans[i] 表示被至少覆盖了 i+1 次的区域的面积
4 vector<T> area_union(const vector<Circle> &circs) {
5   const size_t siz = circs.size();
6   using arc_t = tuple<Point, T, T, T>;
7   vector<vector<arc_t>> arcs(siz);
8   const auto eq = [](const arc_t &u, const arc_t &v) {
9     const auto [u1, u2, u3, u4] = u;
10    const auto [v1, v2, v3, v4] = v;
11    return u1 == v1 && abs(u2 - v2) <= eps && abs(u3 - v3) <= eps &&
12           abs(u4 - v4) <= eps;
13  };
14
15  auto cut_circ = [&](const Circle &ci, const size_t i) {
16    vector<pair<T, int>> evt;
17    evt.push_back({-PI, 0});
18    evt.push_back({PI, 0});
19    int init = 0;
20    for (size_t j = 0; j < circs.size(); j++) {
21      if (i == j) continue;
22      const Circle &cj = circs[j];
23      if (ci.r < cj.r - eps && ci.relation(cj) >= 3) init++;
24      const auto inters = ci.inter(cj);
25      if (inters.size() == 1)
26        evt.push_back(
27          {atan2l((inters[0] - ci.c).y, (inters[0] - ci.c).x), 0});
28      if (inters.size() == 2) {
29        const Point d1 = inters[0] - ci.c, dr = inters[1] - ci.c;
30        T arg1 = atan2l(d1.y, d1.x), argr = atan2l(dr.y, dr.x);
31        if (abs(arg1 + PI) <= eps) arg1 = PI;

```

```

32         if (abs(argr + PI) <= eps) argr = PI;
33         if (argl > argr + eps) {
34             evt.push_back({argl, 1});
35             evt.push_back({PI, -1});
36             evt.push_back({-PI, 1});
37             evt.push_back({argr, -1});
38         } else {
39             evt.push_back({argl, 1});
40             evt.push_back({argr, -1});
41         }
42     }
43 }
44 sort(evt.begin(), evt.end());
45 int sum = init;
46 for (size_t i = 0; i < evt.size(); i++) {
47     sum += evt[i].second;
48     if (abs(evt[i].first - evt[i + 1].first) > eps)
49         arcs[sum].push_back(
50             {ci.c, ci.r, evt[i].first, evt[i + 1].first});
51     if (abs(evt[i + 1].first - PI) <= eps) break;
52 }
53 };
54
55 const auto oint = [](const arc_t &arc) {
56     const auto [cc, cr, l, r] = arc;
57     if (abs(r - l - PI - PI) <= eps) return 2.01 * PI * cr * cr;
58     return cr * cr * (r - l) + cc.x * cr * (sin(r) - sin(l)) -
59         cc.y * cr * (cos(r) - cos(l));
60 };
61
62 for (size_t i = 0; i < circs.size(); i++) {
63     const auto &ci = circs[i];
64     cut_circ(ci, i);
65 }
66 vector<T> ans(siz);
67 for (size_t i = 0; i < siz; i++) {
68     T sum = 0;
69     sort(arcs[i].begin(), arcs[i].end());
70     int cnt = 0;
71     for (size_t j = 0; j < arcs[i].size(); j++) {
72         if (j > 0 && eq(arcs[i][j], arcs[i][j - 1]))
73             arcs[i + (++cnt)].push_back(arcs[i][j]);
74         else
75             cnt = 0, sum += oint(arcs[i][j]);
76     }
77     ans[i] = sum / 2;
78 }
79 return ans;
80 }

```

1.7.4 多边形与圆面积交

```

1 // 圆与多边形面积交
2 T area_inter(const Circle &circ, const Polygon &poly) {
3     const auto cal = [](const Circle &circ, const Point &a, const Point &b) {
4         if ((a - circ.c).toleft(b - circ.c) == 0) return 0.01;
5         const auto ina = circ.is_in(a), inb = circ.is_in(b);
6         const Line ab = {a, b - a};
7         if (ina && inb) return ((a - circ.c) ^ (b - circ.c)) / 2;

```

```

8      if (ina && !inb) {
9          const auto t = circ.inter(ab);
10         const Point p = t.size() == 1 ? t[0] : t[1];
11         const T ans = ((a - circ.c) ^ (p - circ.c)) / 2;
12         const T th = (p - circ.c).ang(b - circ.c);
13         const T d = circ.r * circ.r * th / 2;
14         if ((a - circ.c).toleft(b - circ.c) == 1) return ans + d;
15         return ans - d;
16     }
17     if (!ina && inb) {
18         const Point p = circ.inter(ab)[0];
19         const T ans = ((p - circ.c) ^ (b - circ.c)) / 2;
20         const T th = (a - circ.c).ang(p - circ.c);
21         const T d = circ.r * circ.r * th / 2;
22         if ((a - circ.c).toleft(b - circ.c) == 1) return ans + d;
23         return ans - d;
24     }
25     const auto p = circ.inter(ab);
26     if (p.size() == 2 && Segment{a, b}.dis(circ.c) <= circ.r + eps) {
27         const T ans = ((p[0] - circ.c) ^ (p[1] - circ.c)) / 2;
28         const T th1 = (a - circ.c).ang(p[0] - circ.c),
29                 th2 = (b - circ.c).ang(p[1] - circ.c);
30         const T d1 = circ.r * circ.r * th1 / 2,
31                 d2 = circ.r * circ.r * th2 / 2;
32         if ((a - circ.c).toleft(b - circ.c) == 1) return ans + d1 + d2;
33         return ans - d1 - d2;
34     }
35     const T th = (a - circ.c).ang(b - circ.c);
36     if ((a - circ.c).toleft(b - circ.c) == 1)
37         return circ.r * circ.r * th / 2;
38     return -circ.r * circ.r * th / 2;
39 };
40
41 T ans = 0;
42 for (size_t i = 0; i < poly.p.size(); i++) {
43     const Point a = poly.p[i], b = poly.p[poly.nxt(i)];
44     ans += cal(circ, a, b);
45 }
46 return ans;
47 }

```

1.8 球面基础, 经纬度球面距离

球面距离: 连接球面两点的大圆劣弧 (所有曲线中最短)

球面角: 球面两个大圆弧所在半平面形成的二面角

球面凸多边形: 把一个球面多边形任意一边向两方无限延长成大圆, 其余边都在此大圆的同旁.

球面角盈 E : 球面凸 n 边形的内角和与 $(n-2)\pi$ 的差

离北极夹角 θ , 距离 h 的球冠: $S = 2\pi R h = 2\pi R^2(1 - \cos \theta)$, $V = \frac{\pi h^2}{3}(3R - h)$

球面凸 n 边形面积: $S = ER^2$

```

1 // longitude 经度范围:  $\pm\pi$ , latitude 纬度范围:  $\pm\pi/2$ 
2 LD sphereDis(LD lon1, LD lat1, LD lon2, LD lat2, LD R) {
3     | return R * acos(cos(lat1) * cos(lat2) * cos(lon1 - lon2) + sin(lat1) *
      ↪ sin(lat2)); }

```

1.9 三维几何基础操作

```

1 // p3 是一个三维点/向量类
2 // 并且需要实现了向量的基本运算: +, -, *, /, dot(), unit() (单位化) 等
3
4 using ld = long double;
5
6 // 定义一个用于浮点数比较的极小值 epsilon
7 const ld eps = 1e-9;
8
9 /**
10  * @brief 判断浮点数 x 的符号
11  * @param x 输入的浮点数
12  * @return -1 (如果 x < 0), 0 (如果 x 在精度范围内为零), 1 (如果 x > 0)
13  */
14 int sgn(ld x) {
15     if (std::abs(x) < eps) return 0;
16     return x < 0 ? -1 : 1;
17 }
18
19 // 三维点/向量结构体
20 struct p3 {
21     ld x, y, z;
22
23     // --- 基本运算符重载 ---
24     p3 operator+(const p3 &a) const { return {x + a.x, y + a.y, z + a.z}; }
25     p3 operator-(const p3 &a) const { return {x - a.x, y - a.y, z - a.z}; }
26     p3 operator*(ld s) const { return {x * s, y * s, z * s}; }
27     p3 operator/(ld s) const { return {x / s, y / s, z / s}; }
28
29     // --- 比较运算符重载 ---
30     // 基于 sgn 函数比较两个点/向量是否在精度范围内相等
31     bool operator==(const p3 &a) const {
32         return sgn(x - a.x) == 0 && sgn(y - a.y) == 0 && sgn(z - a.z) == 0;
33     }
34
35     // 计算向量的长度 (模)
36     ld len() const { return std::sqrtl(x * x + y * y + z * z); }
37
38     // 将向量单位化 (返回方向相同, 长度为 1 的向量)
39     p3 unit() const {
40         ld l = len();
41         // 处理零向量的情况, 避免除以零
42         if (sgn(l) == 0) return {0, 0, 0};
43         return *this / l;
44     }
45 };
46
47 /**
48  * @brief 计算两个向量的点积 (Dot Product)
49  */
50 ld dot(const p3 &a, const p3 &b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
51 /**
52  * @brief 计算旋转矩阵 A。
53  * @param n 旋转轴 (需要是单位向量)。
54  * @param cosw 逆时针旋转角度的余弦值。
55  * 一个点 (x_old, y_old, z_old) 变换后的坐标为
56  * new[i] = sum(old[j] * A[i][j]) for j=0..2
57  */
58 void calc(p3 n, ld cosw) {

```

```

59     ld sinw = sqrt(1 - cosw * cosw); // 计算旋转角度的正弦值
60     n = n.unit(); // 将旋转轴向量单位化
61     for (int i = 0; i < 3; i++) {
62         int j = (i + 1) % 3, k = (j + 1) % 3;
63         ld x = n[i], y = n[j], z = n[k];
64         // 对角线元素
65         A[i][i] = (y * y + z * z) * cosw + x * x;
66         // 非对角线元素
67         A[i][j] = x * y * (1 - cosw) + z * sinw;
68         A[i][k] = x * z * (1 - cosw) - y * sinw;
69     }
70 }
71
72 // 计算两个三维向量的叉积
73 p3 cross(p3 a, p3 b) {
74     return p3(a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z,
75             a.x * b.y - a.y * b.x);
76 }
77
78 /**
79  * @brief 计算三个三维向量的混合积
80  * @return 混合积 dot(cross(a, b), c)。其绝对值等于由 a, b, c
81  * 构成的平行六面体的体积。
82  */
83 ld mix(p3 a, p3 b, p3 c) { return dot(cross(a, b), c); }
84
85 // 三维直线/线段结构体
86 struct l3 {
87     p3 s, t; // 由两点 s, t 定义, 方向向量为 t - s
88 };
89
90 // 三维平面结构体
91 struct plane {
92     p3 nor; // 平面的单位法向量
93     ld m; // 原点到平面的有向距离 (满足 dot(nor, P) = m, 其中 P
94           // 是平面上任意一点)
95
96     /**
97      * @brief 构造函数: 通过一个法向量 r 和平面上一点 a 来定义平面。
98      */
99     plane(p3 r, p3 a) : nor(r.unit()), m(dot(nor, a)) {}
100 };
101
102 /**
103  * @brief 计算点 a 在平面 b 上的投影点。
104  * @param a 空间中的一点。
105  * @param b 目标平面。
106  * @return 点 a 在平面 b 上的正交投影点。
107  */
108 p3 project_to_plane(p3 a, plane b) {
109     // a 到平面的有向距离为 (dot(a, b.nor) - b.m)
110     // 从 a 点沿着法向量反方向移动这段距离即可到达投影点
111     return a + b.nor * (b.m - dot(a, b.nor));
112 }
113
114 /**
115  * @brief 计算两条三维直线 x 和 y 之间的最近点对。
116  * @param x 第一条直线。
117  * @param y 第二条直线。
118  * @return 一个 pair, 包含在直线 x 和 y 上的最近点。

```

```

119 * @note 通过求解线性方程组找到使两点间距离最小化的参数 s 和 t。
120 *  $P(s) = x.s + (x.t - x.s) * s$ 
121 *  $Q(t) = y.s + (y.t - y.s) * t$ 
122 */
123 pair<p3, p3> l3_closest(l3 x, l3 y) {
124     p3 dx = x.t - x.s;
125     p3 dy = y.t - y.s;
126     ld a = dot(dx, dx);
127     ld b = dot(dx, dy);
128     ld e = dot(dy, dy);
129     ld d = a * e - b * b; // 行列式, 如果为 0 则两直线平行
130
131     p3 r = x.s - y.s;
132     ld c = dot(dx, r);
133     ld f = dot(dy, r);
134
135     // 求解参数 s 和 t
136     ld s = (b * f - c * e) / d;
137     ld t = (a * f - c * b) / d;
138
139     return {x.s + dx * s, y.s + dy * t};
140 }
141
142 /**
143  * @brief 计算直线 b 与平面 a 的交点。
144  * @param a 平面。
145  * @param b 直线。
146  * @return 交点坐标。
147  * @warning 需要注意分母为零的情况 (直线与平面平行)。
148  */
149 p3 intersect(plane a, l3 b) {
150     // 将直线方程  $P(t) = b.s + (b.t - b.s) * t$  代入平面方程  $a.nor \cdot P = a.m$ 
151     // 解出参数 t
152     ld t = dot(a.nor, a.nor * a.m - b.s) / dot(a.nor, b.t - b.s);
153     return b.s + (b.t - b.s) * t;
154 }
155
156 /**
157  * @brief 计算两个平面 a 和 b 的交线。
158  * @param a 第一个平面。
159  * @param b 第二个平面。
160  * @return 表示交线的 l3 结构体。
161  * @warning 需要注意两平面平行的情况 (叉积为零向量)。
162  */
163 l3 intersect(plane a, plane b) {
164     // 交线的方向向量垂直于两个平面的法向量
165     p3 d = cross(a.nor, b.nor);
166     // 为了找到交线上的一个点, 可以构造第三个平面 (法向量为 d) 再求三平面交点
167     // 这里使用了一种更直接的几何解法
168     p3 d2 = cross(b.nor, d);
169     ld t = dot(d2, a.nor);
170     // s 是交线上的一个点
171     p3 s = d2 * (a.m - dot(b.nor * b.m, a.nor)) / t + b.nor * b.m;
172     return {s, s + d};
173 }
174
175 /**
176  * @brief 计算三个平面 a, b, c 的交点。
177  * @param a 第一个平面。
178  * @param b 第二个平面。

```

```

179 * @param c 第三个平面。
180 * @return 三个平面的唯一交点。
181 * @warning 需要注意分母 mix(c1,c2,c3) 为零的情况（三个平面不交于一点）。
182 */
183 p3 intersect(plane a, plane b, plane c) {
184     // 方法一：先求出 b, c 的交线，再求该直线与平面 a 的交点
185     // return intersect(a, intersect(b, c));
186
187     // 方法二：使用克莱姆法则求解由三个平面方程构成的线性方程组
188     p3 c1(a.nor.x, b.nor.x, c.nor.x); // 系数矩阵的列向量
189     p3 c2(a.nor.y, b.nor.y, c.nor.y);
190     p3 c3(a.nor.z, b.nor.z, c.nor.z);
191     p3 c4(a.m, b.m, c.m); // 常数项向量
192
193     // mix(c1, c2, c3) 是系数矩阵的行列式
194     return 1 / mix(c1, c2, c3) *
195         p3(mix(c4, c2, c3), mix(c1, c4, c3), mix(c1, c2, c4));
196 }

```

1.10 三维凸包

```

1
2
3
4 // -----
5 // 三维凸包
6 // 使用说明：
7 // 1. 将所有三维点放入全局变量 vector<p3> p 中。
8 // 2. 调用 solve() 函数。
9 // 3. 如果 solve() 返回 true，则凸包构造成功。
10 // 4. 结果存储在全局变量 vector<Face> face 中，
11 //    其中每个 Face 是一个包含三个整数的数组 {a, b, c}，
12 //    代表凸包的一个三角面，由点 p[a], p[b], p[c] 构成。
13 //
14 // 前置依赖：
15 // - p3: 三维点/向量类。
16 // - mix(): 三个向量的混合积。
17 // - cross(): 两个向量的叉积。
18 // - N: 一个略大于预计处理的最大点数的常量，用于 mark 数组。
19 // -----
20
21 const int N = 2010;
22
23 vector<p3> p; // 全局变量，存储所有输入的点
24 int mark[N][N], stp; // 标记数组，用于处理边界
25 typedef array<int, 3> Face; // Face 类型，用三个点的索引表示一个三角面
26 vector<Face> face; // 全局变量，存储最终凸包的所有面
27
28 /**
29 * @brief 计算四点构成的四面体的有向体积。
30 * @return 体积 > 0 表示点 d 在平面 (a,b,c) 的法向量正方向一侧。
31 */
32 LD volume(int a, int b, int c, int d) {
33     return mix(p[b] - p[a], p[c] - p[a], p[d] - p[a]);
34 }
35
36 /**
37 * @brief (内部函数) 向凸包面列表中添加一个新面。
38 */

```

```

39 void ins(int a, int b, int c) { face.push_back({a, b, c}); }
40
41 /**
42  * @brief (内部函数) 将点 v 加入到当前的凸包中。
43  * 算法会找到所有 v 能 "看见" 的面, 删除它们,
44  * 然后用 v 和 "视线" 边界上的边构成新面。
45  */
46 void add(int v) {
47     vector<Face> tmp;
48     stp++; // 更新时间戳
49     // 找出所有对点 v 可见的面 (volume < 0), 并标记其边
50     for (auto f : face) {
51         if (sgn(volume(v, f[0], f[1], f[2])) < 0) {
52             for (auto i : f)
53                 for (auto j : f) mark[i][j] = stp;
54         } else {
55             tmp.push_back(f);
56         }
57     }
58     face = tmp; // 更新面列表, 只保留 v 看不见的面
59     // 遍历留下来的面的边, 如果该边是可见与不可见的分界线, 则与 v 构成新面
60     for (int i = 0; i < (int)tmp.size(); i++) {
61         int a = tmp[i][0], b = tmp[i][1], c = tmp[i][2];
62         if (mark[a][b] == stp) ins(b, a, v);
63         if (mark[b][c] == stp) ins(c, b, v);
64         if (mark[c][a] == stp) ins(a, c, v);
65     }
66 }
67
68 const ld eps = 1e-9;
69
70 /**
71  * @brief 判断浮点数 x 的符号
72  * @param x 输入的浮点数
73  * @return -1 表示 x 为负数
74  * 0 表示 x 在精度范围内可视为零
75  * 1 表示 x 为正数
76  */
77 int sgn(ld x) {
78     if (std::abs(x) < eps) {
79         return 0;
80     }
81     return x < 0 ? -1 : 1;
82 }
83
84 /**
85  * @brief (内部函数) 寻找初始的 4 个不共面的点构成一个四面体, 作为凸包的基础。
86  * @return 找到则返回 true, 否则 (所有点共面或共线) 返回 false。
87  */
88 bool Find(int n) {
89     // 找到 3 个不共线的点
90     for (int i = 2; i < n; i++) {
91         p3 ndir = cross(p[0] - p[i], p[1] - p[i]);
92         if (ndir == p3(0, 0, 0)) continue;
93         swap(p[i], p[2]);
94         // 找到第 4 个不共面的点
95         for (int j = i + 1; j < n; j++) {
96             if (sgn(volume(0, 1, 2, j)) != 0) {
97                 swap(p[j], p[3]);
98                 // 添加初始四面体的两个面 (内外侧), 保证初始方向正确

```



```

99         ins(0, 1, 2);
100        ins(0, 2, 1);
101        return true;
102    }
103    }
104    }
105    return false;
106}
107
108mt19937 rng; // 随机数生成器, 用于打乱点集
109
110/**
111 * @brief 主函数, 计算点集 p 的三维凸包。
112 * @return 如果成功构建凸包则返回 true, 否则返回 false。
113 */
114bool solve() {
115    face.clear();
116    int n = (int)p.size();
117    // 随机打乱点集, 这是增量法期望复杂度为  $O(n \log n)$  的关键
118    shuffle(p.begin(), p.end(), rng);
119    // 找到初始四面体
120    if (!Find(n)) return false;
121    // 逐个将剩余的点加入凸包
122    for (int i = 3; i < n; i++) add(i);
123    return true;
124}

```

1.11 相关技巧

1.11.1 点集的最小最大三角形

```

1 pair<T, T> minmax_triangle(const vector<Point> &vec) {
2     if (vec.size() <= 2) return {0, 0};
3     vector<pair<int, int>> evt;
4     evt.reserve(vec.size() * vec.size());
5     T maxans = 0, minans = INF;
6     for (size_t i = 0; i < vec.size(); i++) {
7         for (size_t j = 0; j < vec.size(); j++) {
8             if (i == j) continue;
9             if (vec[i] == vec[j])
10                minans = 0;
11            else
12                evt.push_back({i, j});
13        }
14    }
15    sort(evt.begin(), evt.end(),
16         [&](const pair<int, int> &u, const pair<int, int> &v) {
17             const Point du = vec[u.second] - vec[u.first],
18                 dv = vec[v.second] - vec[v.first];
19             return Argcmp()({du.y, -du.x}, {dv.y, -dv.x});
20         });
21    vector<size_t> vx(vec.size()), pos(vec.size());
22    for (size_t i = 0; i < vec.size(); i++) vx[i] = i;
23    sort(vx.begin(), vx.end(), [&](int x, int y) { return vec[x] < vec[y]; });
24    for (size_t i = 0; i < vx.size(); i++) pos[vx[i]] = i;
25    for (auto [u, v] : evt) {
26        const size_t i = pos[u], j = pos[v];
27        const size_t l = min(i, j), r = max(i, j);
28        const Point vecu = vec[u], vecv = vec[v];

```

```

29     if (l > 0)
30         minans = min(
31             minans, abs((vec[vx[l - 1]] - vecu) ^ (vec[vx[l - 1]] - vecv)));
32     if (r < vx.size() - 1)
33         minans = min(
34             minans, abs((vec[vx[r + 1]] - vecu) ^ (vec[vx[r + 1]] - vecv)));
35     maxans = max({maxans, abs((vec[vx[0]] - vecu) ^ (vec[vx[0]] - vecv)),
36                 abs((vec[vx.back()] - vecu) ^ (vec[vx.back()] - vecv))});
37     if (i < j) swap(vx[i], vx[j]), pos[u] = j, pos[v] = i;
38 }
39 return {minans, maxans};
40 }

```

1.11.2 平面最近点对

```

1 // 平面最近点对
2 // 扫描线, 复杂度 O(nlogn)
3 T closest_pair(vector<Point> points) {
4     sort(points.begin(), points.end());
5     const auto cmpy = [](const Point &a, const Point &b) {
6         if (abs(a.y - b.y) <= eps) return a.x < b.x - eps;
7         return a.y < b.y - eps;
8     };
9     multiset<Point, decltype(cmpy)> s{cmpy};
10    T ans = INF;
11    for (size_t i = 0, l = 0; i < points.size(); i++) {
12        const T sqans = sqrtl(ans) + 1;
13        while (l < i && points[i].x - points[l].x >= sqans)
14            s.erase(s.find(points[l++]));
15        for (auto it = s.lower_bound(Point{-INF, points[i].y - sqans});
16             it != s.end() && it->y - points[i].y <= sqans; it++) {
17            ans = min(ans, points[i].dis2(*it));
18        }
19        s.insert(points[i]);
20    }
21    return ans;
22 }

```

1.11.3 判断多条线段是否有交点

```

1 // 判断多条线段是否有交点
2 // 扫描线, 复杂度 O(nlogn)
3 bool segs_inter(const vector<Segment> &segs) {
4     if (segs.empty()) return false;
5     using seq_t = tuple<T, int, Segment>; // x 坐标 出入点 线段
6     const auto seqcmp = [](const seq_t &u, const seq_t &v) {
7         const auto [u0, u1, u2] = u;
8         const auto [v0, v1, v2] = v;
9         if (abs(u0 - v0) <= eps) return make_pair(u1, u2) < make_pair(v1, v2);
10        return u0 < v0 - eps;
11    };
12    vector<seq_t> seq;
13    for (auto seg : segs) {
14        if (seg.a.x > seg.b.x + eps) swap(seg.a, seg.b);
15        seq.push_back({seg.a.x, 0, seg});
16        seq.push_back({seg.b.x, 1, seg});
17    }
18    sort(seq.begin(), seq.end(), seqcmp);
19    T x_now;

```

```

20     auto cmp = [&](const Segment &u, const Segment &v) {
21         if (abs(u.a.x - u.b.x) <= eps || abs(v.a.x - v.b.x) <= eps)
22             return u.a.y < v.a.y - eps;
23         return ((x_now - u.a.x) * (u.b.y - u.a.y) + u.a.y * (u.b.x - u.a.x)) *
24             (v.b.x - v.a.x) <
25             ((x_now - v.a.x) * (v.b.y - v.a.y) + v.a.y * (v.b.x - v.a.x)) *
26             (u.b.x - u.a.x) -
27             eps;
28     };
29     multiset<Segment, decltype(cmp)> s{cmp};
30     for (const auto [x, o, seg] : seq) {
31         x_now = x;
32         const auto it = s.lower_bound(seg);
33         if (o == 0) {
34             if (it != s.end() && seg.is_inter(*it)) return true;
35             if (it != s.begin() && seg.is_inter(*prev(it))) return true;
36             s.insert(seg);
37         } else {
38             if (next(it) != s.end() && it != s.begin() &&
39                 (*prev(it)).is_inter(*next(it)))
40                 return true;
41             s.erase(it);
42         }
43     }
44     return false;
45 }

```

1.11.4 多边形面积并

```

1 // 多边形面积并
2 // 轮廓积分, 复杂度  $O(n^2 \log n)$ ,  $n$  为边数
3 // ans[i] 表示被至少覆盖了 i+1 次的区域的面积
4 vector<T> area_union(const vector<Polygon> &polys) {
5     const size_t siz = polys.size();
6     vector<vector<pair<Point, Point>>> segs(siz);
7     const auto check = [&](const Point &u, const Segment &e) {
8         return !((u < e.a && u < e.b) || (u > e.a && u > e.b));
9     };
10
11     auto cut_edge = [&](const Segment &e, const size_t i) {
12         const Line le{e.a, e.b - e.a};
13         vector<pair<Point, int>> evt;
14         evt.push_back({e.a, 0});
15         evt.push_back({e.b, 0});
16         for (size_t j = 0; j < polys.size(); j++) {
17             if (i == j) continue;
18             const auto &pj = polys[j];
19             for (size_t k = 0; k < pj.p.size(); k++) {
20                 const Segment s = {pj.p[k], pj.p[pj.nxt(k)]};
21                 if (le.toleft(s.a) == 0 && le.toleft(s.b) == 0) {
22                     evt.push_back({s.a, 0});
23                     evt.push_back({s.b, 0});
24                 } else if (s.is_inter(le)) {
25                     const Line ls{s.a, s.b - s.a};
26                     const Point u = le.inter(ls);
27                     if (le.toleft(s.a) < 0 && le.toleft(s.b) >= 0)
28                         evt.push_back({u, -1});
29                     else if (le.toleft(s.a) >= 0 && le.toleft(s.b) < 0)
30                         evt.push_back({u, 1});

```

```

31     }
32 }
33 }
34 sort(evt.begin(), evt.end());
35 if (e.a > e.b) reverse(evt.begin(), evt.end());
36 int sum = 0;
37 for (size_t i = 0; i < evt.size(); i++) {
38     sum += evt[i].second;
39     const Point u = evt[i].first, v = evt[i + 1].first;
40     if (!(u == v) && check(u, e) && check(v, e))
41         segs[sum].push_back({u, v});
42     if (v == e.b) break;
43 }
44 };
45
46 for (size_t i = 0; i < polys.size(); i++) {
47     const auto &pi = polys[i];
48     for (size_t k = 0; k < pi.p.size(); k++) {
49         const Segment ei = {pi.p[k], pi.p[pi.nxt(k)]};
50         cut_edge(ei, i);
51     }
52 }
53 vector<T> ans(siz);
54 for (size_t i = 0; i < siz; i++) {
55     T sum = 0;
56     sort(segs[i].begin(), segs[i].end());
57     int cnt = 0;
58     for (size_t j = 0; j < segs[i].size(); j++) {
59         if (j > 0 && segs[i][j] == segs[i][j - 1])
60             segs[i + (++cnt)].push_back(segs[i][j]);
61         else
62             cnt = 0, sum += segs[i][j].first ^ segs[i][j].second;
63     }
64     ans[i] = sum / 2;
65 }
66 return ans;
67 }

```

1.11.5 点、线段在简单多边形内

```

1 bool point_in_polygon (cp u, const vector <point> & p) {
2     | int n = (int) p.size (), cnt = 0;
3     | for (int i = 0; i < n; ++i) {
4         | point a = p[i], b = p[(i + 1) % n];
5         | if (pos (u, {a, b})) return true;
6         | int x = turn (a, u, b);
7         | int y = sgn (a.y - u.y);
8         | int z = sgn (b.y - u.y);
9         | if (x > 0 && y <= 0 && z > 0) ++cnt;
10        | if (x < 0 && z <= 0 && y > 0) --cnt; }
11    | return cnt != 0; } // < 0 在逆时针多边形内; > 0 顺时针
12 bool in_polygon (cp u, cp v) {
13     | // u, v in polygon; p contain those u, v on border
14     | for (int i = 0; i < n; i++) {
15         | int j = (i + 1) % n, k = (i + 2) % n;
16         | cp ii = p[i], jj = p[j], kk = p[k];
17         | if (inter_judge_strict({u, v}, {ii, jj})) return 0;
18         | if (point_on_segment (jj, {u, v})) {
19             | bool good = true, left = turn (ii, jj, kk) >= 0;

```

```

20 |         |         | for (auto x : {u, v})
21 |         |         |     if (left)
22 |         |         |         | good &= turn(ii, jj, x) >= 0
23 |         |         |         |         && turn(jj, kk, x) >= 0;
24 |         |         |     else
25 |         |         |         | good &= !(turn(jj, x, kk) > 0
26 |         |         |         |         && turn(jj, ii, x) > 0);
27 |         |         |     if (!good) return 0;
28 |         |     } } return 1; }
29 LD get_far (int uid, int vid) {
30 |     // u -> v in polygon, check the ray u -> polygon
31 |     cp u = p[uid], v = p[vid];
32 |     LD far = 1e9;
33 |     for (int i = 0; i < n; i++) {
34 |         | int j = (i + 1) % n, k = (i + 2) % n;
35 |         | cp ii = p[i], jj = p[j], kk = p[k];
36 |         | if (two_side(ii, jj, {u, v})) {
37 |             | LD s1 = det(jj - ii, u - ii);
38 |             | LD s2 = det(jj - ii, v - ii);
39 |             | if (sgn(s1 - s2) && sgn(s1) != sgn(s2 - s1))
40 |             |         | far = min(far,
41 |             |             |         |         dis(u, line_inter({ii,jj},{u,v})));
42 |             | if (j != uid && point_on_ray(jj, {u, v})) {
43 |                 | bool good = turn(ii, jj, kk) <= 0;
44 |                 | for (auto x : {u - (jj - u), jj + (jj - u)})
45 |                 |         | good &= !(turn(ii, jj, x) > 0
46 |                 |         |         && turn(jj, kk, x) > 0);
47 |                 | if (!good) far = min(far, dis(u, jj));
48 |             | } } return far; }
49 void work() {
50 |     for (int i = 0; i < n; i++)
51 |         | for (int j = 0; j < n; j++) if (i != j) {
52 |             | if (!in_polygon(p[i], p[j])) continue;
53 |             | LD ret = get_far(i,j)+get_far(j,i)-dis(p[i],p[j]);
54 |             | ans = max(ans, ret); } }

```

1.11.6 最小覆盖球

```

1 vector<p3> b; Circle calc() {
2     if(b.empty()) { return Circle(p3(0, 0, 0), 0);
3 }else if(1 == b.size()) {return Circle(b[0], 0);
4 }else if(2 == b.size()) {
5     return Circle((b[0] + b[1]) / 2, (b[0] - b[1]).len() / 2);
6 }else if(3 == b.size()) {
7     LD r = dis(b[0], b[1]) * dis(b[1], b[2]) * dis(b[2], b[0]) / 2 / cross(b[0] -
8         ↪ b[2], b[1] - b[2]).len();
9     return Circle(intersect({b[1] - b[0], (b[1] + b[0]) / 2}, {b[2] - b[1], (b[2] +
10         ↪ b[1]) / 2}, {cross(b[1] - b[0], b[2] - b[0]), b[0]}), r);
11 }else { p3 o(intersect({b[1] - b[0], (b[1] + b[0]) / 2}, {b[2] - b[0], (b[2] +
12     ↪ b[0]) / 2}, {b[3] - b[0], (b[3] + b[0]) / 2})); return Circle(o, (o -
13     ↪ b[0]).len()); } }
14 Circle miniBall(int n) {
15     Circle res(calc());
16     for(int i = 0; i < n; i++) if(!in_circle(a[i], res)) {
17         b.push_back(a[i]); res = miniBall(i); b.pop_back();
18         if (i) { p3 tmp = a[i]; memmove(a + 1, a, sizeof(p3) * i); a[0] = tmp; } }
19     return res; }

```

1.11.7 圆上整点

```

1 vector <LL> solve(LL r) {
2     | vector <LL> ret; // non-negative Y pos
3     | ret.push_back(0);
4     | LL l = 2 * r, s = sqrt(l);
5     | for (LL d=1; d<=s; d++) if (l%d==0) {
6     |     | LL lim=LL(sqrt(l/(2*d)));
7     |     | for (LL a = 1; a <= lim; a++) {
8     |     |     | LL b = sqrt(l/d-a*a);
9     |     |     | if (a*a+b*b==l/d && __gcd(a,b)==1 && a!=b)
10    |     |     | ret.push_back(d*a*b);
11    |     |     | } if (d*d==l) break;
12    |     |     | lim = sqrt(d/2);
13    |     |     | for (LL a=1; a<=lim; a++) {
14    |     |     |     | LL b = sqrt(d - a * a);
15    |     |     |     | if (a*a+b*b==d && __gcd(a,b)==1 && a!=b)
16    |     |     |     | ret.push_back(l/d*a*b);
17    |     | } } return ret; }

```

1.11.8 三相之力

```

1 // 内心
2 point incenter(point a, point b, point c) {
3     double p = dis(a, b) + dis(b, c) + dis(c, a);
4     return (a * dis(b, c) + b * dis(c, a) + c * dis(a, b)) / p;
5 }
6 // 外心
7 point circumcenter(point a, point b, point c) {
8     point p = b - a, q = c - a, s(dot(p, p) / 2, dot(q, q) / 2);
9     double d = det(p, q);
10    return a + point(det(s, {p.y, q.y}), det({p.x, q.x}, s)) / d;
11 }
12 // 垂心
13 point orthocenter(point a, point b, point c) {
14     return a + b + c - circumcenter(a, b, c) * 2.0;
15 }
16 // 费马点
17 point fermat_point(point a, point b, point c) {
18     if (a == b) return a;
19     if (b == c) return b;
20     if (c == a) return c;
21     double ab = dis(a, b), bc = dis(b, c), ca = dis(c, a);
22     double cosa = dot(b - a, c - a) / ab / ca;
23     double cosb = dot(a - b, c - b) / ab / bc;
24     double cosc = dot(b - c, a - c) / ca / bc;
25     double sq3 = PI / 3.0;
26     point mid;
27     if (sgn(cosa + 0.5) < 0)
28         mid = a;
29     else if (sgn(cosb + 0.5) < 0)
30         mid = b;
31     else if (sgn(cosc + 0.5) < 0)
32         mid = c;
33     else if (sgn(det(b - a, c - a)) < 0)
34         mid = line_inter({a, b + (c - b).rot(sq3)}, {b, c + (a - c).rot(sq3)});
35     else
36         mid = line_inter({a, c + (b - c).rot(sq3)}, {c, b + (a - b).rot(sq3)});
37     return mid;

```

```
38 } // minimize(|A-x|+|B-x|+|C-x|)
```

1.12 相关公式

1.12.1 Heron's Formula

$$p = \frac{a+b+c}{2}, a \geq b \geq c,$$

$$S = \sqrt{p(p-a)(p-b)(p-c)},$$

$$S = \frac{1}{2} \sqrt{a^2c^2 - \left(\frac{a^2+c^2-b^2}{2}\right)^2}.$$

1.12.2 四面体内接球球心

假设 s_i 是第 i 个顶点相对面的面积, 则有

$$\begin{cases} x = \frac{s_1x_1 + s_2x_2 + s_3x_3 + s_4x_4}{s_1 + s_2 + s_3 + s_4} \\ y = \frac{s_1y_1 + s_2y_2 + s_3y_3 + s_4y_4}{s_1 + s_2 + s_3 + s_4} \\ z = \frac{s_1z_1 + s_2z_2 + s_3z_3 + s_4z_4}{s_1 + s_2 + s_3 + s_4} \end{cases}$$

体积可以使用 $1/6$ 混合积求, 内接球半径为

$$r = \frac{3V}{s_1 + s_2 + s_3 + s_4}$$

1.12.3 三角形内心

$$\vec{I} = \frac{a\vec{A} + b\vec{B} + c\vec{C}}{a + b + c}$$

1.12.4 三角形外心

$$\vec{O} = \frac{\vec{A} + \vec{B} - \frac{\vec{BC} \cdot \vec{CA}}{\vec{AB} \times \vec{BC}} \vec{AB}^T}{2}$$

1.12.5 三角形垂心

$$\vec{H} = 3\vec{G} - 2\vec{O}$$

1.12.6 三角形偏心

$$\frac{-a\vec{A} + b\vec{B} + c\vec{C}}{-a + b + c}$$

内角的平分线和对边的两个外角平分线交点, 外切圆圆心. 剩余两点的同理.

1.12.7 三角形内接外接圆半径

$$r = \frac{2S}{a + b + c}, R = \frac{abc}{4S}$$

1.12.8 Pick's Theorem 格点多边形面积

$S = I + \frac{B}{2} - 1$. I 内部点, B 边界点.

1.12.9 Euler's Formula 多面体与平面图的点、边、面

Convex polyhedron: $V - E + F = 2$.

Planar graph: $|F| = |E| - |V| + n + 1$, n : #(connected components).

1.12.10 三角公式

$$\begin{aligned}\sin(a) + \sin(b) &= 2 \sin\left(\frac{a+b}{2}\right) \cos\left(\frac{a-b}{2}\right) \\ \sin(a) - \sin(b) &= 2 \cos\left(\frac{a+b}{2}\right) \sin\left(\frac{a-b}{2}\right) \\ \cos(a) + \cos(b) &= 2 \cos\left(\frac{a+b}{2}\right) \cos\left(\frac{a-b}{2}\right) \\ \cos(a) - \cos(b) &= -2 \sin\left(\frac{a+b}{2}\right) \sin\left(\frac{a-b}{2}\right) \\ \sin(a \pm b) &= \sin a \cos b \pm \cos a \sin b \\ \cos(a \pm b) &= \cos a \cos b \mp \sin a \sin b \\ \tan(a \pm b) &= \frac{\tan(a) \pm \tan(b)}{1 \mp \tan(a) \tan(b)} \\ \tan(a) \pm \tan(b) &= \frac{\sin(a \pm b)}{\cos(a) \cos(b)} \\ \sin(na) &= n \cos^{n-1} a \sin a - \binom{n}{3} \cos^{n-3} a \sin^3 a + \binom{n}{5} \cos^{n-5} a \sin^5 a - \dots \\ \cos(na) &= \cos^n a - \binom{n}{2} \cos^{n-2} a \sin^2 a + \binom{n}{4} \cos^{n-4} a \sin^4 a - \dots\end{aligned}$$

1.12.11 超球坐标系

$$\begin{aligned}x_1 &= r \cos(\phi_1) \\ x_2 &= r \sin(\phi_1) \cos(\phi_2) \\ &\vdots \\ x_{n-1} &= r \sin(\phi_1) \cdots \sin(\phi_{n-2}) \cos(\phi_{n-1}) \\ x_n &= r \sin(\phi_1) \cdots \sin(\phi_{n-2}) \sin(\phi_{n-1}) \\ \phi_{n-1} &\in [0, 2\pi] \\ \forall i = 1..n-1 \phi_i &\in [0, \pi]\end{aligned}$$

1.12.12 三维旋转公式

绕着 $(0, 0, 0) - (ux, uy, uz)$ 旋转 θ , (ux, uy, uz) 是单位向量. $\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = R \begin{bmatrix} x \\ y \\ z \end{bmatrix}$

$$R = \begin{pmatrix} \cos \theta + u_x^2(1 - \cos \theta) & u_x u_y(1 - \cos \theta) - u_z \sin \theta & u_x u_z(1 - \cos \theta) + u_y \sin \theta \\ u_y u_x(1 - \cos \theta) + u_z \sin \theta & \cos \theta + u_y^2(1 - \cos \theta) & u_y u_z(1 - \cos \theta) - u_x \sin \theta \\ u_z u_x(1 - \cos \theta) - u_y \sin \theta & u_z u_y(1 - \cos \theta) + u_x \sin \theta & \cos \theta + u_z^2(1 - \cos \theta) \end{pmatrix}.$$

1.12.13 立体角公式

$$\begin{aligned}\phi &: \text{二面角} \\ \Omega &= (\phi_{ab} + \phi_{bc} + \phi_{ac}) \text{ rad} - \pi \text{ sr}\end{aligned}$$

$$\tan\left(\frac{1}{2}\Omega/\text{rad}\right) = \frac{|\vec{a} \vec{b} \vec{c}|}{abc + (\vec{a} \cdot \vec{b})c + (\vec{a} \cdot \vec{c})b + (\vec{b} \cdot \vec{c})a}$$

$$\theta_s = \frac{\theta_a + \theta_b + \theta_c}{2}$$

1.12.14 常用体积公式

- 棱锥 Pyramid $V = \frac{1}{3}Sh$.
- 球 Sphere $V = \frac{4}{3}\pi R^3$.
- 棱台 Frustum
 $V = \frac{1}{3}h(S_1 + \sqrt{S_1 S_2} + S_2)$.
- 椭球 Ellipsoid $V = \frac{4}{3}\pi abc$.

1.12.15 扇形与圆弧重心

扇形重心与圆心距离为 $\frac{4r \sin(\theta/2)}{3\theta}$,
圆弧重心与圆心距离为 $\frac{4r \sin^3(\theta/2)}{3(\theta - \sin(\theta))}$.

1.12.16 高维球体积

$$V_2 = \pi R^2, S_2 = 2\pi R$$

$$V_3 = \frac{4}{3}\pi R^3, S_3 = 4\pi R^2$$

$$V_4 = \frac{1}{2}\pi^2 R^4, S_4 = 2\pi^2 R^3$$

$$\text{Generally, } V_n = \frac{2\pi}{n} V_{n-2}, S_{n-1} = \frac{2\pi}{n-2} S_{n-3}$$

$$\text{Where, } S_0 = 2, V_1 = 2, S_1 = 2\pi, V_2 = \pi$$

2. Tree & Graph

2.1 树的重心

定义

- **定义一 (按子树大小):** 找到一个点, 删去这个点后剩下的所有子树的节点数都不超过总结点数的一半 ($\lfloor n/2 \rfloor$), 那么这个点就是树的重心。
- **定义二 (按最大子树):** 找到一个点, 其所有的子树中最大的子树节点数最少, 那么这个点就是树的重心。

性质

- **数量:** 一棵树最少有一个重心, 最多只有两个重心。当且仅当存在一条边, 断开后形成两个大小均为 $n/2$ 的子树时, 这条边连接的两个节点都是重心。
- **距离和最小:** 树中所有点到重心的距离之和是最小的。如果有两个重心, 那么它们的距离和相等。
- **路径性:** 把两棵树通过一条边相连得到一棵新的树, 那么新树的重心在连接原来两棵树的重心的路径上。
- **动态变化:** 在一棵树上添加或删除一个叶子, 它的重心最多只移动一条边的距离。

2.2 树的直径

定义 树上的最长简单路径。(以下性质基于边权为正)

性质

- **中心性:** 如果一棵树有多条直径, 那么它们必然相交于一点 (这个点可能是某条边的中点)。
- **最远点:** 对于树上任意一点, 距离其最远的点一定是某条直径的端点之一。
- **可合并性 (重要):** 对于树上的两个点集 S 和 T , 若 S 中最远点对是 (x, y) , T 中最远点对是 (u, v) , 那么 $S \cup T$ 中的最远点对一定是 $\{x, y, u, v\}$ 这四个点中某两点组成的点对。

2.3 树链剖分

```
1  /*
2  加完边记得调用 work() 初始化
3  */
4  struct HLD {
5      int n;
6      std::vector<int> siz, top, dep, parent, in, out, seq;
7      std::vector<std::vector<int>> adj;
8      int cur;
9
10     HLD() {}
11     HLD(int n) { init(n); }
12     void init(int n) {
13         this->n = n;
14         siz.resize(n);
15         top.resize(n);
16         dep.resize(n);
17         parent.resize(n);
18         in.resize(n);
19         out.resize(n);
```

```
20     seq.resize(n);
21     cur = 0;
22     adj.assign(n, {});
23 }
24 void addEdge(int u, int v) {
25     adj[u].push_back(v);
26     adj[v].push_back(u);
27 }
28 void work(int root = 0) {
29     top[root] = root;
30     dep[root] = 0;
31     parent[root] = -1;
32     dfs1(root);
33     dfs2(root);
34 }
35 void dfs1(int u) {
36     if (parent[u] != -1) {
37         adj[u].erase(std::find(adj[u].begin(), adj[u].end(), parent[u]));
38     }
39
40     siz[u] = 1;
41     for (auto &v : adj[u]) {
42         parent[v] = u;
43         dep[v] = dep[u] + 1;
44         dfs1(v);
45         siz[u] += siz[v];
46         if (siz[v] > siz[adj[u][0]]) {
47             std::swap(v, adj[u][0]);
48         }
49     }
50 }
51 void dfs2(int u) {
52     in[u] = cur++;
53     seq[in[u]] = u;
54     for (auto v : adj[u]) {
55         top[v] = v == adj[u][0] ? top[u] : v;
56         dfs2(v);
57     }
58     out[u] = cur;
59 }
60 int lca(int u, int v) {
61     while (top[u] != top[v]) {
62         if (dep[top[u]] > dep[top[v]]) {
63             u = parent[top[u]];
64         } else {
65             v = parent[top[v]];
66         }
67     }
68     return dep[u] < dep[v] ? u : v;
69 }
70
71 int dist(int u, int v) { return dep[u] + dep[v] - 2 * dep[lca(u, v)]; }
72
73 int jump(int u, int k) {
74     if (dep[u] < k) {
75         return -1;
76     }
77
78     int d = dep[u] - k;
```

```

80     while (dep[top[u]] > d) {
81         u = parent[top[u]];
82     }
83
84     return seq[in[u] - dep[u] + d];
85 }
86
87 bool isAncestor(int u, int v) { return in[u] <= in[v] && in[v] < out[u]; }
88
89 int rootedParent(int u, int v) {
90     std::swap(u, v);
91     if (u == v) {
92         return u;
93     }
94     if (!isAncestor(u, v)) {
95         return parent[u];
96     }
97     auto it =
98         std::upper_bound(adj[u].begin(), adj[u].end(), v,
99             [&](int x, int y) { return in[x] < in[y]; }) -
100         1;
101     return *it;
102 }
103
104 int rootedSize(int u, int v) {
105     if (u == v) {
106         return n;
107     }
108     if (!isAncestor(v, u)) {
109         return siz[v];
110     }
111     return n - siz[rootedParent(u, v)];
112 }
113
114 int rootedLca(int a, int b, int c) {
115     return lca(a, b) ^ lca(b, c) ^ lca(c, a);
116 }
117
118 int intersection(int x, int y, int u, int v) {
119     vector<int> t = {lca(x, u), lca(x, v), lca(y, u), lca(y, v)};
120     sort(t.begin(), t.end());
121     int l = lca(x, y), r = lca(u, v);
122     if (dep[l] > dep[r]) swap(l, r);
123     if (dep[t[0]] < dep[l] || dep[t[2]] < dep[r]) {
124         return 0;
125     }
126     return 1 + dist(t[2], t[3]);
127 }
128 };

```

2.4 prufer 与树的互相转化

```

1 vector<int> edges_to_prufer(const vector<pair<int, int>>& eg) // [1, n], 定根为 n
2 {
3     int n = eg.size() + 1, i, j, k;
4     vector<int> fir(n + 1), nxt(2 * n + 1), e(2 * n + 1), rd(n + 1);
5     int cnt = 0;
6     for (auto [u, v] : eg) {
7         e[++cnt] = v;

```

```

8     nxt[cnt] = fir[u];
9     fir[u] = cnt;
10    ++rd[v];
11    e[++cnt] = u;
12    nxt[cnt] = fir[v];
13    fir[v] = cnt;
14    ++rd[u];
15    }
16    for (i = 1; i <= n; i++)
17        if (rd[i] == 1) break;
18    int u = i;
19    vector<int> r;
20    for (j = 1; j < n - 1; j++) {
21        for (k = fir[u], u = rd[u] = 0; k; k = nxt[k])
22            if (rd[e[k]]) {
23                r.push_back(e[k]);
24                if ((--rd[e[k]] == 1) && (e[k] < i)) u = e[k];
25            }
26        if (!u) {
27            while (rd[i] != 1) ++i;
28            u = i;
29        }
30    }
31    return r;
32 }
33 vector<pair<int, int>> prufer_to_edges(const vector<int>& p) // [1, n], 定根为 n
34 {
35     int n = p.size(), i, j, k;
36     int m = n + 3;
37     vector<int> cs(m);
38     for (i = 0; i < n; i++) ++cs[p[i]];
39     i = 0;
40     while (cs[++i]);
41     int u = i, v;
42     vector<pair<int, int>> r;
43     for (j = 0; j < n; j++) {
44         cs[u] = 1e9;
45         r.push_back({u, v = p[j]});
46         if ((--cs[v] == 0) && (v < i)) u = v;
47         if (v != u) {
48             while (cs[i] > 0) ++i;
49             u = i;
50         }
51     }
52     r.push_back({u, n + 2});
53     return r;
54 }

```

2.5 树哈希

核心理想 将树的拓扑结构映射成一个数。

- 有根树比较好用的 hash 公式是 $f(p) = \sum_{s \in \text{son}(p)} \text{hash}(f(s)) + C$ 。
- 其中 hash 函数可以是随机数或者多项式。C 是常数，取 1 就行。

- 这样的 hash 公式利于换根，换根公式也容易推导：

$$g(p) = \begin{cases} f(p) & p \text{ is root} \\ f(p) + \text{hash}(g(\text{fa}(p)) - \text{hash}(f(p))) & \text{others} \end{cases}$$

- 有根树的 hash 可以用根节点的 hash 值。
- 无根树的 hash 可以用所有有点作为根节点的 hash 值的异或和作为 hash 值。

复杂度分析

- 随机数 hash 使用了 map，复杂度为 $O(n \log n)$ 。

注意事项

- 防止被卡，可以更改 hash 函数，常数 C ，取模底数（如模 998244353），此时无根树 hash 可以改为所有点 hash 值的加和。
- 使用前记得调用 `calc(int root = 0)`，否则会 `assert`。

```

1 using ull = unsigned long long;
2
3 /**
4  * @brief 树哈希类
5  *
6  * 用于计算有根树和无根树的哈希值。
7  * 核心思想是通过 DFS 和换根 DP
8  * 来为每个节点的子树以及以每个节点为根的整棵树生成一个唯一的哈希值。
9  */
10 class tree_hash {
11 private:
12     // 静态成员，用于在类的所有实例中共享
13     static map<ull, ull> mp; // 哈希值映射，确保相同的子树结构得到相同的随机值
14     static mt19937_64 rng;   // 64 位梅森旋转算法随机数生成器
15
16     int n; // 树的节点数
17     vector<vector<int>>> g; // 邻接表表示的树
18     vector<ull> dp; // dp[i] 存储以 i 为根的子树的哈希值
19     vector<ull> rdp; // rdp[i] 存储以 i 为整棵树的根时的哈希值
20     ull hash_val; // 整棵树的无根哈希值
21     bool calced; // 标记是否已执行计算
22
23 public:
24     /**
25     * @brief 构造函数
26     * @param ng 描述树结构的邻接表
27     */
28     tree_hash(const vector<vector<int>>>& ng) {
29         n = ng.size();
30         g = ng;
31         dp.resize(n, 0);
32         rdp.resize(n, 0);
33         hash_val = 0;
34         calced = false;
35     }
36
37 private:

```

```
38  /**
39   * @brief 为一个子树哈希值生成一个唯一的随机映射值
40   *
41   * 使用 map 来记忆化，确保对于相同的输入 x，总是返回相同的随机数。
42   * 这是为了保证同构的子树贡献的哈希值是相同的。
43   * @param x 子树的哈希值
44   * @return 映射后的唯一哈希值
45   */
46  static ull get(ull x) {
47      if (mp.count(x)) return mp[x];
48      return mp[x] = rng();
49  }
50
51  /**
52   * @brief 第一次 DFS，计算每个节点的子树哈希值
53   * @param p 当前节点
54   * @param fa 父节点
55   */
56  void dfs(int p, int fa) {
57      dp[p] = 1; // 初始值，可以看作是节点自身的贡献
58      for (auto s : g[p]) {
59          if (s != fa) {
60              dfs(s, p);
61              // 将所有子节点的子树哈希值通过 get() 映射后累加
62              dp[p] += get(dp[s]);
63          }
64      }
65  }
66
67  /**
68   * @brief 第二次 DFS（换根 DP），计算以每个节点为根时整棵树的哈希值
69   * @param p 当前节点
70   * @param fa 父节点
71   */
72  void rdfs(int p, int fa) {
73      if (fa == -1) {
74          // 如果是根节点，其换根哈希值就是它的子树哈希值
75          rdp[p] = dp[p];
76      } else {
77          // 否则，它的换根哈希值 = 它的子树哈希值 + 从父节点那边传来的哈希值
78          // 父节点传来的值 = (父节点的换根哈希值 - 当前子树对父节点的贡献)
79          ull parent_contribution = rdp[fa] - get(dp[p]);
80          rdp[p] = dp[p] + get(parent_contribution);
81      }
82
83      for (auto s : g[p]) {
84          if (s != fa) {
85              rdfs(s, p);
86          }
87      }
88  }
89
90  public:
91  /**
92   * @brief 执行计算
93   * @param root 任意指定的根节点，默认为 0
94   */
95  void calc(int root = 0) {
96      dfs(root, -1);
97      rdfs(root, -1);
```

```

98
99 // 计算无根树哈希值，通常用所有节点的有根哈希值的异或和
100 // 也可以用和，但异或可以避免顺序问题
101 hash_val = 0;
102 for (int i = 0; i < n; i++) {
103     hash_val ^= rdp[i];
104 }
105 calced = true;
106 }
107
108 /**
109  * @brief 获取以节点 p 为根的有根树哈希值
110  * @param p 节点编号
111  * @return 哈希值
112  */
113 ull rooted_hash(int p) {
114     assert(calced && "calc() must be called before getting hash values.");
115     return rdp[p];
116 }
117
118 /**
119  * @brief 获取无根树的哈希值
120  * @return 哈希值
121  */
122 ull unrooted_hash() {
123     assert(calced && "calc() must be called before getting hash values.");
124     return hash_val;
125 }
126 };
127
128 // 初始化静态成员
129 mt19937_64 tree_hash::rng(
130     chrono::steady_clock::now().time_since_epoch().count());
131 map<ull, ull> tree_hash::mp;

```

2.6 内向基环树

```

1 // 返回 p 所在连通块的一个环上点
2 int check(int p) {
3     if (vis[p]) return p;
4     vis[p] = 1;
5     return check(ne[p]);
6 }
7
8 vector<int> cir;
9 int p = check(i);
10 int q = p;
11 while (1) {
12     cir.push_back(q);
13     if (ne[q] == p) break;
14     q = ne[q];
15 }

```

2.7 Hopcroft-Karp $O(\sqrt{VE})$

```

1 vector<int> E[N]; // 邻接表, E[u] 存储左侧顶点 u 连接的所有右侧顶点
2 vector<int> ml, mr; // ml[i] 存储左侧顶点 i 匹配的右侧顶点, mr[i] 存储右侧顶点
3 // i 匹配的左侧顶点。0 表示未匹配。

```



```

4 vector<int> a, p; // HK 算法内部使用的辅助数组, 无需关心具体含义
5
6 /**
7  * @brief 计算二分图的最大匹配 (Hopcroft-Karp 算法)
8  * @param nl 左侧顶点的数量 (编号从 1 到 nl)
9  * @param nr 右侧顶点的数量 (编号从 1 到 nr)
10 */
11 void match(int nl, int nr) {
12     ml.assign(nl + 1, 0);
13     mr.assign(nr + 1, 0);
14     while (true) {
15         bool ok = false;
16         a.assign(nl + 1, 0);
17         p.assign(nl + 1, 0);
18         static queue<int> q;
19         while (!q.empty()) q.pop(); // 清空队列
20         for (int i = 1; i <= nl; i++)
21             if (!ml[i]) a[i] = p[i] = i, q.push(i);
22         while (!q.empty()) {
23             int x = q.front();
24             q.pop();
25             if (ml[a[x]]) continue;
26             for (auto y : E[x]) {
27                 if (!mr[y]) {
28                     for (ok = 1; y; x = p[x]) mr[y] = x, swap(ml[x], y);
29                     break;
30                 } else if (!p[mr[y]])
31                     q.push(y = mr[y]), p[y] = x, a[y] = a[x];
32             }
33             if (ok) break; // 找到一条增广路后就跳出
34         }
35         if (!ok) break; // 找不到更多增广路, 算法结束
36     }
37 }

```

2.8 Hungarian $O(VE/w)$

```

1 using B = bitset<N>;
2 B edge[N]; // 邻接矩阵的 bitset 形式。edge[i][j] 为 1 表示左侧顶点 i 和右侧顶点
3             // j 之间有边
4 bool dfs(int x, B& unvis, vector<int>& match) {
5     for (B z = edge[x];;) {
6         z &= unvis;
7         int y = z._Find_first();
8         if (y == N) return 0;
9         unvis.reset(y);
10        if (!match[y] || dfs(match[y], unvis, match)) return match[y] = x, 1;
11    }
12 }
13 // 返回一个 vector, ret[i] 表示左侧顶点 i 匹配的右侧顶点。0 表示未匹配。
14 vector<int> match(int nl, int nr) {
15     B unvis;
16     unvis.set();
17     vector<int> match(nr + 1), ret(nl + 1);
18     for (int i = 1; i <= nl; ++i)
19         if (dfs(i, unvis, match)) unvis.set();
20     for (int i = 1; i <= nr; ++i) ret[match[i]] = i;
21     return ret[0] = 0, ret;

```

22 }

2.9 Shuffle 一般图最大匹配 $O(VE)$

```

1  const int N = 1e3 + 10;
2  // 带随机化的、基于增广路搜索的启发式算法
3  // 该做法能在洛谷通过 70 % 的测试点 疑似没用
4
5  int n, m, vis[N];
6  int mat[N]; // 匹配数组。mat[i] = j 表示顶点 i 和顶点 j 匹配。若 mat[i] = 0, 则
7              // i 未匹配。
8  vector<int> E[N];
9  bool dfs(int tim, int x) {
10     shuffle(E[x].begin(), E[x].end(), rng);
11     vis[x] = tim;
12     for (auto y : E[x]) {
13         int z = mat[y];
14         if (vis[z] == tim) continue;
15         mat[x] = y, mat[y] = x, mat[z] = 0;
16         if (!z || dfs(tim, z)) return true;
17         mat[x] = 0, mat[y] = z, mat[z] = y;
18     }
19     return false;
20 }
21 int main() { // 暗含二分图性质跑一次即可
22     cin >> n >> m;
23     while (m--) {
24         int u, v;
25         cin >> u >> v;
26         E[u].push_back(v);
27         E[v].push_back(u);
28     }
29
30     for (int _ = 0; _ < 10; _++) {
31         fill(vis + 1, vis + n + 1, 0);
32         for (int i = 1; i <= n; ++i)
33             if (!mat[i]) dfs(i, i);
34     }
35     int res = 0;
36     for (int i = 1; i <= n; i++) res += mat[i] > 0;
37     cout << res / 2 << "\n";
38     for (int i = 1; i <= n; i++) cout << mat[i] << " \n"[i == n];
39     return 0;
40 }

```

2.10 KM 最大权匹配 $O(V^3)$

```

1  /*
2  此模板要求边权为正。
3  传入的邻接矩阵编号从 0 开始。
4  如果想求完美匹配，需要给每条边加上足够大的偏移量 ( > n * 最大边权 便可)。
5  */
6  template <class T>
7  struct MaxAssignment {
8      public:
9          T solve(int nx, int ny, std::vector<std::vector<T>> a) {
10              assert(0 <= nx && nx <= ny);
11              assert(int(a.size()) == nx);

```

```

12     for (int i = 0; i < nx; ++i) {
13         assert(int(a[i].size()) == ny);
14         for (auto x : a[i]) assert(x >= 0);
15     }
16
17     auto update = [&](int x) {
18         for (int y = 0; y < ny; ++y) {
19             if (lx[x] + ly[y] - a[x][y] < slack[y]) {
20                 slack[y] = lx[x] + ly[y] - a[x][y];
21                 slackx[y] = x;
22             }
23         }
24     };
25
26     costs.resize(nx + 1);
27     costs[0] = 0;
28     lx.assign(nx, std::numeric_limits<T>::max());
29     ly.assign(ny, 0);
30     xy.assign(nx, -1);
31     yx.assign(ny, -1);
32     slackx.resize(ny);
33     for (int cur = 0; cur < nx; ++cur) {
34         std::queue<int> que;
35         visx.assign(nx, false);
36         visy.assign(ny, false);
37         slack.assign(ny, std::numeric_limits<T>::max());
38         p.assign(nx, -1);
39
40         for (int x = 0; x < nx; ++x) {
41             if (xy[x] == -1) {
42                 que.push(x);
43                 visx[x] = true;
44                 update(x);
45             }
46         }
47
48         int ex, ey;
49         bool found = false;
50         while (!found) {
51             while (!que.empty() && !found) {
52                 auto x = que.front();
53                 que.pop();
54                 for (int y = 0; y < ny; ++y) {
55                     if (a[x][y] == lx[x] + ly[y] && !visy[y]) {
56                         if (yx[y] == -1) {
57                             ex = x;
58                             ey = y;
59                             found = true;
60                             break;
61                         }
62                         que.push(yx[y]);
63                         p[yx[y]] = x;
64                         visy[y] = visx[yx[y]] = true;
65                         update(yx[y]);
66                     }
67                 }
68             }
69             if (found) break;
70
71             T delta = std::numeric_limits<T>::max();

```

```

72         for (int y = 0; y < ny; ++y)
73             if (!visy[y]) delta = std::min(delta, slack[y]);
74         for (int x = 0; x < nx; ++x)
75             if (visx[x]) lx[x] -= delta;
76         for (int y = 0; y < ny; ++y) {
77             if (visy[y]) {
78                 ly[y] += delta;
79             } else {
80                 slack[y] -= delta;
81             }
82         }
83         for (int y = 0; y < ny; ++y) {
84             if (!visy[y] && slack[y] == 0) {
85                 if (yx[y] == -1) {
86                     ex = slackx[y];
87                     ey = y;
88                     found = true;
89                     break;
90                 }
91                 que.push(yx[y]);
92                 p[yx[y]] = slackx[y];
93                 visy[y] = visx[yx[y]] = true;
94                 update(yx[y]);
95             }
96         }
97     }
98
99     costs[cur + 1] = costs[cur];
100     for (int x = ex, y = ey, ty; x != -1; x = p[x], y = ty) {
101         costs[cur + 1] += a[x][y];
102         if (xy[x] != -1) costs[cur + 1] -= a[x][xy[x]];
103         ty = xy[x];
104         xy[x] = y;
105         yx[y] = x;
106     }
107 }
108 return costs[nx];
109 }
110
111 // 返回左部点的匹配点
112
113 std::vector<int> assignment() { return xy; }
114
115 // 返回所有顶点的顶标
116 std::pair<std::vector<T>, std::vector<T>> labels() {
117     return std::make_pair(lx, ly);
118 }
119
120 std::vector<T> weights() { return costs; }
121
122 private:
123     std::vector<T> lx, ly, slack, costs;
124     std::vector<int> xy, yx, p, slackx;
125     std::vector<bool> visx, visy;
126 };

```

2.11 有向图欧拉回路/路径

- check 欧拉回路: 所有点 $\text{degree_in} = \text{degree_out}$

- check **欧拉路径**: 所有点 $\text{degree_in} = \text{degree_out}$, 或者存在一对点出入度相差 1, 其他点 $\text{degree_in} = \text{degree_out}$

```

1  /**
2  * @brief 有向图欧拉路/回路求解类
3  *
4  * 使用 Hierholzer 算法（逐步插入回路法）寻找欧拉通路。
5  * 核心思想是从一个节点开始 DFS，将经过的边删除，直到无法前进时将节点入栈。
6  * 最终栈中逆序的节点序列即为一条欧拉路。
7  */
8  class EulerGraphDir {
9  public:
10     int n;           // 图的节点数
11     vector<vector<int>> g; // 邻接表
12     vector<int> din, dout; // 节点的入度和出度
13     vector<int> path;    // 存储找到的欧拉路
14
15 private:
16     vector<int> cur; // 当前弧优化：记录每个节点当前访问到了哪条边
17
18     /**
19      * @brief Hierholzer 算法核心
20      * @param u 当前节点
21      */
22     void dfs(int u) {
23         // 使用引用和后置 ++ 实现当前弧优化，遍历 u 的所有出边
24         for (int& i = cur[u]; i < g[u].size(); ) {
25             int v = g[u][i];
26             i++; // 移动到下一条边（这条边被"删除"）
27             dfs(v);
28         }
29         path.push_back(u); // 回溯时将节点加入路径
30     }
31
32 public:
33     /**
34      * @brief 构造函数
35      * @param n_ 节点数量
36      */
37     EulerGraphDir(int n_) : n(n_) {
38         g.resize(n);
39         din.assign(n, 0);
40         dout.assign(n, 0);
41         cur.assign(n, 0);
42     }
43
44     /**
45      * @brief 添加一条有向边
46      * @param u 起点
47      * @param v 终点
48      */
49     void add_edge(int u, int v) {
50         g[u].push_back(v);
51         dout[u]++;
52         din[v]++;
53     }
54
55     /**
56      * @brief 检查是否存在欧拉回路
57      * @return 如果所有点的入度都等于出度，则返回 true

```

```
58     */
59     bool has_euler_circuit() {
60         for (int i = 0; i < n; i++) {
61             if (din[i] != dout[i]) {
62                 return false;
63             }
64         }
65         // 注意: 还需要检查图的连通性, 这里省略, 假设图是连通的
66         return true;
67     }
68
69     /**
70     * @brief 检查是否存在欧拉路
71     * @return 如果满足欧拉路条件, 返回 true
72     */
73     bool has_euler_path() {
74         int start_node = -1, end_node = -1;
75         int odd_count = 0;
76         for (int i = 0; i < n; i++) {
77             if (din[i] != dout[i]) {
78                 odd_count++;
79                 if (dout[i] == din[i] + 1) {
80                     if (start_node != -1) return false; // 超过一个起点
81                     start_node = i;
82                 } else if (din[i] == dout[i] + 1) {
83                     if (end_node != -1) return false; // 超过一个终点
84                     end_node = i;
85                 } else {
86                     return false; // 度数差不为 1
87                 }
88             }
89         }
90         // 要么所有点入度 = 出度, 要么恰好一个起点一个终点
91         return odd_count == 0 || odd_count == 2;
92     }
93
94     /**
95     * @brief 求解欧拉路/回路
96     * @return 如果不存在则返回 false
97     */
98     bool solve() {
99         int start_node = -1;
100         int odd_out = 0, odd_in = 0;
101
102         for (int i = 0; i < n; i++) {
103             if (dout[i] > din[i]) {
104                 odd_out++;
105                 if (dout[i] - din[i] > 1) return false;
106                 start_node = i;
107             } else if (din[i] > dout[i]) {
108                 odd_in++;
109                 if (din[i] - dout[i] > 1) return false;
110             }
111         }
112
113         // 判断是否存在欧拉路或回路
114         if (!(odd_out == 0 && odd_in == 0) || (odd_out == 1 && odd_in == 1)) {
115             return false;
116         }
117     }
```

```

118 // 如果是欧拉回路, 从第一个有出度的点开始
119 if (start_node == -1) {
120     for (int i = 0; i < n; i++) {
121         if (dout[i] > 0) {
122             start_node = i;
123             break;
124         }
125     }
126     // 如果是空图或者只有孤立点, 也算合法
127     if (start_node == -1) return true;
128 }
129
130 dfs(start_node);
131 reverse(path.begin(), path.end());
132
133 // 检查是否所有边都被访问
134 int edge_count = 0;
135 for(int i = 0; i < n; ++i) edge_count += dout[i];
136 return path.size() == edge_count + 1;
137 }
138
139 /**
140  * @brief 对邻接表排序以获得字典序最小的欧拉路
141  */
142 void min_lexicographical() {
143     for (auto& vec : g) {
144         sort(vec.begin(), vec.end());
145     }
146 }
147 };

```

2.12 2-SAT, 强连通分量

```

1 struct TwoSat {
2     int n;
3     vector<vector<int>> e;
4     vector<int> ans;
5     TwoSat(int n) : n(n), e(2 * n), ans(n) {}
6
7     void add(int u, bool f, int v, bool g) {
8         e[2 * u + !f].push_back(2 * v + g);
9         e[2 * v + !g].push_back(2 * u + f);
10    }
11
12    bool work() {
13        vector<int> id(2 * n, -1), dfn(2 * n, -1), low(2 * n, -1);
14        vector<int> stk;
15        int tot = 0, cnt = 0;
16        auto tarjan = [&](auto self, int u) -> void {
17            stk.push_back(u);
18            dfn[u] = low[u] = tot++;
19            for (auto v : e[u]) {
20                if (dfn[v] == -1) {
21                    self(self, v);
22                    low[u] = min(low[u], low[v]);
23                } else if (id[v] == -1) {
24                    low[u] = min(low[u], dfn[v]);
25                }
26            }

```

```

27
28     if (dfn[u] == low[u]) {
29         int v;
30         do {
31             v = stk.back();
32             stk.pop_back();
33             id[v] = cnt;
34         } while (v != u);
35         cnt++;
36     }
37 };
38
39 // 改为反向遍历就是字典序最大。
40 for (int i = 0; i < 2 * n; i++) {
41     if (dfn[i] == -1) {
42         tarjan(tarjan, i);
43     }
44 }
45 // 输出字典序最小的可行解。
46 for (int i = 0; i < n; i++) {
47     if (id[2 * i] == id[2 * i + 1]) return 0;
48     ans[i] = id[2 * i] > id[2 * i + 1];
49 }
50 // 如果需要判断某个变量的取值是否唯一，需要判断由 0 能否到达 1 之类的,  $O(n^2)$ 
51 return 1;
52 }
53 }

```

2.13 Bitset Kosaraju

```

1 bitset<N> e[N], re[N], vis;
2
3 // e 是原图的邻接矩阵, re 是反图的邻接矩阵
4 // e[i][j] = 1 表示存在一条从 i 到 j 的有向边
5 // re[j][i] = 1 同样表示 i 到 j 的边
6
7 vector<int> sta;
8 void dfs0(int x, bitset<N> e[]) {
9     vis.reset(x);
10    while (true) {
11        int go = (e[x] & vis)._Find_first();
12        if (go == N) break;
13        dfs0(go, e);
14    }
15    sta.push_back(x);
16 }
17 // 返回一个二维向量，每个内层 vector 包含一个强连通分量的所有顶点
18 vector<vector<int>> solve() { // re 需要连好反向边
19     vis.set();
20     for (int i = 1; i <= n; ++i)
21         if (vis.test(i)) dfs0(i, e);
22     vis.set();
23     auto s = sta;
24     vector<vector<int>> ret;
25     for (int i = n - 1; i >= 0; --i)
26         if (vis.test(s[i])) {
27             sta.clear(), dfs0(s[i], re), ret.push_back(sta);
28         }
29     return ret;

```


30 }

2.14 边双连通分量

```
1 struct EBCC {
2     int n;
3     std::vector<std::vector<int>>> adj;
4     std::vector<int> stk;
5     std::vector<int> dfn, low, bel;
6     int cur, cnt;
7
8     EBCC() {}
9     EBCC(int n) { init(n); }
10
11     void init(int n) {
12         this->n = n;
13         adj.assign(n, {});
14         dfn.assign(n, -1);
15         low.resize(n);
16         bel.assign(n, -1);
17         stk.clear();
18         cur = cnt = 0;
19     }
20
21     void addEdge(int u, int v) {
22         adj[u].push_back(v);
23         adj[v].push_back(u);
24     }
25
26     void dfs(int x, int p) {
27         dfn[x] = low[x] = cur++;
28         stk.push_back(x);
29
30         for (auto y : adj[x]) {
31             if (y == p) {
32                 continue;
33             }
34             if (dfn[y] == -1) {
35                 dfs(y, x);
36                 low[x] = std::min(low[x], low[y]);
37             } else if (bel[y] == -1 && dfn[y] < dfn[x]) {
38                 low[x] = std::min(low[x], dfn[y]);
39             }
40         }
41
42         if (dfn[x] == low[x]) {
43             int y;
44             do {
45                 y = stk.back();
46                 bel[y] = cnt;
47                 stk.pop_back();
48             } while (y != x);
49             cnt++;
50         }
51     }
52
53     std::vector<int> work() {
54         dfs(0, -1);
55         return bel;
```

```

56     }
57
58     struct Graph {
59         int n; // 缩点后新图的节点数量
60         std::vector<std::pair<int, int>>
61             edges; // 缩点后新图的边集，可自行修改为邻接表
62         std::vector<int> siz; // 包含了多少个原始图的节点
63         std::vector<int> cnte; // 包含了多少条原始图的边
64     };
65     Graph compress() {
66         Graph g;
67         g.n = cnt;
68         g.siz.resize(cnt);
69         g.cnte.resize(cnt);
70         for (int i = 0; i < n; i++) {
71             g.siz[bel[i]]++;
72             for (auto j : adj[i]) {
73                 if (bel[i] < bel[j]) {
74                     // 因为添加的一定是割边，所以不存在重复。
75                     g.edges.emplace_back(bel[i], bel[j]);
76                 } else if (i < j) {
77                     g.cnte[bel[i]]++;
78                 }
79             }
80         }
81         return g;
82     }
83 };

```

2.15 点双连通分量

不可以传入自环，否则孤立点不会被统计，有时候要考虑特判孤立点。

```

1  /*
2  用于求解无向图的点双连通分量（VBCC），并能构建对应的圆方树。
3  圆方树是一个新图，其中包含两种类型的节点：
4  1. 方点：代表原始图中的一个点双连通分量。
5  2. 圆点：代表原始图中的一个点。
6  树中的边连接一个圆点和一个方点，当且仅当该点属于该点双连通分量。
7
8  当然也有给割点单独新建一个点的建树方法。
9  */
10 struct VBCC {
11     int n; // 原始图的节点数
12     vector<vector<int>> adj;
13     vector<vector<int>> col; // 存储每个点双连通分量包含的节点
14     vector<int> dfn, low;
15     vector<int> stk;
16     int cur, cnt; // 时间戳计数器和点双连通分量计数器
17     vector<int> cut; // 标记每个节点是否为割点
18
19     VBCC(int n) { init(n); }
20
21     void init(int n) {
22         this->n = n;
23         adj.assign(n, {});
24         dfn.assign(n, -1);
25         low.resize(n);
26         col.assign(n, {});

```

```
27     cut.assign(n, 0);
28     stk.clear();
29     cnt = cur = 0;
30 }
31
32 void addEdge(int u, int v) {
33     adj[u].push_back(v);
34     adj[v].push_back(u);
35 }
36
37 void tarjan(int x, int root) {
38     low[x] = dfn[x] = cur++;
39     stk.push_back(x);
40     if (x == root && !adj[x].size()) {
41         col[cnt++].push_back(x);
42         return;
43     }
44
45     int flg = 0;
46     for (auto y : adj[x]) {
47         if (dfn[y] == -1) {
48             tarjan(y, root);
49
50             low[x] = min(low[x], low[y]);
51
52             if (dfn[x] <= low[y]) {
53                 flg++;
54                 if (x != root || flg > 1) {
55                     cut[x] = 1;
56                 }
57                 int pre = 0;
58                 do {
59                     pre = stk.back();
60                     col[cnt].push_back(pre);
61                     stk.pop_back();
62                 } while (pre != y);
63                 col[cnt++].push_back(x);
64             }
65         } else {
66             low[x] = min(low[x], dfn[y]);
67         }
68     }
69 }
70
71 void work() {
72     for (int i = 0; i < n; i++) {
73         if (dfn[i] == -1) {
74             tarjan(i, i);
75         }
76     }
77 }
78
79 // 新图（圆方树）的结构体定义
80 struct Graph {
81     int n; // 新图的节点数量
82     vector<pair<int, int>> edges; // 新图的边集
83 };
84
85 Graph compress() {
86     // 节点编号规则:
```

```

87     // 0 到 n - 1 是 圆点 代表原图中的点
88     // n 开始是 方点 代表点双连通分量
89
90     Graph g;
91     g.n = n + cnt;
92
93     // 构建新图的边
94     for (int i = 0; i < cnt; i++) {
95         for (auto u : col[i]) {
96             g.edges.emplace_back(u, i + n);
97         }
98     }
99
100     return g;
101 }
102 };

```

2.16 Dominator Tree 支配树

```

1  vector<int> G[MAXN], R[MAXN], son[MAXN];
2  int ufs[MAXN]; // R 是反图, son 存的是 sdom 树上的儿子
3  int idom[MAXN], sdom[MAXN], anc[MAXN];
4  // anc: sdom 的 dfn 最小的祖先
5  int pr[MAXN], dfn[MAXN], id[MAXN], stamp;
6  int findufs(int x) { if (ufs[x] == x) return x;
7      | int t = ufs[x]; ufs[x] = findufs(ufs[x]);
8      | if (dfn[sdom[anc[x]]] > dfn[sdom[anc[t]]])
9      |     | anc[x] = anc[t];
10     | return ufs[x]; }
11 void dfs(int x) {
12     | dfn[x] = ++stamp; id[stamp] = x; sdom[x] = x;
13     | for (int y : G[x]) if (!dfn[y]) { pr[y] = x; dfs(y); } }
14 void get_dominator(int n) {
15     | for (int i = 1; i <= n; i++) ufs[i] = anc[i] = i;
16     | dfs(1);
17     | for (int i = n; i > 1; i--) { int x = id[i];
18     |     | for (int y : R[x]) if (dfn[y]) { findufs(y);
19     |     |     | if (dfn[sdom[x]] > dfn[sdom[anc[y]]])
20     |     |     |     | sdom[x] = sdom[anc[y]]; }
21     |     | son[sdom[x]].push_back(x); ufs[x] = pr[x];
22     |     | for (int u : son[pr[x]]) { findufs(u);
23     |     |     | idom[u] = (sdom[u] == sdom[anc[u]] ?
24     |     |     |     | pr[x] : anc[u]); }
25     |     | son[pr[x]].clear(); }
26     | for (int i = 2; i <= n; i++) { int x = id[i];
27     |     | if (idom[x] != sdom[x]) idom[x] = idom[idom[x]];
28     |     | son[idom[x]].push_back(x); } }

```

2.17 Dinic 最大流

复杂度证明思路 假设 dist 为残量网络上的距离. Dinic 一轮增广会找到一个极大的长度为 $\text{dist}(s, t)$ 的增广路集合 blocking flow, 增广后 $\text{dist}(s, t)$ 将会增大. 因此只有 $O(V)$ 轮; 如果一轮增广是 $O(VE)$ 的, 总复杂度是 $O(V^2E)$.

单位流量网络 在 0-1 流量图上 Dinic 有更好的性质.

- 复杂度为 $O(\min\{V^{2/3}, E^{1/2}\}E)$.
- $\text{dist}(s, t) = d$, 残量网络上至多还存在 E/d 的流.
- 每个点只有一个入/出度时复杂度 $O(V^{1/2}E)$, 例如 Hopcroft-Karp.

```

1  const ll INF = 1e9;
2  int nn, mm, s, t, cnt = 1, h[N], ans;
3  struct node {
4      int v, nex;
5      ll w;
6  } e[M];
7  void add(int u, int v, ll w) {
8      e[++cnt] = {v, h[u], w};
9      h[u] = cnt;
10 }
11 int d[N], now[N];
12 int bfs() {
13     for (int i = 1; i <= nn; i++) d[i] = INF;
14     queue<int> q;
15     d[s] = 0;
16     now[s] = h[s];
17     q.push(s);
18     while (q.size()) {
19         int p = q.front();
20         q.pop();
21         for (int i = h[p]; i; i = e[i].nex) {
22             int v = e[i].v;
23             if (e[i].w > 0 && d[v] == INF) {
24                 d[v] = d[p] + 1;
25                 now[v] = h[v];
26                 q.push(v);
27                 if (v == t) return 1;
28             }
29         }
30     }
31     return 0;
32 }
33 ll dfs(int p, ll sum) {
34     if (p == t) return sum;
35     ll k = 0, flow = 0;
36     for (int i = now[p]; i && sum; i = e[i].nex) {
37         now[p] = i;
38         ll v = e[i].v, w = e[i].w;
39         if (w > 0 && d[v] == d[p] + 1) {
40             k = dfs(v, min(sum, (ll)w));
41             if (k == 0) d[v] = INF;
42             e[i].w -= k;
43             e[i ^ 1].w += k;
44             sum -= k;
45             flow += k;
46         }
47     }
48     return flow;
49 }
50 void solve() {
51     for (int i = 1; i <= nn; i++) h[i] = 0;
52     ans = 0;
53     cnt = 1;
54     // add(u,v,w),add(v,u,0);
55     while (bfs()) ans += dfs(s, INF);
56 }

```

```

1  template <class T>
2  struct MaxFlow {

```

```
3 struct _Edge {
4     int to;
5     T cap;
6     _Edge(int to, T cap) : to(to), cap(cap) {}
7 };
8
9 int n;
10 std::vector<_Edge> e;
11 std::vector<std::vector<int>> g;
12 std::vector<int> cur, h;
13
14 MaxFlow() {}
15 MaxFlow(int n) { init(n); }
16
17 void init(int n) {
18     this->n = n;
19     e.clear();
20     g.assign(n, {});
21     cur.resize(n);
22     h.resize(n);
23 }
24
25 bool bfs(int s, int t) {
26     h.assign(n, -1);
27     std::queue<int> que;
28     h[s] = 0;
29     que.push(s);
30     while (!que.empty()) {
31         const int u = que.front();
32         que.pop();
33         for (int i : g[u]) {
34             auto [v, c] = e[i];
35             if (c > 0 && h[v] == -1) {
36                 h[v] = h[u] + 1;
37                 if (v == t) {
38                     return true;
39                 }
40                 que.push(v);
41             }
42         }
43     }
44     return false;
45 }
46
47 T dfs(int u, int t, T f) {
48     if (u == t) {
49         return f;
50     }
51     auto r = f;
52     for (int &i = cur[u]; i < int(g[u].size()); ++i) {
53         const int j = g[u][i];
54         auto [v, c] = e[j];
55         if (c > 0 && h[v] == h[u] + 1) {
56             auto a = dfs(v, t, std::min(r, c));
57             e[j].cap -= a;
58             e[j ^ 1].cap += a;
59             r -= a;
60             if (r == 0) {
61                 return f;
62             }
63         }
64     }
65 }
```

```
63     }
64 }
65 return f - r;
66 }
67 void addEdge(int u, int v, T c) {
68     g[u].push_back(e.size());
69     e.emplace_back(v, c);
70     g[v].push_back(e.size());
71     e.emplace_back(u, 0);
72 }
73 T flow(int s, int t) {
74     T ans = 0;
75     while (bfs(s, t)) {
76         cur.assign(n, 0);
77         ans += dfs(s, t, std::numeric_limits<T>::max());
78     }
79     return ans;
80 }
81
82 std::vector<bool> minCut() {
83     std::vector<bool> c(n);
84     for (int i = 0; i < n; i++) {
85         c[i] = (h[i] != -1);
86     }
87     return c;
88 }
89
90 struct Edge {
91     int from;
92     int to;
93     T cap;
94     T flow;
95 };
96
97 // 返回每条边的流量情况, 按照添加边的顺序
98 std::vector<Edge> edges() {
99     std::vector<Edge> a;
100     for (int i = 0; i < e.size(); i += 2) {
101         Edge x;
102         x.from = e[i + 1].to;
103         x.to = e[i].to;
104         x.cap = e[i].cap + e[i + 1].cap;
105         x.flow = e[i + 1].cap;
106         a.push_back(x);
107     }
108     return a;
109 }
110
111 // 构建残留网络
112 std::vector<vector<int>> build() {
113     std::vector<std::vector<int>> g(n);
114     // 在这个实现中, e[i] 和 e[i^1] 是一对相反的边。
115     // e[i] 是一条从 e[i^1].to 到 e[i].to 的边。
116     for (size_t i = 0; i < e.size(); i++) {
117         // 如果一条边的剩余容量大于 0, 那么它就存在于残留网络中
118         if (e[i].cap > 0) {
119             // 找到这条边的起点。
120             // e[i] 的反向边是 e[i^1], 反向边的终点就是 e[i] 的起点。
121             int from = e[i ^ 1].to;
122             // 边的终点
```

```

123         int to = e[i].to;
124         g[from].push_back(to);
125     }
126 }
127 // 返回构建好的残留网络邻接表
128 return g;
129 }
130 };

```

2.18 原始对偶费用流

```

1 bool bfs() {
2     for (int i = S; i <= T; i++) cur[i] = head[i]; // S-T?
3     for (int i = S; i <= T; i++) dep[i] = 0;      // S-T?
4     dep[S] = 1;
5     queue<int> q;
6     q.push(S);
7     while (!q.empty()) {
8         int x = q.front();
9         q.pop();
10        for (int i = head[x]; i; i = e[i].nxt) {
11            auto [nxt, v, w, f] = e[i];
12            if (f && h[v] == h[x] + w && !dep[v]) {
13                dep[v] = dep[x] + 1, q.push(v);
14            }
15        }
16    }
17    return !!dep[T];
18 }
19 int dfs(int x, int flow) {
20     if (x == T) return flow;
21     int used = 0, ret;
22     for (int &i = cur[x]; i; i = e[i].nxt) {
23         auto [nxt, v, w, f] = e[i];
24         if (dep[v] == dep[x] + 1 && h[v] == h[x] + w && f &&
25             (ret = dfs(v, min(flow - used, f)))) {
26             e[i].f -= ret;
27             e[i ^ 1].f += ret;
28             used += ret;
29             if (flow == used) break;
30         }
31     }
32     return used;
33 }
34 typedef pair<value, int> pii; // Unusual!
35 pii solve() {                // return {cost, flow}
36     value cost = 0;
37     int flow = 0;
38     for (int i = S; i <= T; i++) h[i] = 0; // S-T?
39     for (bool first = true;;) {
40         priority_queue<pii, vector<pii>, greater<pii>> q;
41         for (int i = S; i <= T; i++) dis[i] = INF_value; // S-T?
42         dis[S] = 0;
43         if (first) {
44             // TODO: SSSP, Bellman-Ford / DP on DAG
45             first = false;
46         } else {
47             q.push({0, S});
48             while (!q.empty()) {

```



```

49         auto [d, x] = q.top();
50         q.pop();
51         if (dis[x] != d) continue;
52         for (int o = head[x]; o; o = e[o].nxt) {
53             value w = d + e[o].w + h[x] - h[e[o].v];
54             if (e[o].f > 0 && dis[e[o].v] > w) {
55                 dis[e[o].v] = w;
56                 q.push({w, e[o].v});
57             }
58         }
59     }
60 }
61 if (dis[T] >= INF_value) break;
62 // 所有点必须可达, 否则加 h[i] += min(dis[i], dis[T])
63 for (int i = S; i <= T; i++) h[i] += dis[i]; // S-T?
64 int f = 0;
65 while (bfs()) f += dfs(S, INF_int);
66 cost += f * h[T];
67 flow += f;
68 }
69 return {cost, flow};
70 }

```

```

1 template <class T>
2 struct MinCostFlow {
3     struct _Edge {
4         int to;
5         T cap;
6         T cost;
7         _Edge(int to_, T cap_, T cost_) : to(to_), cap(cap_), cost(cost_) {}
8     };
9     int n;
10    std::vector<_Edge> e;
11    std::vector<std::vector<int>>> g;
12    std::vector<T> h, dis;
13    std::vector<int> pre;
14    bool dijkstra(int s, int t) {
15        dis.assign(n, std::numeric_limits<T>::max());
16        pre.assign(n, -1);
17        std::priority_queue<std::pair<T, int>, std::vector<std::pair<T, int>>,
18                           std::greater<std::pair<T, int>>>
19        que;
20        dis[s] = 0;
21        que.emplace(0, s);
22        while (!que.empty()) {
23            T d = que.top().first;
24            int u = que.top().second;
25            que.pop();
26            if (dis[u] != d) {
27                continue;
28            }
29            for (int i : g[u]) {
30                int v = e[i].to;
31                T cap = e[i].cap;
32                T cost = e[i].cost;
33                if (cap > 0 && dis[v] > d + h[u] - h[v] + cost) {
34                    dis[v] = d + h[u] - h[v] + cost;
35                    pre[v] = i;
36                    que.emplace(dis[v], v);

```

```
37     }
38   }
39   }
40   return dis[t] != std::numeric_limits<T>::max();
41 }
42 MinCostFlow() {}
43 MinCostFlow(int n_) { init(n_); }
44 void init(int n_) {
45     n = n_;
46     e.clear();
47     g.assign(n, {});
48 }
49 void addEdge(int u, int v, T cap, T cost) {
50     g[u].push_back(e.size());
51     e.emplace_back(v, cap, cost);
52     g[v].push_back(e.size());
53     e.emplace_back(u, 0, -cost);
54 }
55 std::pair<T, T> flow(int s, int t) {
56     T flow = 0;
57     T cost = 0;
58     h.assign(n, 0);
59     while (dijkstra(s, t)) {
60         for (int i = 0; i < n; ++i) {
61             h[i] += dis[i];
62         }
63         T aug = std::numeric_limits<int>::max();
64         for (int i = t; i != s; i = e[pre[i] ^ 1].to) {
65             aug = std::min(aug, e[pre[i]].cap);
66         }
67         for (int i = t; i != s; i = e[pre[i] ^ 1].to) {
68             e[pre[i]].cap -= aug;
69             e[pre[i] ^ 1].cap += aug;
70         }
71         flow += aug;
72         cost += aug * h[t];
73     }
74     return std::make_pair(flow, cost);
75 }
76 struct Edge {
77     int from;
78     int to;
79     T cap;
80     T cost;
81     T flow;
82 };
83 std::vector<Edge> edges() {
84     std::vector<Edge> a;
85     for (int i = 0; i < e.size(); i += 2) {
86         Edge x;
87         x.from = e[i + 1].to;
88         x.to = e[i].to;
89         x.cap = e[i].cap + e[i + 1].cap;
90         x.cost = e[i].cost;
91         x.flow = e[i + 1].cap;
92         a.push_back(x);
93     }
94     return a;
95 }
```

```
96 };
```

2.19 虚树

```
1 int dfn[N];
2 int h[N], m, a[N << 1], len; // 存储关键点
3
4 bool cmp(int x, int y) {
5     return dfn[x] < dfn[y]; // 按照 dfs 序排序
6 }
7
8 void build() {
9     sort(h + 1, h + m + 1, cmp); // 把关键点按照 dfs 序排序
10    for (int i = 1; i < m; ++i) {
11        a[++len] = h[i];
12        a[++len] = lca(h[i], h[i + 1]); // 插入 lca
13    }
14    a[++len] = h[m];
15    sort(a + 1, a + len + 1, cmp); // 把所有虚树上的点按照 dfs 序排序
16    len = unique(a + 1, a + len + 1) - a - 1; // 去重
17    for (int i = 1, lc; i < len; ++i) {
18        lc = lca(a[i], a[i + 1]);
19        conn(lc, a[i + 1]); // 连边 lc -> a[i+1]
20    }
21 }
```

2.20 网络流总结

最小割集, 最小割必须边以及可行边

最小割集 从 S 出发, 在残余网络中 BFS 所有权值非 0 的边 (包括反向边), 得到点集 $\{S\}$, 另一集为 $\{V\} - \{S\}$.

最小割集必须点 残余网络中 S 直接连向的点必在 S 的割集中, 直接连向 T 的点必在 T 的割集中; 若这些点的并集为全集, 则最小割方案唯一.

最小割可行边 在残余网络中求强联通分量, 将强联通分量缩点后, 剩余的边即为最小割可行边, 同时这些边也必然满流.

最小割必须边 在残余网络中求强联通分量, 若 S 出发可到 u , T 出发可到 v , 等价于 $scc_S = scc_u$ 且 $scc_T = scc_v$, 则该边为必须边.

常见问题

最大权闭合子图 点权, 限制条件形如: 选择 A 则必须选择 B , 选择 B 则必须选择 C, D . 建图方式: B 向 A 连边, CD 向 B 连边. 求解: S 向正权点连边, 负权点向 T 连边, 其余边容量 ∞ , 求最小割, 答案为 S 所在最小割集.

二次布尔型 (文理分科) n 个点分为两类, i 号点有 l_i 或 r_i 的代价, i, j 同属一侧分别获得 l_{ij} 或 r_{ij} 的代价, 问最小代价. $L \rightarrow i: (l_i + 1/2 \sum_j l_{ij}), i \rightarrow R: (r_i + 1/2 \sum_j r_{ij}), i \leftrightarrow j: 1/2(l_{ij} + r_{ij})$. 实现时边权乘 2 为整数, 求解后答案除 2 为整数. 图拆点可以看作二分图.

如果是二元限制是分类不同时有一个代价 d_{ij} , 建图可以简化为 $L \rightarrow i: l_i, i \rightarrow R: r_i, i \leftrightarrow j: d_{ij}$. 经典例子: xor 最小值, 按位拆开建图.

混合图欧拉回路 把无向边随便定向, 计算每个点的入度和出度, 如果有某个点出入度之差 $\deg_i = \text{in}_i - \text{out}_i$ 为奇数, 肯定不存在欧拉回路. 对于 $\deg_i > 0$ 的点, 连接边 $(i, T, \deg_i/2)$; 对于 $\deg_i < 0$ 的点, 连接边 $(S, i, -\deg_i/2)$. 最后检查是否满流即可.

二物流 水源 S_1 , 水汇 T_1 , 油源 S_2 , 油汇 T_2 , 每根管道流量共用. 求流量和最大. 建超级源 SS_1 汇 TT_1 , 连边 $SS_1 \rightarrow S_1, SS_1 \rightarrow S_2, T_1 \rightarrow TT_1, T_2 \rightarrow TT_1$, 设最大流为 x_1 . 建超级源 SS_2 汇 TT_2 , 连边 $SS_2 \rightarrow S_1, SS_2 \rightarrow T_2, T_1 \rightarrow TT_2, S_2 \rightarrow TT_2$, 设最大流为 x_2 . 则最大流中水流量 $\frac{x_1+x_2}{2}$, 油流量 $\frac{x_1-x_2}{2}$.

无源汇有上下界可行流 每条边 (u, v) 有一个上界容量 $C_{u,v}$ 和下界容量 $B_{u,v}$, 我们让下界变为 0, 上界变为 $C_{u,v} - B_{u,v}$, 但这样做流量不守恒. 建立超级源点 SS 和超级汇点 TT , 用 du_i 来记录每个节点的流量情况, 即流入减流出, $du_i = \sum B_{j,i} - \sum B_{i,j}$, 添加一些附加弧. 当 $du_i > 0$ 时, 连边 (SS, i, du_i) ; 当 $du_i < 0$ 时, 连边 $(i, TT, -du_i)$. 最后对 (SS, TT) 求一次最大流即可, 当所有附加边全部满流时 (即 $\maxflow == du_i > 0$) 时有可行解.

有源汇有上下界最大可行流 首先, 为求解有源汇上下界最大流问题, 需将原网络 G 转化为无源汇的循环网络 G_{circ} . 具体操作为在原图基础上增设一条从汇点 T 到源点 S 的边 (T, S) , 其流量限制为 $[0, +\infty]$. 接着, 执行无源汇有上下界可行流, 若存在可行流, 记下边 (T, S) 上分配到的流量作为初始可行流流量 $v_{initial}$.

在确认存在可行流后, 移除 SS, TT 及相关边和 (T, S) 边, 进入增广阶段. 我们基于阶段一求得的可行流 f , 在原图的边上构建残量网络 G_{res} . 具体的建边逻辑如下:

- 对于每条边 (u, v) , 若 $f(u, v) < c(u, v)$, 则在 G_{res} 中添加一条前向边 (u, v) , 其残量为 $c(u, v) - f(u, v)$.
- 对于每条边 (u, v) , 若 $f(u, v) > b(u, v)$, 则在 G_{res} 中添加一条反向边 (v, u) , 其残量为 $f(u, v) - b(u, v)$.

在此网络中, 以原图的源点 S 和汇点 T 为起点和终点, 再次运行最大流算法. 此次求得的最大流值即为在初始可行流基础上可增加的附加流量 v_{add} . 最终, 原问题的最大流值等于两部分之和:

$$\text{最大流} = v_{initial} + v_{add}$$

有源汇有上下界最小可行流 首先, 求解有源汇上下界最小流的第一步与最大流完全相同: 将原网络 G 转化为无源汇的循环网络 G_{circ} , 具体操作为在原图基础上增设一条从汇点 T 到源点 S 的边 (T, S) , 其流量限制为 $[0, +\infty]$. 接着, 执行无源汇有上下界可行流的判断方法, 若存在可行流, 记下边 (T, S) 上分配到的流量作为初始可行流流量 $v_{initial}$.

在确认存在可行流后, 移除 SS, TT 及相关边和 (T, S) 边, 进入退流阶段. 我们基于阶段一求得的可行流 f , 在原图的边上构建残量网络 G_{res} . 具体的建边逻辑如下:

- 对于每条边 (u, v) , 若 $f(u, v) < c(u, v)$, 则在 G_{res} 中添加一条前向边 (u, v) , 其残量为 $c(u, v) - f(u, v)$.
- 对于每条边 (u, v) , 若 $f(u, v) > b(u, v)$, 则在 G_{res} 中添加一条反向边 (v, u) , 其残量为 $f(u, v) - b(u, v)$.

与最大流不同, 我们在此网络中, 以原图的汇点 T 为源、源点 S 为汇, 再次运行最大流算法. 此次求得的最大流值即为可以从初始流中抵消或退回的流量, 记为 v_{reduce} . 最终, 原问题的最小流值等于初始流减去最大可退流量:

$$\text{最小流} = v_{initial} - v_{reduce}$$

有上下界费用流 如果求无源汇有上下界最小费用可行流或有源汇有上下界最小费用最大可行流, 用 1.6.3.1/1.6.3.2 的构图方法, 给边加上费用即可. 求有源汇有上下界最小费用最小可行流, 要先用 1.6.3.3 的方法建图, 先求出一个保证必要边满流情况下的最小费用. 如果费用全部非负, 那么这时的费用就是答案. 如果费用有负数, 那么流多了可能更好, 继续做从 S 到 T 的流量任意的最小费用流, 加上原来的费用就是答案.

费用流消负环 新建超级源 SS 汇 TT , 对于所有流量非空的负权边 e , 先流满 ($ans += e.f * e.c$, $e.rev.f += e.f$, $e.f = 0$), 再连边 $SS \rightarrow e.to$, $e.from \rightarrow TT$, 流量均为 $e.f (> 0)$, 费用均为 0. 再连边 $T \rightarrow S$ 流量 ∞ 费用 0. 此时没有负环了. 做一遍 SS 到 TT 的最小费用最大流, 将费用累加 ans , 拆掉 $T \rightarrow S$ 的那条边 (此边的流量为残量网络中 $S \rightarrow T$ 的流量). 此时负环已消, 再继续跑最小费用最大流.

整数线性规划转费用流

首先将约束关系转化为所有变量下界为 0, 上界没有要求, 并满足一些等式, 每个变量在均在等式左边且出现恰好两次, 系数为 +1 和 -1, 优化目标为 $\max \sum v_i x_i$ 的形式. 将等式看做点, 等式 i 右边的值 b_i 若为正, 则 S 向 i 连边 $(b_i, 0)$, 否则 i 向 T 连边 $(-b_i, 0)$. 将变量看做边, 记变量 x_i 的上界为 m_i (无上界则 $m_i = \text{inf}$), 将 x_i 系数为 +1 的那个等式 u 向系数为 -1 的等式 v 连边 (m_i, v_i) .

2.21 Gomory-Hu 无向图最小割树 $O(V^3E)$

每次随便找两个点 s, t 求在**原图**的最小割, 在最小割树上连 (s, t, w_{cut}) , 递归对由割集隔开的部分继续做. 在得到的树上, 两点最小割即为树上瓶颈路. 实现时, 由于是随意找点, 可以写为分治的形式.

2.22 Stoer-Wagner 无向图最小割 $O(VE + V^2 \log V)$

```

1  const int N = 601;
2  int f[N], siz[N], G[N][N];
3  int getf(int x) {return f[x] == x ? x : f[x] = getf(f[x]);}
4  int dis[N], vis[N], bin[N];
5  int n, m;
6  int contract(int &s, int &t) { // Find s,t
7      memset(vis, 0, sizeof(vis));
8      memset(dis, 0, sizeof(dis));
9      int i, j, k, mincut, maxc;
10     for (i = 1; i <= n; i++) {
11         k = -1; maxc = -1;
12         for (j = 1; j <= n; j++)
13             if (!bin[j] && !vis[j] && dis[j] > maxc) {
14                 k = j;
15                 maxc = dis[j];
16             }
17         if (k == -1) return mincut;
18         s = t; t = k; mincut = maxc; vis[k] = true;
19         for (j = 1; j <= n; j++)
20             if (!bin[j] && !vis[j]) dis[j] += G[k][j];
21     } return mincut; }
22 const int inf = 0x3f3f3f3f;
23 int solve() {
24     int mincut, i, j, s, t, ans;
25     for (mincut = inf, i = 1; i < n; i++) {
26         ans = contract(s, t);
27         bin[t] = true;
28         if (mincut > ans) mincut = ans;
29         if (mincut == 0) return 0;
30         for (j = 1; j <= n; j++)
31             if (!bin[j]) G[s][j] = (G[j][s] += G[j][t]);
32     } return mincut; }
33 int main() {
34     cin >> n >> m;
35     for (int i = 1; i <= n; ++i) f[i] = i, siz[i] = 1;
36     for (int i = 1, u, v, w; i <= m; ++i) {
37         cin >> u >> v >> w;
38         int fu = getf(u), fv = getf(v);
39         if (fu != fv) {
40             if (siz[fu] > siz[fv]) swap(fu, fv);
41             f[fu] = fv, siz[fv] += siz[fu];
42             G[u][v] += w, G[v][u] += w;
43         }
44     }
45     cout << (siz[getf(1)] != n ? 0 : solve());
46 }

```

2.23 弦图

弦图的定义 连接环中不相邻的两个点的边称为弦. 一个无向图称为弦图, 当图中任意长度都大于 3 的环都至少有一个弦.

单纯点 一个点称为单纯点当 $\{v\} \cup A(v)$ 的导出子图为一个团. 任何一个弦图都至少有一个单纯点, 不是完全图的弦图至少有两个不相邻的单纯点.

完美消除序列 一个序列 $v_1, v_2, \dots, v_n$ 满足 v_i 在 $v_i, \dots, v_n$ 的诱导子图中为一个单纯点. 一个无向图是弦图当且仅当它有一个完美消除序列.

最大势算法 从 n 到 1 的顺序依次给点标号. 设 label_i 表示第 i 个点与多少个已标号的点相邻, 每次选择 label 最大的未标号的点进行标号. 用桶维护优先队列可以做到 $O(n + m)$.

弦图的判定 判定最大势算法输出是否合法即可. 如果依次判断是否构成团, 时间复杂度为 $O(nm)$. 考虑优化, 设 $v_{i+1}, \dots, v_n$ 中所有与 v_i 相邻的点依次为 $N(v_i) = \{v_{j_1}, \dots, v_{j_k}\}$. 只需判断 v_{j_1} 是否与 $v_{j_2}, \dots, v_{j_k}$ 相邻即可. 时间复杂度 $O(n + m)$.

弦图的染色 完美消除序列**从后往前**染色, 染上出度的 mex .

最大独立集 完美消除序列**从前往后**能选就选.

团数 最大团的点数. 一般团团数 $\leq$ 色数, 弦图团数 = 色数.

极大团 弦图的极大团一定为 $\{x\} \cup N(x)$.

最小团覆盖 用最少的团覆盖所有的点. 设最大独立集为 $\{p_1, \dots, p_t\}$, 则 $\{p_1 \cup N(p_1), \dots, p_t \cup N(p_t)\}$ 为最小团覆盖.

弦图 k 染色计数 $\prod_{v \in V} k - N(v) + 1$.

区间图 每个顶点代表一个区间, 有边当且仅当区间有交. 区间图是弦图, 一个完美消除序列是右端点排序.

```

1 vector <int> L[N];
2 int seq[N], lab[N], col[N], id[N], vis[N];
3 void mcs() {
4     for (int i = 0; i < n; i++) L[i].clear();
5     fill(lab + 1, lab + n + 1, 0);
6     fill(id + 1, id + n + 1, 0);
7     for (int i = 1; i <= n; i++) L[0].push_back(i);
8     int top = 0;
9     for (int k = n; k; k--) {
10         int x = -1;
11         for ( ; ; ) {
12             if (L[top].empty()) top --;
13             else {
14                 x = L[top].back(), L[top].pop_back();
15                 if (lab[x] == top) break;
16             }
17         }
18         seq[k] = x; id[x] = k;
19         for (auto v : E[x]) {
20             if (!id[v]) {
21                 L[++lab[v]].push_back(v);
22                 top = max(top, lab[v]);
23             }
24         }
25     }
26     bool check() {
27         fill(vis + 1, vis + n + 1, 0);
28         for (int i = n; i; i--) {
29             int x = seq[i];
30             vector <int> to;
31             for (auto v : E[x])

```

```

32 |         int w = to.front();
33 |         for (auto v : to) if (id[v] < id[w]) w = v;
34 |         for (auto v : E[w]) vis[v] = i;
35 |         for (auto v : to)
36 |             if (v != w && vis[v] != i) return false;
37 |     } return true; }
38 void color() {
39     fill(vis + 1, vis + n + 1, 0);
40     for (int i = n; i; i--) {
41         int x = seq[i];
42         for (auto v : E[x]) vis[col[v]] = x;
43         for (int c = 1; !col[x]; c++)
44             if (vis[c] != x) col[x] = c;
45     } }

```

2.24 Minimum Mean Cycle 最小平均值环 $O(n^2)$

```

1 // 点标号为 1,2,...,n, 0 为虚拟源点向其他点连权值为 0 的单向边.
2 // f[i][v] : 从 0 到 v 恰好经过 i 条路的最短路
3 ll f[N][N] = {Inf}; int u[M], v[M], w[M]; f[0][0] = 0;
4 for(int i = 1; i <= n + 1; i++)
5     for(int j = 0; j < m; j++)
6         f[i][v[j]] = min(f[i][v[j]], f[i - 1][u[j]] + w[j]);
7 double ans = Inf;
8 for(int i = 1; i <= n; i++) {
9     double t = -Inf;
10    for(int j = 1; j <= n; j++)
11        t = max(t, (f[n][i] - f[j][i]) / (double)(n - j));
12    ans = min(t, ans); }

```

2.25 一般图最大匹配 - Blossom

```

1 int n, m, l, r, u, v, res, q[N], p[N], h[N], fa[N], col[N];
2 vector<int> G[N];
3 inline int find(int x) {return fa[x] == x ? x : fa[x] = find(fa[x]);}
4 int lca(int u, int v) {
5     static int fl[N], tim; ++tim;
6     while (fl[u] != tim) {
7         if (u) fl[u] = tim, u = find(p[h[u]]);
8         swap(u, v);
9     } return u; }
10 void blossom(int u, int v, int t) {
11     while (find(u) != t) {
12         p[u] = v, v = h[u], fa[u] = fa[v] = t;
13         if (col[v] == 1) col[v] = 2, q[++r] = v;
14         u = p[v]; } }
15 bool match(int rt) {
16     for (int i = 1; i <= n; i++) fa[i] = i, col[i] = 0;
17     l = r = 1, q[1] = rt, col[rt] = 2;
18     while (l <= r) {
19         u = q[l++];
20         for (int v : G[u]) if (!col[v]) {
21             col[v] = 1, col[h[v]] = 2, p[v] = u, q[++r] = h[v];
22             if (!h[v]) {
23                 while (u) u = h[p[v]], h[v] = p[v], h[p[v]] = v, v = u;
24                 return true;
25             } }
26     } else if (col[v] == 2) {

```

```

27 |   |   |   | int t = lca(u, v); blossom(u, v, t), blossom(v, u, t);
28 |   |   |   }
29 |   } return false;
30 } // for(int i=1;i<=n;i++)if(!h[i]&&match(i))++res;

```

2.26 图论结论

2.26.1 最小乘积问题原理

每个元素有两个权值 $\{x_i\}$ 和 $\{y_i\}$, 要求在某个限制下 (例如生成树, 二分图匹配) 使得 $\sum x \sum y$ 最小. 对于任意一种符合限制的选取方法, 记 $X = \sum x_i, Y = \sum y_i$, 可看做平面内一点 (X, Y) . 答案必在下凸壳上, 找出该下凸壳所有点, 即可枚举获得最优答案. 可以递归求出此下凸壳所有点, 分别找出距 x, y 轴最近的两点 A, B , 分别对应于 $\sum y_i, \sum x_i$ 最小. 找出距离线段最远的点 C , 则 C 也在下凸壳上, C 点满足 $AB \times AC$ 最小, 也即

$$(X_B - X_A)Y_C + (Y_A - Y_B)X_C - (X_B - X_A)Y_A - (Y_B - Y_A)X_A$$

最小, 后两项均为常数, 因此将所以权值改成 $(X_B - X_A)y_i + (Y_B - Y_A)x_i$, 求同样问题 (例如最小生成树, 最小权匹配) 即可. 求出 C 点以后, 递归 AC, BC .

2.26.2 最小环

无向图最小环: 每次 floyd 到 k 时, 判断 1 到 $k-1$ 的每一个 i, j :

$$\text{ans} = \min\{\text{ans}, d(i, j) + G(i, k) + G(k, j)\}.$$

有向图最小环: 做完 floyd 后, $d(i, i)$ 即为经过 i 的最小环.

2.26.3 度序列的可图性

判断一个度序列是否可转化为简单图, 除了一种贪心构造的方法外, 下列方法更快速. EG 定理: 将度序列从大到小排序得到 $\{d_i\}$, 此序列可转化为简单图当且仅当 $\sum d_i$ 为偶数, 且对于任意的 $1 \leq k \leq n-1$ 满足 $\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(k, d_i)$.

2.26.4 切比雪夫距离与曼哈顿距离转化

曼哈顿转切比雪夫: $(x+y, x-y)$, 适用于一些每次只能向四联通的格子走一格的问题. 切比雪夫转曼哈顿: $(\frac{x+y}{2}, \frac{x-y}{2})$, 适用于统计距离.

2.26.5 树链的交

```

1 bool cmp(int a,int b){return dep[a]<dep[b];}
2 path merge(path u, path v){
3   | int d[4], c[2];
4   | if (!u.x||!v.x) return path(0, 0);
5   | d[0]=lca(u.x,v.x); d[1]=lca(u.x,v.y);
6   | d[2]=lca(u.y,v.x); d[3]=lca(u.y,v.y);
7   | c[0]=lca(u.x,u.y); c[1]=lca(v.x,v.y);
8   | sort(d,d+4,cmp); sort(c,c+2,cmp);
9   | if (dep[c[0]] <= dep[d[0]] && dep[c[1]] <= dep[d[2]])
10  |   | return path(d[2],d[3]);
11  | else return path(0, 0); }

```

2.26.6 带修改 MST

维护少量修改的 MST (银川 21: 求有 16 个 'e1 or e2' 的限制条件的 MST)

找出必须边将修改边标 $-\infty$, 在 MST 上的其余边为必须边, 以此缩点.

找出无用边将修改边标 ∞ , 不在 MST 上的其余边为无用边, 删除之.

假设修改边数为 k , 操作后图中最多剩下 $k + 1$ 个点和 $2k$ 条边.

2.26.7 差分约束

$x_r - x_l \leq c : \text{add}(l, r, c)$ $x_r - x_l \geq c : \text{add}(r, l, -c)$

2.26.8 Segment Tree Beats

区间 $\min$, 区间求和. 维护最大值 m , 严格次大值 s 以及最大值个数 t . 现在假设我们要让区间 $[L, R]$ 对 x 取 $\min$, 先在线段树中定位若干个节点, 对于每个节点分三种情况讨论: 1, 当 $m \leq x$ 时直接退出; 2, 当 $se < x < ma$ 时, 只会影响到所有最大值, 所以把 num 加上 $t * (x - ma)$, 把 ma 更新为 x , 打上标记退出; 3, 当 $se \geq x$ 时递归. 均摊 $O(\log^2 n)$.

2.26.9 二分图

最小点覆盖 = 最大匹配数. 独立集与覆盖集互补. 最小点覆盖构造方法: 对二分图流图求割集, 跨过的边指示最小点覆盖. Hall 定理 $G = (X, Y, E)$, $|M| = |X| \Leftrightarrow \forall S \subseteq X, |S| \leq |A(S)|$.

2.26.10 稳定婚姻问题

男士按自己喜欢程度从高到底依次向每位女士求婚, 女士遇到更喜欢的男士时就接受他, 并抛弃以前的配偶. 被抛弃的男士继续按照列表向剩下的女士依次求婚, 直到所有人都有配偶. 算法一定能得到一个匹配, 而且这个匹配一定是稳定的. 时间复杂度 $O(n^2)$.

2.26.11 竞赛图 Landau's Theorem

n 个点竞赛图点按出度按升序排序, 前 i 个点的出度之和不小于 $\frac{i(i-1)}{2}$, 度数总和等于 $\frac{n(n-1)}{2}$. 否则可以用优先队列构造出方案.

2.26.12 Ramsey Theorem $R(3,3)=6$, $R(4,4)=18$

6 个人中存在 3 人相互认识或者相互不认识.

2.26.13 树的计数 Prufer 序列

prufer 编码长度为 $n - 2$, 且度数为 d_i 的点在 prufer 编码中出现 $d_i - 1$ 次.

由树得到序列: 总共需要 $n - 2$ 步, 第 i 步在当前的树中寻找具有最小标号的叶子节点, 将与其相连的点的标号设为 Prufer 序列的第 i 个元素 p_i , 并将此叶子节点从树中删除, 直到最后得到一个长度为 $n - 2$ 的 Prufer 序列和一个只有两个节点的树.

由序列得到树: 先将所有点的度赋初值为 1, 然后加上它的编号在 Prufer 序列中出现的次数, 得到每个点的度; 执行 $n - 2$ 步, 第 i 步选取具有最小标号的度为 1 的点 u 与 $v = p_i$ 相连, 得到树中的一条边, 并将 u 和 v 的度减一. 最后再把剩下的两个度为 1 的点连边, 加入到树中.

相关结论: n 个点完全图, 每个点度数依次为 $d_1, d_2, \dots, d_n$, 这样生成树的棵树为: $\frac{(n-2)!}{(d_1-1)!(d_2-1)!\dots(d_n-1)!}$.

左边有 n_1 个点, 右边有 n_2 个点的完全二分图的生成树棵树为 $n_1^{n_2-1} \times n_2^{n_1-1}$.

m 个连通块, 每个连通块有 c_i 个点, 把他们全部连通的生成树方案数: $(\sum c_i)^{m-2} \prod c_i$

2.26.14 有根树计数 1,1,2,4,9,20,48,115,286,719,1842,4766

无标号 $a_{n+1} = 1/n \sum_{k=1}^n (\sum_{d|k} d \cdot a(d)) \cdot a(n-k+1)$

2.26.15 无根树计数

n 是奇数时, 有 $a_n - \sum_i^{n/2} a_i a_{n-i}$ 种不同的无根树.

n 是偶数时, 有 $a_n - \sum_i^{n/2} a_i a_{n-i} + \frac{1}{2} a_{n/2} (a_{n/2} + 1)$ 种不同的无根树.

2.26.16 生成树计数 Kirchhoff's Matrix-Tree Theorem

Kirchhoff Matrix $T = \text{Deg} - A$, Deg 是度数对角阵, A 是邻接矩阵. 无向图度数矩阵是每个点度数; 有向图度数矩阵是每个点入度.

邻接矩阵 $A[u][v]$ 表示 $u \rightarrow v$ 边个数, 重边按照边数计算, 自环不计入度数.

无向图生成树计数: $c = |K|$ 的任意 1 个 $n - 1$ 阶主子式

有向图外向树计数: $c = |$ 去掉根所在的那阶得到的主子式

2.26.17 有向图欧拉回路计数 BEST Theorem

$$ec(G) = t_w(G) \prod_{v \in V} (\deg(v) - 1)!$$

其中 $\deg$ 为入度 (欧拉图中等于出度), $t_w(G)$ 为以 w 为根的外向树的个数. 相关计算参考生成树计数.

欧拉连通图中任意两点外向树个数相同: $t_v(G) = t_w(G)$.

2.26.18 Tutte Matrix

Tutte matrix A of a graph $G = (V, E)$:

$$A_{ij} = \begin{cases} x_{ij} & \text{if } (i, j) \in E \text{ and } i < j \\ -x_{ij} & \text{if } (i, j) \in E \text{ and } i > j \\ 0 & \text{otherwise} \end{cases}$$

where x_{ij} are indeterminates. The determinant of this skew-symmetric matrix is then a polynomial (in the variables x_{ij} , $i < j$): this coincides with the square of the pfaffian of the matrix A and is non-zero (as a polynomial) if and only if a perfect matching exists.

2.26.19 Edmonds Matrix

Edmonds matrix A of a balanced ($|U| = |V|$) bipartite graph $G = (U, V, E)$:

$$A_{ij} = \begin{cases} x_{ij} & (u_i, v_j) \in E \\ 0 & (u_i, v_j) \notin E \end{cases}$$

where the x_{ij} are indeterminates. G 有完美匹配当且仅当关于 x_{ij} 的多项式 $\det(A_{ij})$ 不恒为 0. 完美匹配的个数等于多项式中单项式的个数.

2.26.20 有向图无环定向, 色多项式

图的色多项式 $P_G(q)$ 对图 G 的 q -染色计数.

Triangle K_3 : $x(x-1)(x-2)$

Complete graph K_n : $x(x-1)(x-2) \cdots (x-(n-1))$

Tree with n vertices : $x(x-1)^{n-1}$

Cycle C_n : $(x-1)^n + (-1)^n(x-1)$

acyclic orientations of an n -vertex graph G is $(-1)^n P_G(-1)$.

2.26.21 拟阵交问题

拟阵定义: S, T 是独立集, 则 S 子集是, 若 $|S| > |T|$, 则 S 能扩充 T . 最大带权拟阵交问题: 全集 U 中每个元素都有权值 w_i . 设同一个全集 U 上有两个满足拟阵性质的集族 $\mathcal{F}_1, \mathcal{F}_2$. 对于 $k = 1..|U|$, 分别求出一个集合 S , 满足 $S \in \mathcal{F}_1 \cap \mathcal{F}_2$ 且 $|S|$ 恰好为 k 的前提下, S 中元素权值和最小.

设集合大小为 k 时已经求出了答案 S . 现在希望求出集合大小为 $k+1$ 的答案. U 中所有元素分为两个集合: 当前答案集合 S , 和剩余集合 $T = U \setminus S$. 考虑 T 中的某个元素 x_i . 记 $A = \{x_i | S \cup \{x_i\} \in \mathcal{F}_1\}$, $B = \{x_i | S \cup \{x_i\} \in \mathcal{F}_2\}$. 如果 T 中某个元素 $x_i \notin A$, 说明 x_i 加进 S 中形成了某个“环”, 从而不满足 $\mathcal{F}_1$ 的限制. 考虑这个“环”上每个元素 y_j , 满足 $S \setminus \{y_j\} \cup \{x_i\} \in \mathcal{F}_1$, 将 y_j 向每个 x_i 连边. 如果 T 中某个元素 $x_i \notin B$, 同理找出 S 中每一个元素 y_j 使得 $S \setminus \{y_j\} \cup \{x_i\} \in \mathcal{F}_2$, 将 x_i 向 y_j 连边. 现在求出从 A 到 B 的多源多汇最短路, 权值在点上, 若点属于 T 则权值为正, 否则属于 S , 权值为负. 最短路上的每个 T 中的点放进 S , S

中的点放进 T , 则完成了一次增广. 由于每次增广路的起点和终点都在 T 中, 所以每次增广都会使得 $|S|$ 增加 1.

最大拟阵交问题可以去掉权值直接求增广路.

2.26.22 双极定向

```

1 //双极定向: 给定无向图和两个极点 s,t, 要求将每条边定向后成为 DAG, 使得 s 可达所有点, 所
  ↳ 有点均可达 t
2 //topo 为定向后 DAG 的拓扑序, 边 (u,v) 定向为 u->v 当且仅当拓扑序中 u 在 v 的前面.
3 int n, dfn[N], low[N], stamp, p[N], preorder[N], topo[N];
4 bool fucked = 0, sign[N]; vector<int> G[N];
5 void dfs(int x, int fa, int s, int t){
6     | dfn[x] = low[x] = ++stamp;
7     | preorder[stamp] = x, p[x] = fa;
8     | if (x == s) dfs(t, x, s, t);
9     | for (int y : G[x]){
10        |     | if (x == s && y == t) continue;
11        |     | if (!dfn[y]){
12        |         | if (x == s) fucked = true;
13        |         | dfs(y, x, s, t);
14        |         | low[x] = min(low[x], low[y]); }
15        |     | else if (dfn[y] < dfn[x] && y != fa)
16        |         | low[x] = min(low[x], dfn[y]); } }
17 bool bipolar_orientation(int s, int t){
18     | G[s].push_back(t), G[t].push_back(s);
19     | stamp = fucked = 0, dfs(s, s, s, t);
20     | for (int i = 1; i <= n; i++)
21     |     | if (i != s && (!dfn[i] || low[i] >= dfn[i]))
22     |         | fucked = true;
23     | if (fucked) return false;
24     | sign[s] = 0; //memset sign[] is not necessary
25     | int pre[n + 5], suf[n + 5]; // list
26     | suf[0] = s; pre[s] = 0, suf[s] = t;
27     | pre[t] = s, suf[t] = n + 1; pre[n + 1] = t;
28     | for (int i = 3; i <= n; i++){
29     |     | int v = preorder[i];
30     |     | if (!sign[preorder[low[v]]]){ // insert before p[v]
31     |         | int P = pre[p[v]];
32     |         | pre[v] = P, suf[v] = p[v];
33     |         | suf[P] = pre[p[v]] = v; }
34     |     | else{ // insert after p[v]
35     |         | int S = suf[p[v]];
36     |         | pre[v] = p[v], suf[x] = S;
37     |         | suf[p[v]] = pre[S] = v; }
38     |     | sign[p[x]] = !sign[preorder[low[x]]]; }
39     | for (int x = s, cnt = 0; x != n + 1; x = suf[x])
40     |     | topo[++cnt] = x;
41     | return true; }

```

2.26.23 图中的环

没有奇环的图是**二分图**, 没有偶环的图是**仙人掌**. 判定没有奇环仅用深度奇偶性判即可; 判定没有偶环的图需要记录覆盖次数判定是否存在奇环有交.

3. Data Structure

3.1 回滚并查集

```
1 struct DSU {
2     std::vector<int> siz;
3     std::vector<int> f;
4     std::vector<std::array<int, 2>> his;
5
6     DSU() {}
7     DSU(int n) { init(n); }
8
9     void init(int n) {
10         f.resize(n);
11         std::iota(f.begin(), f.end(), 0);
12         siz.assign(n, 1);
13     }
14
15     bool same(int x, int y) { return find(x) == find(y); }
16
17     int find(int x) {
18         while (f[x] != x) {
19             x = f[x];
20         }
21         return x;
22     }
23
24     bool merge(int x, int y) {
25         x = find(x);
26         y = find(y);
27         if (x == y) {
28             return false;
29         }
30         if (siz[x] < siz[y]) {
31             std::swap(x, y);
32         }
33         his.push_back({x, y});
34         siz[x] += siz[y];
35         f[y] = x;
36         return true;
37     }
38
39     int time() { return his.size(); }
40
41     void revert(int tm) {
42         while (his.size() > tm) {
43             auto [x, y] = his.back();
44             his.pop_back();
45             f[y] = y;
46             siz[x] -= siz[y];
47         }
48     }
49 };
```

3.2 莫队合集

3.2.1 普通莫队

```

1 struct Q {
2     int l, r, id;
3 } q[N];
4 bool cmp(Q x, Q y) {
5     if (pos[x.l] == pos[y.l])
6         return x.r < y.r;
7     else
8         return pos[x.l] < pos[y.l];
9 }
10 sort(q + 1, q + 1 + m);
11 for (int i = 1, l = 1, r = 0; i <= m; ++i) {
12     while (l > q[i].l) add(a[--l]); // 左扩展
13     while (r < q[i].r) add(a[++r]); // 右扩展
14     while (l < q[i].l) del(a[l++]); // 左删除
15     while (r > q[i].r) del(a[r--]); // 右删除
16 }

```

3.2.2 树上莫队

```

1 // 预处理部分
2 void dfs2(int u, int t) {
3     top[u] = t;
4     a[++tim] = u, in[u] = tim;
5     if (son[u]) dfs2(son[u], t);
6     for (auto v : e[u]) {
7         if (v == fa[u] || v == son[u]) continue;
8         dfs2(v, v);
9     }
10    a[++tim] = u, out[u] = tim;
11 }
12
13 // 处理询问部分
14 int lc = lca(x, y);
15 q[++tot].t = t;
16 q[tot].id = tot;
17 if (in[x] > in[y]) swap(x, y); // 先 x 后 y
18 if (lca == x) {                // 直链情况
19     q[tot].l = in[x];
20     q[tot].r = in[y];
21 } else { // 折链情况
22     q[tot].l = out[x];
23     q[tot].r = in[y];
24     q[tot].lca = lc; // 以便补偿
25 }

```

3.2.3 带修莫队

单点修改, 区间查询颜色个数模板题, 块长为 $n^{2/3}$, 时间复杂度为 $n^{3/5}$

```

1 #include <bits/stdc++.h>
2 #define ll long long
3
4 using namespace std;
5
6 const int N = 2e5 + 10, M = 1e6 + 10;

```

```
7
8 struct Q {
9     int l, r, t, id;
10 } q[N], qt[N];
11 /*
12 q 表示普通查询
13 qt 表示修改操作
14 */
15
16 int n, m, tot, c[N], B, ti;
17 int cnt[M], res, ans[N];
18 inline bool cmp(Q a, Q b) {
19     if (a.l / B == b.l / B) {
20         if (a.r / B == b.r / B) {
21             return a.t < b.t;
22         }
23         return a.r / B < b.r / B;
24     }
25     return a.l < b.l;
26 }
27
28 int l = 1, r = 0, t = 0;
29 inline void move(int x, int t) {
30     cnt[c[x]] += t;
31     if (cnt[c[x]] == 0) res--;
32     if (cnt[c[x]] == t) res++;
33 }
34
35 inline void flow(int t) {
36     if (!t) return;
37     if (qt[t].l >= 1 && qt[t].l <= r) {
38         if (--cnt[c[qt[t].l]] == 0) res--;
39         if (++cnt[qt[t].r] == 1) res++;
40     }
41     swap(c[qt[t].l], qt[t].r);
42 }
43
44 void solve() {
45     scanf("%d%d", &n, &m);
46     for (int i = 1; i <= n; i++) {
47         scanf("%d", c + i);
48     }
49     for (int i = 1; i <= m; i++) {
50         char op[2];
51         int l, r;
52         cin >> op >> l >> r;
53         if (op[0] == 'Q') {
54             q[tot].id = tot, q[tot].t = ti, q[tot].l = l, q[tot++].r = r;
55         } else {
56             qt[++ti].l = l, qt[ti].r = r;
57         }
58     }
59     B = pow(n, 0.666);
60     sort(q, q + tot, cmp);
61     for (int i = 0; i < tot; i++) {
62         while (q[i].t > t) flow(++t);
63         while (q[i].t < t) flow(t--);
64         while (q[i].l < 1) move(--l, 1);
65         while (q[i].r > r) move(++r, 1);
66         while (q[i].l > 1) move(l++, -1);
```

```

67     while (q[i].r < r) move(r--, -1);
68     ans[q[i].id] = res;
69 }
70 for (int i = 0; i < tot; i++) printf("%d\n", ans[i]);
71 }
72
73 int main() {
74     int t = 1;
75     while (t--) solve();
76     return 0;
77 }

```

3.2.4 回滚莫队

当区间的加点操作易于实现，而删点操作难以实现时，使用此种算法。

询问排序 首先对序列进行分块，块大小通常取 $\sqrt{n}$ 。然后将所有询问按下述规则排序：

- 以左端点 l 所在的块编号为第一关键字，升序排序。
- 以右端点 r 为第二关键字，升序排序。

指针移动 我们按块的顺序处理所有询问。

1. **同块内询问**：如果一个询问的左右端点 l, r 在同一个块内，由于块长不超过 $\sqrt{n}$ ，直接暴力扫描该区间计算答案即可。
2. **跨块询问**：对于左端点在块 T 内的所有询问，我们进行如下操作：
 - 首先将莫队区间的左端点 l 初始化为块 T 的右边界加一 ($R[T] + 1$)，右端点 r 初始化为 $R[T]$ 。此时区间为空。
 - **移动右指针**：由于同一块内的询问是按 r 升序排列的，我们可以只向右移动右指针 r ，不断加点，直到满足当前询问的 r 。对于整个块 T 的所有询问，指针 r 只会单调向右移动。
 - **移动左指针**：对于每一个询问，我们从 $R[T] + 1$ 出发，将左指针 l 向左移动到询问所需的位置，这个过程只涉及加点操作。移动距离不会超过块的大小。
 - **回答与回滚**：回答当前询问后，我们需要撤销刚才左指针的所有移动，让 l 回滚到 $R[T] + 1$ 的位置，以恢复基准状态，准备下一个询问。这个回滚操作只撤销数据结构的变化，而不需要更新答案。
3. 处理完一个块内的所有询问后，按照相同方式处理下一个块。

只删不加 当删点操作易于实现，而加点操作困难时使用。其流程与只加不减对称。

- **排序**：左端点所在块升序，右端点降序。
- **指针移动**：对于左端点在块 T 内的询问，将莫队区间初始化为 $[L[T], n]$ 。右指针 r 只会单调向左移动（删点），左指针 l 从 $L[T]$ 临时向右移动（删点），回答后再回滚。

```

1 struct Q {
2     int l, r, id;
3 };
4
5 bool cmp(const Q& a, const Q& b) {
6     if (be[a.l] != be[b.l]) {
7         return be[a.l] < be[b.l];

```

```

8     }
9     return a.r < b.r;
10 }
11
12 void solve() {
13
14     for (const auto& query : q) {
15         // 情况 1: 左右端点在同一个块内, 直接暴力求解
16         if (belo[query.l] == belo[query.r]) {
17             continue;
18         }
19
20         // 情况 2: 左端点进入了一个新的块, 重置所有状态和指针
21         if (be[query.l] != last) {
22             last = be[query.l];
23             // 将 r 和 l 重置到当前块的右边界
24             r = R[last];
25             l = r + 1;
26         }
27
28         // 移动右指针
29         while (r < query.r) {
30             add(++r);
31         }
32
33         // 在移动左指针前, 保存当前 [l, r] 区间的答案
34         int pre = l; // 记录移动前左指针的位置, 便于回滚
35
36         // 临时移动左指针
37         while (l > query.l) {
38             add(--l);
39         }
40
41         // 计算答案
42
43         // 回滚左指针的移动, 恢复到 [pre, r] 的状态
44         while (l < pre) {
45             del(l++);
46         }
47         // 恢复之前保存的状态变量
48     }
49 }

```

3.2.5 二次离线莫队

给你一个长为 n 的序列 a , m 次询问, 每次查询一个区间的逆序对数。

数据范围: $1 \leq n, m \leq 10^5$, $0 \leq a_i \leq 10^9$ 。

直接莫队每次转移至少是 $O(\log n)$ 的, 观察可以发现我们每次转移要求的信息是「一个数在某个区间内的排名」, 而这一信息关于区间可以差分, 因此考虑二次离线。

我们将 $[l, r]$ 拓展到 $[l, r + 1]$, 要求的是在 $[l, r]$ 区间内有多少比 a_{r+1} 大的数。我们用 $f(x, r)$ 表示在 $[1, r]$ 区间内比 a_x 大的数, $g(x, r)$ 表示 $[1, r]$ 区间内比 a_x 小的数。则答案的变化量可以写作 $f(r + 1, r) - f(r + 1, l - 1)$ 。

其余指针移动同理。

对于这些式子中的 $f(x, x - 1)$ 和 $g(x, x - 1)$, 我们可以使用树状数组提前 $O(n \log n)$ 预处理出来; 对于其余的 $f(x, p)$ 和 $g(x, p)$ 项, 我们将 x 离线存在 l 或 r 进行批量处理。

这里有一个优化空间的技巧：我们发现处理每个询问时，离线到 p 上的 x 都是连续的一段，因此我们不需要把每次移动都存下，只需要存下调整的一段即可。这样我们的空间可以从总次数 $O(n\sqrt{m})$ 下降到询问数 $O(m)$ 。

现在我们来处理我们二次离线下来的问题：向一个集合中加入数，查询一个数在集合中的排名。莫队总共移动端点的次数是 $O(n\sqrt{m})$ 的，而数组总共只有 $O(n)$ 的长度，因此我们考虑使用 $O(\sqrt{n})$ 插入、 $O(1)$ 查询的值域分块解决这个问题。

至此，我们在 $O(n\sqrt{m} + n\sqrt{n})$ 的时间复杂度和 $O(n + m)$ 的空间复杂度下解决了此题。

最后值得注意的是，我们求得的是每次的答案变化量而非答案本身，需要对其进行前缀和才能得到最终的答案。

总结就是，我们首先把询问离线到莫队上面，然后我们不直接利用莫队计算答案，而是把进行莫队过程的答案的变化量离线下来计算。

```

1 #include <bits/stdc++.h>
2
3 #define ll long long
4 #define lowbit(x) (x & (-x))
5 using namespace std;
6 std::mt19937 rng(std::chrono::steady_clock::now().time_since_epoch().count());
7
8 const int N = 1e5 + 10;
9
10 int n, m, a[N], B, s[N];
11 int l[510], r[510], pos[N];
12
13 void add(int x, int t) {
14     while (x <= n) {
15         s[x] += t;
16         x += lowbit(x);
17     }
18 }
19
20 int query(int x) {
21     int res = 0;
22     while (x) {
23         res += s[x];
24         x -= lowbit(x);
25     }
26     return res;
27 }
28
29 struct Q {
30     int l, r, id;
31     bool operator<(const Q &W) const {
32         if (pos[l] != pos[W.l])
33             return pos[l] < pos[W.l];
34         else
35             return r < W.r;
36     }
37 } q[N];
38
39 struct seg {
40     int x, y, op, id;
41 };
42
43 vector<seg> lq[N], rq[N];
44
45 ll ans[N], g[N], f[N], tag[510], val[N], cnt[N];

```

```
46
47 void solve() {
48     cin >> n >> m;
49     vector<int> v;
50     for (int i = 1; i <= n; i++) cin >> a[i], v.push_back(a[i]);
51
52     sort(v.begin(), v.end());
53     v.erase(unique(v.begin(), v.end()), v.end());
54     for (int i = 1; i <= n; i++)
55         a[i] = lower_bound(v.begin(), v.end(), a[i]) - v.begin() + 1;
56
57     B = sqrt(n * 2);
58     int tot = (n - 1) / B + 1;
59     for (int i = 1; i <= tot; i++) l[i] = r[i - 1] + 1, r[i] = min(n, i * B);
60     for (int i = 1; i <= tot; i++) {
61         for (int j = l[i]; j <= r[i]; j++) pos[j] = i;
62     }
63     for (int i = 1; i <= m; i++) {
64         cin >> q[i].l >> q[i].r;
65         q[i].id = i;
66     }
67
68     for (int i = 1; i <= n; i++) {
69         add(a[i], 1);
70         f[i] = query(n) - query(a[i]);
71         g[i] = query(a[i] - 1);
72         f[i] += f[i - 1];
73         g[i] += g[i - 1];
74     }
75     sort(q + 1, q + m + 1);
76
77     for (int i = 1, l = 1, r = 0; i <= m; i++) {
78         int id = q[i].id;
79         if (l > q[i].l) {
80             // l->q[i].l
81             // [l,r] 中比 a[x] 小的 g(l,r,a[x])
82             // g(1,r) - g(1,l-1) l-1
83             ans[id] += g[q[i].l - 1] - g[l - 1];
84             rq[r].push_back({q[i].l, l - 1, 1, id});
85             l = q[i].l;
86         }
87
88         if (r < q[i].r) {
89             // [l,r] 中比 a[x] 大的
90             // f(1,r) - f(1,l-1)
91             ans[id] += f[q[i].r] - f[r];
92             lq[l - 1].push_back({r + 1, q[i].r, -1, id});
93             r = q[i].r;
94         }
95         if (l < q[i].l) {
96             ans[id] += g[q[i].l - 1] - g[l - 1];
97             rq[r].push_back({l, q[i].l - 1, -1, id});
98             l = q[i].l;
99         }
100        if (r > q[i].r) {
101            ans[id] += f[q[i].r] - f[r];
102            lq[l - 1].push_back({q[i].r + 1, r, 1, id});
103            r = q[i].r;
104        }
105    }
```

```

106
107     for (int i = 1; i <= n; i++) {
108         cnt[a[i]]++;
109         for (int j = pos[a[i]] + 1; j <= tot; j++) tag[j]++;
110         for (int j = a[i] + 1; j <= r[pos[a[i]]]; j++) val[j]++;
111         for (auto [x, y, op, id] : rq[i]) {
112             for (int j = x; j <= y; j++)
113                 ans[id] += (val[a[j]] + tag[pos[a[j]]]) * op;
114         }
115         for (auto [x, y, op, id] : lq[i]) {
116             for (int j = x; j <= y; j++) {
117                 ans[id] += (i - (val[a[j]] + tag[pos[a[j]]]) - cnt[a[j]]) * op;
118             }
119         }
120     }
121     q[0].id = 0;
122     for (int i = 1; i <= m; i++) ans[q[i].id] += ans[q[i - 1].id];
123     for (int i = 1; i <= m; i++) cout << ans[i] << '\n';
124 }
125
126 int main() {
127     ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
128     int t = 1;
129     while (t--) solve();
130     return 0;
131 }

```

3.3 树状数组

```

1  template <typename T>
2  struct Fenwick {
3      int n;
4      vector<T> a;
5
6      Fenwick(int n_ = 0) { init(n_); }
7
8      void init(int n_) {
9          n = n_;
10         a.assign(n, T{});
11     }
12
13     void add(int x, const T &v) {
14         for (int i = x + 1; i <= n; i += i & -i) {
15             a[i - 1] = a[i - 1] + v;
16         }
17     }
18
19     // [0, x)
20     T sum(int x) {
21         T ans{};
22         for (int i = x; i > 0; i -= i & -i) {
23             ans = ans + a[i - 1];
24         }
25         return ans;
26     }
27
28     // [l, r)
29     T query(int l, int r) { return sum(r) - sum(l); }
30

```

```
31 // 返回 sum(x) 小于等于 k 的最大的 x
32 int select(const T &k) {
33     int x = 0;
34     T cur{};
35     for (int i = 1 << __lg(n); i; i /= 2) {
36         if (x + i <= n && cur + a[x + i - 1] <= k) {
37             x += i;
38             cur = cur + a[x - 1];
39         }
40     }
41     return x;
42 }
43 };
```

3.4 单点修改线段树

```
1 #include<bits/stdc++.h>
2
3 template <class Info>
4 struct SegmentTree {
5     int n;
6
7     std::vector<Info> info;
8     SegmentTree() : n(0) {}
9     SegmentTree(int n_, Info v_ = Info()) { init(n_, v_); }
10    template <class T>
11    SegmentTree(std::vector<T> init_) {
12        init(init_);
13    }
14    void init(int n_, Info v_ = Info()) { init(std::vector(n_, v_)); }
15    template <class T>
16    void init(std::vector<T> init_) {
17        n = init_.size();
18        info.assign(4 << std::__lg(n), Info());
19        std::function<void(int, int, int)> build = [&](int p, int l, int r) {
20            if (r - l == 1) {
21                info[p] = init_[l];
22                return;
23            }
24            int m = (l + r) / 2;
25            build(2 * p, l, m);
26            build(2 * p + 1, m, r);
27            pull(p);
28        };
29        build(1, 0, n);
30    }
31    void pull(int p) { info[p] = info[2 * p] + info[2 * p + 1]; }
32    void modify(int p, int l, int r, int x, const Info &v) {
33        if (r - l == 1) {
34            info[p] = v;
35            return;
36        }
37        int m = (l + r) / 2;
38        if (x < m) {
39            modify(2 * p, l, m, x, v);
40        } else {
41            modify(2 * p + 1, m, r, x, v);
42        }
43        pull(p);
44    }
```

```
44     }
45     void modify(int p, const Info &v) { modify(1, 0, n, p, v); }
46     Info rangeQuery(int p, int l, int r, int x, int y) {
47         if (l >= y || r <= x) {
48             return Info();
49         }
50         if (l >= x && r <= y) {
51             return info[p];
52         }
53         int m = (l + r) / 2;
54         return rangeQuery(2 * p, l, m, x, y) +
55             rangeQuery(2 * p + 1, m, r, x, y);
56     }
57     Info rangeQuery(int l, int r) { return rangeQuery(1, 0, n, l, r); }
58
59     /**
60      * @brief 在线段树上二分，查找区间 [x, y) 中第一个满足 pred 条件的元素位置。
61      * * @tparam F 谓词函数类型，应当是 bool(const Info&)
62      * @param pred 谓词函数，如果当前区间的合并信息满足条件，则返回 true
63      * @return int 第一个满足条件的位置索引，如果不存在则返回 -1
64      */
65     template <class F>
66     int findFirst(int p, int l, int r, int x, int y, F &&pred) {
67         // 如果当前区间 [l, r) 与查询区间 [x, y) 没有交集，直接返回
68         if (l >= y || r <= x) {
69             return -1;
70         }
71         // 如果当前区间被查询区间完全包含，并且整个区间的信息都不满足谓词，
72         // 说明这个区间内不可能有答案，进行剪枝
73         if (l >= x && r <= y && !pred(info[p])) {
74             return -1;
75         }
76         // 如果到达叶子节点，说明找到了一个满足条件的最小单位，返回其位置
77         if (r - l == 1) {
78             return l;
79         }
80         int m = (l + r) / 2;
81         // 优先在左子树中查找，因为要找 " 第一个"
82         int res = findFirst(2 * p, l, m, x, y, pred);
83         // 如果左子树没找到，再去右子树中查找
84         if (res == -1) {
85             res = findFirst(2 * p + 1, m, r, x, y, pred);
86         }
87         return res;
88     }
89     template <class F>
90     int findFirst(int l, int r, F &&pred) {
91         return findFirst(1, 0, n, l, r, pred);
92     }
93
94     /**
95      * @brief 在线段树上二分，查找区间 [x, y) 中最后一个满足 pred
96      * 条件的元素位置。
97      * * @tparam F 谓词函数类型，应当是 bool(const Info&)
98      * @param pred 谓词函数，如果当前区间的合并信息满足条件，则返回 true
99      * @return int 最后一个满足条件的位置索引，如果不存在则返回 -1
100     */
101     template <class F>
102     int findLast(int p, int l, int r, int x, int y, F &&pred) {
103         // 如果当前区间 [l, r) 与查询区间 [x, y) 没有交集，直接返回
```

```

104     if (l >= y || r <= x) {
105         return -1;
106     }
107     // 如果当前区间被查询区间完全包含, 并且整个区间的信息都不满足谓词,
108     // 说明这个区间内不可能有答案, 进行剪枝
109     if (l >= x && r <= y && !pred(info[p])) {
110         return -1;
111     }
112     // 如果到达叶子节点, 说明找到了一个满足条件的最小单位, 返回其位置
113     if (r - l == 1) {
114         return l;
115     }
116     int m = (l + r) / 2;
117     // 优先在右子树中查找, 因为要找 " 最后一个"
118     int res = findLast(2 * p + 1, m, r, x, y, pred);
119     // 如果右子树没找到, 再去左子树中查找
120     if (res == -1) {
121         res = findLast(2 * p, l, m, x, y, pred);
122     }
123     return res;
124 }
125 template <class F>
126 int findLast(int l, int r, F &&pred) {
127     return findLast(1, 0, n, l, r, pred);
128 }
129 };
130
131 struct Info {
132     int x = 0;
133 };
134
135 // 区间最大值
136 Info operator+(const Info &a, const Info &b) { return {std::max(a.x, b.x)}; }
137
138 int main() {
139
140     std::vector<Info> a = {{1}, {5}, {2}, {8}, {6}, {7}, {4}, {9}};
141     SegmentTree<Info> tr(a);
142
143     // 在一个子区间 [0, 5) 中查找第一个值 >= 6 的元素
144     int val = 6;
145     int pos = tr.findFirst(0, 5, [&](const Info& v) {
146         return v.x >= val;
147     });
148
149     return 0;
150 }

```

3.5 懒标记线段树

```

1 template <class Info, class Tag>
2 struct LazySegmentTree {
3     int n;
4     vector<Info> info;
5     vector<Tag> tag;
6     LazySegmentTree() : n(0) {}
7     LazySegmentTree(int n_, Info v_ = Info()) {
8         init(n_, v_);
9     } // 初始为同一种结点

```

```
10  template <class T>
11  LazySegmentTree(vector<T> init_) {
12      init(init_);
13  } // 针对序列初始化
14
15  void init(int n_, Info v_ = Info()) { init(vector(n_, v_)); }
16  template <class T>
17  void init(vector<T> init_) {
18      n = init_.size();
19      info.assign(4 << __lg(n), Info());
20      tag.assign(4 << __lg(n), Tag());
21      // 结点信息为左闭右开
22      function<void(int, int, int)> build = [&](int p, int l, int r) {
23          if (r - l == 1) {
24              info[p] = init_[l];
25              return;
26          }
27          int m = (l + r) / 2;
28          build(2 * p, l, m), build(2 * p + 1, m, r);
29          pull(p);
30      };
31      build(1, 0, n);
32  }
33
34  void pull(int p) {
35      info[p] = info[2 * p] + info[2 * p + 1];
36  } // 向上合并结点
37  void apply(int p, const Tag &v) {
38      info[p].apply(v);
39      tag[p].apply(v);
40  }
41  void push(int p) {
42      apply(2 * p, tag[p]);
43      apply(2 * p + 1, tag[p]);
44      tag[p] = Tag();
45  } // 懒标记下传
46  void modify(int p, int l, int r, int x, const Info &v) {
47      if (r - l == 1) {
48          info[p] = v;
49          return;
50      }
51      int m = (l + r) / 2;
52      push(p);
53      if (x < m) {
54          modify(2 * p, l, m, x, v);
55      } else {
56          modify(2 * p + 1, m, r, x, v);
57      }
58      pull(p);
59  }
60  void modify(int p, const Info &v) { modify(1, 0, n, p, v); }
61  Info rangeQuery(int p, int l, int r, int x, int y) {
62      if (l >= y || r <= x) {
63          return Info();
64      }
65      if (l >= x && r <= y) {
66          return info[p];
67      }
68      int m = (l + r) / 2;
69      push(p);
```

```
70     return rangeQuery(2 * p, l, m, x, y) +
71         rangeQuery(2 * p + 1, m, r, x, y);
72 }
73 Info rangeQuery(int l, int r) { return rangeQuery(1, 0, n, l, r); }
74 void rangeApply(int p, int l, int r, int x, int y, const Tag &v) {
75     if (l >= y || r <= x) {
76         return;
77     }
78     if (l >= x && r <= y) {
79         apply(p, v);
80         return;
81     }
82     int m = (l + r) / 2;
83     push(p);
84     rangeApply(2 * p, l, m, x, y, v);
85     rangeApply(2 * p + 1, m, r, x, y, v);
86     pull(p);
87 }
88 void rangeApply(int l, int r, const Tag &v) {
89     return rangeApply(1, 0, n, l, r, v);
90 }
91
92 template <class F>
93 int findFirst(int p, int l, int r, int x, int y, F &&pred) {
94     if (l >= y || r <= x) {
95         return -1;
96     }
97     if (l >= x && r <= y && !pred(info[p])) {
98         return -1;
99     }
100     if (r - l == 1) {
101         return l;
102     }
103     int m = (l + r) / 2;
104     push(p);
105     int res = findFirst(2 * p, l, m, x, y, pred);
106     if (res == -1) {
107         res = findFirst(2 * p + 1, m, r, x, y, pred);
108     }
109     return res;
110 }
111 template <class F>
112 int findFirst(int l, int r, F &&pred) {
113     return findFirst(1, 0, n, l, r, pred);
114 }
115 template <class F>
116 int findLast(int p, int l, int r, int x, int y, F &&pred) {
117     if (l >= y || r <= x) {
118         return -1;
119     }
120     if (l >= x && r <= y && !pred(info[p])) {
121         return -1;
122     }
123     if (r - l == 1) {
124         return l;
125     }
126     int m = (l + r) / 2;
127     push(p);
128     int res = findLast(2 * p + 1, m, r, x, y, pred);
129     if (res == -1) {
```



```

130         res = findLast(2 * p, l, m, x, y, pred);
131     }
132     return res;
133 }
134 template <class F>
135 int findLast(int l, int r, F &&pred) {
136     return findLast(1, 0, n, l, r, pred);
137 }
138 };
139
140 struct Tag {
141     int x = 0;
142     void apply(const Tag &t) & { x = std::max(x, t.x); }
143 };
144
145 struct Info {
146     int x = 0;
147     void apply(const Tag &t) & { x = std::max(x, t.x); }
148 };
149
150 Info operator+(const Info &a, const Info &b) { return {std::max(a.x, b.x)}; }

```

3.6 线段树合并与分裂

```

1 int ln[M], rn[M], tot, cnt[M], rt[N];
2 // 合并 左闭右开
3 int merge(int u, int v, int l, int r) {
4     if (!u || !v) return u + v;
5     if (l == r - 1) {
6         cnt[u] += cnt[v];
7         return u;
8     }
9     int mid = (l + r) / 2;
10    ln[u] = merge(ln[u], ln[v], l, mid);
11    rn[u] = merge(rn[u], rn[v], mid, r);
12    cnt[u] = cnt[ln[u]] + cnt[rn[u]];
13    return u;
14 }
15
16 // 这里需要和主席树区别开, 不需要共享节点
17 void insert(int& u, int l, int r, int x) {
18     if (!u) u = ++tot;
19     cnt[u]++;
20     if (l == r - 1) return;
21     int mid = (l + r) / 2;
22     if (x < mid)
23         insert(ln[u], l, mid, x);
24     else
25         insert(rn[u], mid, r, x);
26 }
27
28 // op = 0 表示前 k 个给 x 后面的给 y
29 // op = 1 表示后 k 个给 x 前面的给 y
30 void split(int x, int& y, int k, int op) {
31     if (!x) return;
32     y = ++tot;
33     if (!op) {
34         int s = cnt[ln[x]];
35         if (s >= k)

```

```

36         split(ln[x], ln[y], k, op), swap(rn[x], rn[y]));
37     else
38         split(rn[x], rn[y], k - s, op);
39     cnt[y] = cnt[x] - k, cnt[x] = k;
40 } else {
41     int s = cnt[rn[x]];
42     if (s >= k)
43         split(rn[x], rn[y], k, op), swap(ln[x], ln[y]);
44     else
45         split(ln[x], ln[y], k - s, op);
46     cnt[y] = cnt[x] - k, cnt[x] = k;
47 }
48 }

```

3.7 笛卡尔树

```

1 struct tree {
2     int n, rt;
3     vector<int> ln, rn;
4
5     // 按照数组下标构成二叉搜索树
6     // 按照 w 构成小根堆。
7     void build(vector<int>& w) {
8         this->n = w.size();
9         ln.resize(n, -1), rn.resize(n, -1);
10
11         vector<int> stk;
12         for (int i = 0; i < n; i++) {
13             int last = -1;
14             // 如果要改成大根堆, 只需将下面的 > 改为 <
15             while (!stk.empty() && w[stk.back()] > w[i])
16                 last = stk.back(), stk.pop_back();
17             if (!stk.empty()) rn[stk.back()] = i;
18             if (last != -1) ln[i] = last;
19             stk.push_back(i);
20         }
21         rt = stk[0];
22     }
23 };

```

3.8 点分治

```

1 vector<int> e[N];
2 int vis[N], s[N], maxs[N] = {(int)1e9};
3 void getrt(int p, int fa, int siz, int &rt) {
4     s[p] = 1;
5     maxs[p] = 0;
6     for (auto [v, c] : e[p]) {
7         if (v == fa || vis[v]) continue;
8         s[p] += s[v];
9         maxs[p] = max(maxs[p], s[v]);
10    }
11    maxs[p] = max(maxs[p], siz - s[p]);
12    if (maxs[p] < maxs[rt]) rt = p;
13 }
14 void cal(int p, int siz) {
15     int rt = 0;
16     getrt(p, 0, siz, rt);

```

```

17     for (auto [v, c] : e[rt]) {
18         if (vis[v]) continue;
19         // 计算相关贡献
20     }
21     vis[rt] = 1;
22     for (auto [v, c] : e[rt]) {
23         if (vis[v]) continue;
24         if (s[v] > s[rt]) s[v] = siz - s[rt];
25         cal(v, s[v]);
26     }
27 }
28 // 调用: cal(1,n);

```

3.9 点分树

在点分治过程中, 对于每一个找到的重心, 将上一层分治时的重心设为它的父亲, 可以得到点分树。

显然点分树的树高是 $O(\log n)$ 的。

对于任意两点 x, y , 它们在点分树上的 lca 一定在原树的从 x 到 y 的简单路径上。

因此可以将固定某个点 u 为端点的简单路径按照 u 在点分树上的祖先划分为 $O(\log n)$ 个等价类去处理。

3.10 树上启发式合并

```

1 // 轻重链剖分的性质保证每个点只会被加入和删除 log 次
2
3 struct HLD {
4     vector<vector<int>> e;
5     vector<int> sz, son;
6     int hson; // 不需要被添加的重子树
7     HLD(int n) {
8         e.resize(n + 1);
9         sz.resize(n + 1);
10        son.resize(n + 1);
11    }
12
13    void addEdge(int u, int v) {
14        e[u].push_back(v);
15        e[v].push_back(u);
16    }
17
18    void dfs1(int u, int f) {
19        sz[u] = 1;
20        for (auto v : e[u]) {
21            if (v == f) continue;
22            dfs1(v, u);
23            sz[u] += sz[v];
24            if (sz[v] > sz[son[u]]) son[u] = v;
25        }
26    }
27
28    void add(int u, int f) {
29        /*
30        加点逻辑
31        */
32        for (auto v : e[u]) {
33            if (v == f || v == hson) continue;
34            add(v, u);
35        }
36    }

```

```
37
38 void sub(int u, int f) {
39     /*
40     删点逻辑。
41     */
42     for (auto v : e[u]) {
43         if (v == f) continue;
44         sub(v, u);
45     }
46 }
47
48 void dfs2(int u, int f, int op) {
49     for (auto v : e[u]) {
50         if (v == f || v == son[u]) continue;
51         dfs2(v, u, 0);
52     }
53     if (son[u]) {
54         dfs2(son[u], u, 1);
55     }
56     hson = son[u];
57     add(u, f);
58
59     /*
60     获取该点的答案。
61     */
62
63     if (!op) {
64         sub(u, f);
65     }
66 }
67
68 void work(int rt = 1) {
69     dfs1(rt, 0);
70     dfs2(rt, 0, 0);
71 }
72 };
```

3.11 $O(n \log n) - O(1)$ LCA

```
1 int dfn[N], mi[19][N], tot;
2 vector<int> e[N];
3 int get(int x, int y) { return dfn[x] < dfn[y] ? x : y; }
4 void dfs(int u, int f) {
5     dfn[u] = ++tot;
6     mi[0][dfn[u]] = f;
7     for (int v : e[u])
8         if (v != f) dfs(v, u);
9 }
10 int lca(int u, int v) {
11     if (u == v) return u;
12     u = dfn[u], v = dfn[v];
13     if (u > v) swap(u, v);
14     int d = __lg(v - u);
15     u++;
16     return get(mi[d][u], mi[d][v - (1 << d) + 1]);
17 }
```

3.12 LCT 动态树

```

1 // 记得初始化 mn; 维护虚子树: access link cut pushup
2 int fa[MX], ch[MX][2], w[MX], mn[MX], mark[MX];
3 int get(int x) {return x == ch[fa[x]][1];}
4 int nrt(int x) {return get(x) || x == ch[fa[x]][0];}
5 void pushup(int x) {
6     | mn[x] = w[x];
7     | if (lch(x)) mn[x] = min(mn[x], mn[lch(x)]);
8     | if (rch(x)) mn[x] = min(mn[x], mn[rch(x)]); }
9 void rev(int x) {mark[x] ^= 1, swap(lch(x), rch(x));}
10 void pushdown(int x) {
11     | if (mark[x]) {
12     |     | if (lch(x)) rev(lch(x));
13     |     | if (rch(x)) rev(rch(x));
14     |     | mark[x] = false; } }
15 void rot(int x) {
16     | int f = fa[x], gf = fa[f];
17     | int which = get(x), W = ch[x][!which];
18     | if (nrt(f)) ch[gf][ch[gf][1] == f] = x;
19     | ch[x][!which] = f, ch[f][which] = W;
20     | if (W) fa[W] = f;
21     | fa[f] = x, fa[x] = gf;
22     | pushup(f); }
23 void splay(int x) {
24     | static int stk[MX];
25     | int f = x, dep = 0; stk[++dep] = f;
26     | while (nrt(f)) stk[++dep] = f = fa[f];
27     | while (dep) pushdown(stk[dep--]);
28     | while (nrt(x)) {
29     |     | if (nrt(f = fa[x])) rot(get(x) == get(f) ? f : x);
30     |     | rot(x);
31     | } pushup(x); }
32 void access(int x) {
33     | for(int y = 0; x; x = fa[y = x])
34     |     | splay(x), rch(x) = y, pushup(x); }
35 void makeroot(int x) {access(x), splay(x), rev(x);}
36 void split(int x, int y) {makeroot(x), access(y), splay(y);}
37 int findroot(int x) {
38     | access(x), splay(x);
39     | while (lch(x)) pushdown(x), x = lch(x);
40     | return splay(x), x; }
41 void link(int x, int y) {
42     | makeroot(x);
43     | if (findroot(y) != x) fa[x] = y; }
44 void cut(int x, int y) {
45     | makeroot(x);
46     | if (findroot(y) != x || fa[y] != x || lch(y)) return;
47     | rch(x) = fa[y] = 0, pushup(x); }

```

3.13 可持久化 Treap

```

1 /* 不可持久化: 把 copy(a, b) 换成 a = b, 并且去除新建结点 */
2 const int MX = (2e5 + 233) * 18 * 8;
3 int vcnt;
4 struct node {
5     int sz, ch[2], pri;
6     int rev; LL sum; int val;
7 } tr[MX];

```

```

8 int newnode(int v) {
9     static mt19937 rng(114514);
10    int x = ++vcnt;
11    tr[x].sz = 1;
12    lch = rch = 0;
13    tr[x].pri = rng();
14    tr[x].sum = tr[x].val = v;
15    tr[x].rev = false;
16    return x; }
17 void copy(int x, int y) { tr[x] = tr[y]; }
18 int merge(int x, int y) {
19     if (!x || !y) return x + y;
20     int z = ++vcnt;
21     if (tr[x].pri < tr[y].pri) {
22         pushdown(x); copy(z, x);
23         tr[z].ch[1] = merge(tr[z].ch[1], y);
24     } else {
25         pushdown(y); copy(z, y);
26         tr[z].ch[0] = merge(x, tr[z].ch[0]); }
27     pushup(z); return z; }
28 void split(int x, int dsz, int &r1, int &r2) {
29     if (!x) {
30         r1 = r2 = 0;
31     } else {
32         pushdown(x);
33         if (tr[lch].sz + 1 <= dsz) {
34             r1 = ++vcnt; copy(r1, x);
35             split(tr[r1].ch[1], dsz - 1 - tr[lch].sz
36                 , tr[r1].ch[1], r2);
37             pushup(r1);
38         } else {
39             r2 = ++vcnt; copy(r2, x);
40             split(tr[r2].ch[0], dsz, r1, tr[r2].ch[0]);
41             pushup(r2); } } }

```

3.14 PBDS 实现平衡树

```

1 #include <bits/extc++.h>
2 #include <bits/stdc++.h>
3 using namespace std;
4 using namespace __gnu_pbds;
5 struct node {
6     int a, b;
7 };
8 struct cmp {
9     bool operator()(const node& a, const node& b) const {
10         if (a.a != b.a) return a.a < b.a;
11         return a.b < b.b;
12     }
13 };
14 // 基于自定义类型的平衡树
15 tree<node, // 元素类型
16     null_type, // 无映射值
17     cmp, // 比较策略
18     rb_tree_tag, // 底层使用红黑树
19     tree_order_statistics_node_update // 支持排名更新
20 >
21 pbds;
22

```

```

23 // 基于 int 类型的平衡树
24 tree<int, null_type, std::less<int>, rb_tree_tag,
25     tree_order_statistics_node_update>
26 tr;
27 void solve() {
28     // 插入
29     pbds.insert({1, 1});
30     pbds.insert({1, 2});
31     pbds.insert({2, 2});
32
33     // 访问
34     for (auto [a, b] : pbds) cout << a << ' ' << b << endl;
35
36     // 删除
37     pbds.erase({1, 0}); // 无用
38     pbds.erase({1, 1});
39
40     // 获取元素的排名
41     // 注意, 返回值为 0base
42     cout << pbds.order_of_key({2, 2}) + 1 << endl; // 2
43     cout
44         << pbds.order_of_key({1, 0}) + 1
45         << endl; // 1
46         // (元素可以不在集合中, 但仍然可以查找排名, 此时可当作其在集合)
47
48     // 查找排名第 k 小 (注意, 这里同样为 0base)
49     auto no = pbds.find_by_order(2 - 1);
50     cout << no->a << ' ' << no->b << endl; //(2,2)
51
52     // 存在性
53     auto it = pbds.find({1, 0});
54     cout << (it != pbds.end()) << endl; // 0
55
56     // 二分
57     auto itt = pbds.lower_bound({2, 1});
58     cout << itt->a << ' ' << itt->b << endl; //(2,2)
59
60     // 清空
61     pbds.clear();
62 }
63 int main() {
64     solve();
65     return 0;
66 }

```

3.15 PBDS 实现可并堆

```

1 #include <bits/extc++.h>
2 #include <bits/stdc++.h>
3
4 using namespace __gnu_pbds;
5 using namespace std;
6
7 template <typename T, typename Cmp = std::less<T>>
8 using heap = __gnu_pbds::priority_queue<T, Cmp>;
9
10 void solve() {
11     // 创建一个小根堆 (默认是大根堆, std::less<int> 使其成为最大堆)
12     // 使用 std::greater<int> 来创建一个最小堆

```

```
13     heap<int, std::greater<int>> h1;
14
15     // push: 插入元素
16     h1.push(10);
17     h1.push(5);
18     h1.push(20);
19
20     // 获取堆顶元素（最小值）
21     cout << "h1 堆顶元素: " << h1.top() << endl; // 5
22
23     // 弹出堆顶元素
24     h1.pop();
25     cout << "h1 pop 后堆顶元素: " << h1.top() << endl; // 10
26
27     // 合并操作
28     heap<int, std::greater<int>> h2;
29     h2.push(8);
30     h2.push(10);
31
32     cout << " 合并前 h1 的大小: " << h1.size() << endl; // 2
33     cout << " 合并前 h2 的大小: " << h2.size() << endl; // 2
34
35     // join: 将 h2 合并到 h1 中。操作后, h2 会被清空
36     // 该操作复杂度为 O(1)
37     h1.join(h2);
38
39     cout << " 合并后 h1 的大小: " << h1.size() << endl; // 4
40     cout << " 合并后 h2 的大小: " << h2.size() << endl; // 0
41     cout << " 合并后 h1 的堆顶: " << h1.top() << endl; // 8
42
43     // 遍历堆中元素
44     // 注意: 遍历顺序不保证有序
45     for (int val : h1) {
46         cout << val << " ";
47     }
48     cout << endl;
49
50     // 修改元素
51     // modify 需要一个指向元素的迭代器
52     // AI 称 push 和 pop 某个元素不会使其他迭代器元素失效
53     auto it = h1.push(12); // push 会返回一个迭代器
54     h1.push(30);
55
56     cout << " 修改前堆顶: " << h1.top() << endl; // 8
57
58     // 将值为 12 的元素修改为 2
59     h1.modify(it, 2);
60
61     cout << " 修改后堆顶: " << h1.top() << endl; // 2
62
63     // 清空堆
64     h1.clear();
65 }
66
67 int main() {
68     solve();
69     return 0;
70 }
```


3.16 手写 bitset

```

1 struct Bitset {
2     typedef unsigned int ui;
3     typedef unsigned long long ll;
4     #define all(x) (x).begin(), (x).end()
5     const static ll B = -1llu;
6     vector<ll> a;
7     int n;
8     Bitset() {}
9     Bitset(int _n) : n(_n), a(_n + 63 >> 6) {}
10    bool operator[](int x) const {
11        assert(x >= 0 && x < n);
12        return a[x >> 6] >> (x & 63) & 1;
13    }
14    void set(int x, bool y) {
15        assert(x >= 0 && x < n);
16        a[x >> 6] = (a[x >> 6] & (B ^ 1 << (x & 63))) | ((ll)y << (x & 63));
17    }
18    void set(int x) {
19        assert(x >= 0 && x < n);
20        a[x >> 6] |= 1llu << (x & 63);
21    }
22    void set() {
23        memset(a.data(), 0xff, a.size() * sizeof a[0]);
24        a.back() &= (1llu << 1 + (n - 1 & 63)) - 1;
25    }
26    void reset(int x) { a[x >> 6] &= ~(1llu << (x & 63)); }
27    void reset() { memset(a.data(), 0, a.size() * sizeof a[0]); }
28    int count() const {
29        int r = 0;
30        for (ll x : a) r += __builtin_popcountll(x);
31        return r;
32    }
33    Bitset &operator|=(const Bitset &o) {
34        assert(n == o.n);
35        for (int i = 0; i < a.size(); i++) a[i] |= o.a[i];
36        return *this;
37    }
38    Bitset operator|(Bitset o) {
39        o |= *this;
40        return o;
41    }
42    Bitset &operator&=(const Bitset &o) {
43        assert(n == o.n);
44        for (int i = 0; i < a.size(); i++) a[i] &= o.a[i];
45        return *this;
46    }
47    Bitset operator&(Bitset o) {
48        o &= *this;
49        return o;
50    }
51    Bitset &operator^=(const Bitset &o) {
52        assert(n == o.n);
53        for (int i = 0; i < a.size(); i++) a[i] ^= o.a[i];
54        return *this;
55    }
56    Bitset operator^(Bitset o) {
57        o ^= *this;
58        return o;

```

```

59     }
60     Bitset operator~() const {
61         auto r = *this;
62         for (ll &x : r.a) x = ~x;
63         return r;
64     }
65     Bitset &operator<=(int x) {
66         if (x >= n) {
67             fill(all(a), 0);
68             return *this;
69         }
70         assert(x >= 0);
71         int y = x >> 6;
72         x &= 63;
73         for (int i = (int)a.size() - 1; i > y; i--)
74             a[i] = a[i - y] << x | a[i - y - 1] >> 64 - x;
75         a[y] = a[0] << x;
76         memset(a.data(), 0, y * sizeof a[0]);
77         // fill_n(a.begin(), y, 0);
78         a.back() &= (1llu << 1 + (n - 1 & 63)) - 1;
79         return *this;
80     }
81     Bitset operator<<(int x) {
82         auto r = *this;
83         r <<= x;
84         return r;
85     }
86     Bitset &operator>>=(int x) {
87         if (x >= n) {
88             fill(all(a), 0);
89             return *this;
90         }
91         assert(x >= 0);
92         int y = x >> 6, R = (int)a.size() - y - 1;
93         x &= 63;
94         for (int i = 0; i < R; i++)
95             a[i] = a[i + y] >> x | a[i + y + 1] << 64 - x;
96         a[R] = a.back() >> x;
97         memset(a.data() + R + 1, y * sizeof a[0]);
98         // fill(R+1+all(a), 0);
99         return *this;
100     }
101     Bitset operator>>>(int x) {
102         auto r = *this;
103         r >>= x;
104         return r;
105     }
106     void range_set(int l, int r) // [l, r) to 1
107     {
108         if (l >> 6 == r >> 6) {
109             a[l >> 6] |= (1llu << r - l) - 1 << (l & 63);
110             return;
111         }
112         if (l & 63) {
113             a[l >> 6] |= ~((1llu << (l & 63)) - 1); // [l&63, 64)
114             l = (l >> 6) + 1 << 6;
115         }
116         if (r & 63) {
117             a[r >> 6] |= (1llu << (r & 63)) - 1;
118             r = (r >> 6) - 1 << 6;

```

```

119     }
120     memset(a.data() + (l >> 6), 0xff, (r - l >> 6) * sizeof a[0]);
121 }
122 void range_reset(int l, int r) //[l,r) to 0
123 {
124     if (l >> 6 == r >> 6) {
125         a[l >> 6] &= ~((11lu << r - l) - 1 << (l & 63));
126         return;
127     }
128     if (l & 63) {
129         a[l >> 6] &= (11lu << (l & 63)) - 1; //[l&63,64)
130         l = (l >> 6) + 1 << 6;
131     }
132     if (r & 63) {
133         a[r >> 6] &= ~((11lu << (r & 63)) - 1);
134         r = (r >> 6) - 1 << 6;
135     }
136     memset(a.data() + (l >> 6), 0, (r - l >> 6) * sizeof a[0]);
137 }
138 void range_set(int l, int r, bool x) //[l,r)
139 {
140     if (x)
141         range_set(l, r);
142     else
143         range_reset(l, r);
144 }
145 };

```

3.17 珂朵莉树

```

1 struct ODT {
2     struct Node_t {
3         int l, r;
4         mutable int v;
5         // mutable 关键字允许在 const 成员函数或 const 对象中修改
6
7         Node_t(const int &l, const int &r, const int &v)
8             : l(l), r(r), v(v) {}
9
10        bool operator<(const Node_t &o) const { return l < o.l; }
11    };
12
13    set<Node_t> s;
14    /*
15    将原本包含点 x 的区间（设为 [l, r]）分裂为两个区间 [l, x) 和
16    [x, r], 并返回指向后者的迭代器。
17    */
18    auto split(int x) {
19        auto it = s.lower_bound(Node_t(x, 0, 0));
20        if (it != s.end() && it->l == x) return it;
21        --it;
22        int l = it->l, r = it->r, v = it->v;
23        s.erase(it);
24        s.insert(Node_t(l, x - 1, v));
25        return s.insert(Node_t(x, r, v)).first;
26    }
27
28    // 区间赋值
29    void assign(int l, int r, int v) {

```

```
30     auto itr = split(r + 1); // 必须先分裂 r + 1 否则迭代器可能失效。
31     auto itl = split(l);
32     s.erase(itl, itr);
33     s.insert(Node_t(l, r, v));
34 }
35
36 /*
37 提取一段区间进行操作。
38 要保证均摊复杂度需要在调用 perform(l,r) 后立刻 assign(l,r,v)
39 */
40 void perform(int l, int r) {
41     auto itr = split(r + 1), itl = split(l);
42     for (; itl != itr; ++itl) {
43     }
44 }
45 };
```

4. String

4.1 最小表示法

两种不同方法实现的最小表示法

```

1  int min_pos(vector<int> a) { // 0-based
2      | int n = a.size(), i = 0, j = 1, k = 0;
3      | while (i < n && j < n && k < n) {
4      |     | auto u = a[(i + k) % n]; auto v = a[(j + k) % n];
5      |     | int t = u > v ? 1 : (u < v ? -1 : 0);
6      |     | if (t == 0) k++;
7      |     | else {
8      |         | if (t > 0) i += k + 1; else j += k + 1;
9      |         | if (i == j) j++;
10     |         | k = 0;
11     |     | }
12     | } return min(i, j);
13 }
14
15 int Duval(const string& s)//return begin of min expression
16 {
17     int n = s.length();
18     int last = -1;
19     for(int i = 0; i < n;){
20         int j = i + 1, k = i;
21         while(j < n && s[k] <= s[j]){
22             if(s[j] > s[k])k = i;
23             else k++;
24             j++;
25         }
26
27         last = i;
28         while(i <= k){//i + j - k <= j
29             i += j - k;
30         }
31         if(j == n)break;//find last but not empty
32     }
33     return last;
34 };

```

4.2 Manacher

```

1  vector<int> manacher(string& s) {
2      | //用 '#' 分割字符
3      string t = "#";
4      for (auto c : s) {
5          t += c;
6          t += '#';
7      }
8      int n = t.size();
9      vector<int> r(n); //回文半径
10     for (int i = 0, j = 0; i < n; i++) {
11         if (2 * j - i >= 0 && j + r[j] > i) {
12             r[i] = min(r[2 * j - i], j + r[j] - i);
13         }
14         while (i - r[i] >= 0 && i + r[i] < n && t[i - r[i]] == t[i + r[i]]) r[i]++;
15     }

```

```

16         if (i + r[i] > j + r[j]) {
17             j = i;
18         }
19     }
20     return r;
21 }

```

4.3 KMP, exKMP

```

1 void kmp(char *s, int n) { // 1-based
2     fail[0] = fail[1] = 0;
3     for (int i = 1; i < n; i++) { int j = fail[i];
4         while (j && s[i + 1] != s[j + 1]) j = fail[j];
5         if (s[i + 1] == s[j + 1]) fail[i + 1] = j + 1;
6         else fail[i + 1] = 0; } }
7 void exkmp(char *s, int *a, int n) { // 1-based
8     int l = 0, r = 0; a[1] = n;
9     for (int i = 2; i <= n; i++) {
10        a[i] = i > r ? 0 : min(r - i + 1, a[i - l + 1]);
11        while (i + a[i] <= n && s[1 + a[i]] == s[i + a[i]]) a[i]++;
12        if (i + a[i] - 1 > r) {l = i; r = i + a[i] - 1;}}

```

4.4 Border 树

实际上还是 kmp 构建的失配树 (Border 树), 加上了 slink 指针等附加结构

slink 的含义与 PAM+dp 中的 slink 相似, 都是指向公差 diff 与自身不等的上一个 border。

slink 将原串的 border 结构划分成 $\log n$ 段等差数列, diff 是公差。

```

1 struct KMP
2 {
3     string s;
4     vector<int>fail;
5     vector<int>slink;
6     vector<int>diff;
7     vector<vector<int>>>ot;
8     int suff;
9
10    KMP(){init();}
11    KMP(string& ss){init();for(auto ch :ss)add(ch);}
12
13    void init()
14    {
15        suff = 0;
16        s += '#';
17        fail.assign(1, 0);
18        slink.assign(1, -1);
19        diff.assign(1, -1);
20        ot.emplace_back();
21    }
22
23
24    void newNode()
25    {
26        fail.emplace_back();
27        diff.emplace_back();
28        slink.emplace_back();
29        ot.emplace_back();

```

```

30     }
31
32     void add(char c)
33     {
34         s += c;
35         newNode();
36         int last = s.length() - 1;
37         int cur = suff;
38         while(cur != 0 && s[cur + 1] != s[last]){
39             cur = fail[cur];
40         }
41         if(s[last] == s[cur + 1] && last != 1)fail[last] = cur + 1;
42         else fail[last] = cur;
43         diff[last] = last - fail[last];
44         slink[last] = diff[last] == diff[fail[last]] ? slink[fail[last]] :
            ↪ fail[last];
45         ot[fail[last]].push_back(last);
46         suff = fail[last];
47     }
48
49 };

```

4.5 AC 自动机

```

1 struct ACAM {
2     static constexpr int M = 26;
3     struct Node {
4         int len;
5         int link;
6         array<int, M> next;
7         Node() : len{0}, link{0}, next{} {}
8     };
9
10    vector<Node> t;
11
12    ACAM() { init(); }
13
14    void init() {
15        // 设置了一个 0 号节点长度是-1, 避免 build 时特判
16        t.assign(2, Node());
17        t[0].next.fill(1);
18        t[0].len = -1;
19    }
20
21    int newNode() {
22        t.emplace_back();
23        return t.size() - 1;
24    }
25
26    int add(const std::string& a) {
27        int p = 1;
28        for (auto c : a) {
29            int x = c - 'a';
30            if (!t[p].next[x]) {
31                t[p].next[x] = newNode();
32                t[t[p].next[x]].len = t[p].len + 1;
33            }
34            p = t[p].next[x];
35        }

```

```

36     return p;
37 }
38
39 void build() {
40     queue<int> q;
41     q.push(1);
42
43     while (!q.empty()) {
44         int x = q.front();
45         q.pop();
46
47         for (int i = 0; i < M; i++) {
48             if (t[x].next[i] == 0) {
49                 t[x].next[i] = t[t[x].link].next[i];
50             } else {
51                 t[t[x].next[i]].link = t[t[x].link].next[i];
52                 q.push(t[x].next[i]);
53             }
54         }
55     }
56 }
57
58 int next(int p, int x) { return t[p].next[x]; }
59
60 int link(int p) { return t[p].link; }
61
62 int len(int p) { return t[p].len; }
63
64 int size() { return t.size(); }
65 };

```

```

1 int cnt = 0;
2 #define K 26
3 struct node {
4     int son[K], num, fail;
5     void clear() {
6         num = fail = 0;
7         memset(son, 0, sizeof(son));
8     }
9 } tr[N];
10 void insert(string s) {
11     int p = 0;
12     for (int i = 0; i < s.length(); i++) {
13         int ch = s[i] - 'a';
14         if (!tr[p].son[ch]) tr[p].son[ch] = ++cnt, tr[cnt].clear();
15         p = tr[p].son[ch];
16     }
17     tr[p].num++;
18 }
19 vector<int> e[N];
20 void getfail() {
21     queue<int> q;
22     for (int i = 0; i < K; i++)
23         if (tr[0].son[i]) q.push(tr[0].son[i]), e[0].push_back(tr[0].son[i]);
24     while (q.size()) {
25         int p = q.front();
26         q.pop();
27         for (int i = 0; i < K; i++) {
28             if (tr[p].son[i]) {

```



```

29         int a = tr[p].son[i], b = tr[tr[p].fail].son[i];
30         tr[a].fail = b, q.push(a);
31         e[b].push_back(a);
32     } else
33         tr[p].son[i] = tr[tr[p].fail].son[i];
34     }
35 }
36 }
37 // 清空: cnt=0, tr[0].clear();
38 // for(int i=1; i<=n; i++) insert(s[i]);
39 // for(int i=0; i<=cnt; i++) e[i].clear();

```

4.6 Lyndon Word Decomposition

```

1 //满足 s 的最后缀等于 s 本身的串 s 称为 Lyndon 串.
2 //等价于: s 是它自己的所有循环移位中唯一最小的一个.
3 //任意字符串 s 可以分解为  $s = s_1 s_2 s_k$ , 其中  $s_i$  是 Lyndon 串,  $s_i \geq s_{i+1}$ . 且这种分解方法是唯一
  ↳ 的.
4 //后缀排序后, 排名的所有前缀最小值构成了 Ly 分解的左端点.
5 void mnsuf(char *s, int *mn, int n){ // 每个前缀的最小后缀
6     | //1 - base, 求 Lyndon 分解去掉 mn 即可
7     for(int i = 1; i <= n; )
8         int j = i + 1, k = i; mn[i] = i;
9         for(; j <= n && s[k] <= s[j]; j++){
10             if(s[k] < s[j]) k = mn[j] = i;
11             else mn[j] = mn[k] + j - k, k++;
12         for(; i <= k; i += j - k) {} } //lyn+=s[i..i+k-j-1]
13 void mxsuf(char *s, int *mx, int n){ // 每个前缀的最大后缀
14     fill(mx + 1, mx + n + 1, 0); // 1 - base
15     for(int i = 1; i <= n; ){
16         int j = i + 1, k = i; !mx[i] ? mx[i] = i : 0;
17         for(; j <= n && s[k] >= s[j]; j++){
18             !mx[j] ? mx[j] = i : 0;
19             s[k] > s[j] ? k = i : k++; }
20         for(; i <= k; i += j - k) {} } }

```

```

1 //Duval 算法实现的 Chen-Fox-Lyndon 分解
2 //返回所有的 CFL 分解中 Lyndon 串的位置
3 //0 base 闭区间
4 vector<pair<int,int>>Duval(const string& s)
5 {
6     int n = s.length();
7     vector<pair<int,int>>lyn; //
8     for(int i = 0; i < n; ){
9         int j = i + 1, k = i;
10        while(j < n && s[k] <= s[j]){
11            if(s[j] > s[k]) k = i;
12            else k++;
13        }
14        while(i <= k){ //i + j - k <= j
15            lyn.emplace_back(i, i + j - k - 1);
16            i += j - k;
17        }
18    }
19    return lyn;
20 }
21 };

```

4.7 后缀自动机

```
1 struct SAM {
2     static constexpr int M = 26; // 字符集大小
3     struct Node {
4         int len; // 该状态代表的最长字符串的长度
5         int link; // 后缀链接
6         array<int, M> nxt; // 转移边
7         Node() : len{}, link{}, nxt{} {}
8     };
9     vector<Node> t; // 状态节点数组
10    int last = 1; // 指向上一个完整字符串对应的状态节点
11
12    SAM() { init(); }
13
14    void init() {
15        t.assign(2, Node());
16        t[0].len = -1; // 虚空节点, 方便处理
17        t[0].nxt.fill(1);
18        // t[1] 是初始状态节点, 代表空串""
19    }
20
21    int newNode() {
22        t.push_back(Node());
23        return t.size() - 1;
24    }
25
26    void extend(int x) {
27        int cur = newNode();
28        t[cur].len = t[last].len + 1;
29        int p = last;
30        while (p != 0 && t[p].nxt[x] == 0) {
31            t[p].nxt[x] = cur;
32            p = t[p].link;
33        }
34        int q = t[p].nxt[x];
35        if (p == 0) {
36            t[cur].link = 1;
37        } else if (t[q].len == t[p].len + 1) {
38            t[cur].link = q;
39        } else {
40            int clone = newNode();
41            t[clone].link = t[q].link;
42            t[clone].nxt = t[q].nxt;
43            t[clone].len = t[p].len + 1;
44            t[cur].link = clone;
45            t[q].link = clone;
46            while (p != 0 && t[p].nxt[x] == q) {
47                t[p].nxt[x] = clone;
48                p = t[p].link;
49            }
50        }
51        last = cur;
52        return;
53    }
54
55    int nxt(int p, int x) { return t[p].nxt[x]; }
56    int link(int p) { return t[p].link; }
57    int len(int p) { return t[p].len; }
58    int size() { return t.size(); }
```

```

59
60     static inline int num(int x) { return x - 'a'; }
61     SAM(string &s) {
62         init();
63         build(s);
64     }
65
66     // 完整构建 SAM
67     void build(string &s) {
68         for (auto &c : s) {
69             extend(num(c));
70         }
71         get_out_linktree();
72     }
73
74     // ---- 可选部分 ---- //
75     vector<vector<int>> ot; // out-link-tree, parent 树
76     vector<int> endpos_size;
77
78     // 构建 parent 树
79     void get_out_linktree() {
80         ot.assign(t.size(), {});
81         for (int i = 2; i < t.size(); i++) {
82             ot[t[i].link].push_back(i);
83         }
84     }
85
86     // 计算每个状态的 endpos 集合大小
87     void calc_endpos_size(string &s) {
88         endpos_size.resize(t.size());
89         int p = 1;
90         for (auto c : s) {
91             p = t[p].nxt[num(c)];
92             endpos_size[p]++;
93         }
94         auto dfs = [&](auto &&self, int p) -> void {
95             for (auto s : ot[p]) {
96                 self(self, s);
97                 endpos_size[p] += endpos_size[s];
98             }
99         };
100         dfs(dfs, 1);
101         endpos_size[1] = 1;
102     }
103 };

```

4.8 后缀数组

```

1 struct SA {
2     int n;
3     std::vector<int> sa, rk, lc;
4     /*
5     sa[k] = 后缀排名为 k 的后缀起始位置 (k = 0..n-1)
6     rk[pos] = 后缀 s[pos..] 的排名 (0..n-1)
7     lc[i] = LCP(sa[i], sa[i+1])
8
9     求任意两个后缀的 LCP: LCP(s[a, n], s[b, n])
10    LCP(LCP(s[1], s[1 + 1]) ... LCP(s[r - 1], s[r]))
11    min(lc[1] ... lc[r-1]) = rmq(1, r-1)

```

```

12
13  */
14  SA(std::string s) {
15      n = s.size();
16      sa.resize(n);
17      lc.resize(n - 1);
18      rk.resize(n);
19      std::iota(sa.begin(), sa.end(), 0);
20      std::sort(sa.begin(), sa.end(),
21              [&](int a, int b) { return s[a] < s[b]; });
22      rk[sa[0]] = 0;
23      for (int i = 1; i < n; i++) {
24          rk[sa[i]] = rk[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);
25      }
26      int k = 1;
27      std::vector<int> tmp, cnt(n);
28      tmp.reserve(n);
29      while (rk[sa[n - 1]] < n - 1) {
30          tmp.clear();
31          for (int i = 0; i < k; i++) {
32              tmp.push_back(n - k + i);
33          }
34          for (auto i : sa) {
35              if (i >= k) {
36                  tmp.push_back(i - k);
37              }
38          }
39          std::fill(cnt.begin(), cnt.end(), 0);
40          for (int i = 0; i < n; i++) {
41              cnt[rk[i]]++;
42          }
43          for (int i = 1; i < n; i++) {
44              cnt[i] += cnt[i - 1];
45          }
46          for (int i = n - 1; i >= 0; i--) {
47              sa[--cnt[rk[tmp[i]]]] = tmp[i];
48          }
49          std::swap(rk, tmp);
50          rk[sa[0]] = 0;
51          for (int i = 1; i < n; i++) {
52              rk[sa[i]] =
53                  rk[sa[i - 1]] + (tmp[sa[i - 1]] < tmp[sa[i]] ||
54                               sa[i - 1] + k == n ||
55                               tmp[sa[i - 1] + k] < tmp[sa[i] + k]);
56          }
57          k *= 2;
58      }
59      for (int i = 0, j = 0; i < n; i++) {
60          if (rk[i] == 0) {
61              j = 0;
62          } else {
63              for (j -= j > 0; i + j < n && sa[rk[i] - 1] + j < n &&
64                   s[i + j] == s[sa[rk[i] - 1] + j];) {
65                  j++;
66              }
67              lc[rk[i] - 1] = j;
68          }
69      }
70  }

```

71 };

4.9 Suffix Balanced Tree 后缀平衡树

```

1 // 后缀平衡树每次在字符串开头添加或删除字符，考虑在当前字符串 S 前插入一个字符 c，那么相
  ↳ 当于在后缀平衡树中插入一个新的后缀 cS，简单的话可以使用预处理哈希二分 LCP 判断两个后
  ↳ 缀的大小作 cmp，直接写 set，时间复杂度  $O(n \lg^2 n)$ 。为了方便可以把字符反过来做
2 // 例题：加一个字符或删除一个字符，同时询问不同子串个数
3 struct cmp{
4     | bool operator()(int a,int b){
5     |     | int p=lcp(a,b); //注意这里是后面加，lcp 是反过来的
6     |     | if(a==p) return 0; if(b==p) return 1;
7     |     | return s[a-p]<s[b-p];}
8 }; set<int,cmp>S; set<int,cmp>::iterator il,ir;
9 void del(){S.erase(L--);} //在后面删字符
10 void add(char ch){ //在后面加字符
11     | s[++L]=ch;
12     | mx=0;
13     | il=ir=S.lower_bound(L);
14     | if(il!=S.begin()) mx=max(mx,lcp(L,*--il));
15     | if(ir!=S.end()) mx=max(mx,lcp(L,*ir));
16     | an[L]=an[L-1]+L-mx;
17     | S.insert(L);
18 }
19 LL getan(){printf("%lld\n",an[L]);} //询问不同子串个数

```

4.10 广义后缀自动机 (离线)

使用 insert 方法插入。全部插入完后需要 build。

```

1 constexpr int ALPHA_SIZE = 26;
2 struct GSAM
3 {
4     struct Node{
5         array<int,ALPHA_SIZE>next;
6         int link;
7         int len;
8         Node():next{},link{},len{}{}
9     };
10    vector<Node>t;
11    static constexpr int root = 1;
12
13    GSAM()
14    {
15        init();
16    }
17
18    void init()
19    {
20        t.assign(2,{});
21        t[0].next.fill(1);
22        t[0].len = -1;
23    }
24
25    int newNode()
26    {
27        t.emplace_back();
28        return t.size() - 1;
29    }
30 }

```

```
29     }
30
31     int num(char c)
32     {
33         return c - 'a';
34     }
35
36     void insert(const vector<int>& v)//insert trie
37     {
38         int p = root;
39         for(auto x : v){
40             if(t[p].next[x] == 0){
41                 t[p].next[x] = newNode();
42             }
43             p = t[p].next[x];
44         }
45         return;
46     }
47
48     void insert(const string& s)//insert trie
49     {
50         int p = root;
51         for(auto c : s){
52             int x = num(c);
53             if(t[p].next[x] == 0){
54                 t[p].next[x] = newNode();
55             }
56             p = t[p].next[x];
57         }
58         return;
59     }
60
61     int insertSAM(int last, int x)//return last
62     {
63         int cur = t[last].next[x];
64         t[cur].len = t[last].len + 1;
65         int p = t[last].link;
66         while(p != 0 && t[p].next[x] == 0){
67             t[p].next[x] = cur;
68             p = t[p].link;
69         }
70
71         if(p == 0){
72             t[cur].link = 1;
73         }
74         else{
75             int q = t[p].next[x];
76             if(t[p].len + 1 == t[q].len){
77                 t[cur].link = q;
78             }
79             else{
80                 int clone = newNode();
81
82                 //t[clone].next = t[q].next;
83                 for(int i = 0; i < ALPHA_SIZE; i++){
84                     t[clone].next[i] = t[t[q].next[i]].len != 0 ? t[q].next[i] : 0;
85                 }
86                 t[clone].link = t[q].link;
87                 t[clone].len = t[p].len + 1;
88                 while(p != 0 && t[p].next[x] == q){
```

```

89         t[p].next[x] = clone;
90         p = t[p].link;
91     }
92     t[q].link = t[cur].link = clone;
93 }
94 }
95 return cur;
96 }
97
98 void build()
99 {
100     queue<pair<int,int>>q;
101     for(int i = 0;i < ALPHA_SIZE;i++){
102         if(t[root].next[i] != 0){
103             q.push(make_pair(root,i));
104         }
105     }
106
107     while(!q.empty()){
108         auto [la, x] = q.front();
109         q.pop();
110         int last = insertSAM(la,x);
111         for(int i = 0;i < ALPHA_SIZE;i++){
112             if(t[last].next[i] != 0){
113                 q.push(make_pair(last,i));
114             }
115         }
116     }
117 }
118
119 // inline int& next(const int& p,const char& c)noexcept
120 // {
121 //     return t[p].next[num(c)];
122 // }
123 };

```

4.11 广义在线 SAM

```

1 struct SAM{
2     int tot,fail[MM],len[MM],t[MM][26];
3     SAM(){tot=1;}
4     int insert(int c,int last){
5         if(t[last][c]){
6             int p=last,q=t[p][c];
7             if(len[p]+1==len[q])return q;
8             else { int nq=++tot;
9                 fail[nq]=fail[q];fail[q]=nq;
10                len[nq]=len[p]+1;memcpy(t[nq],t[q],sizeof(t[q]));
11                for(;p && t[p][c]==q;p=fail[p])t[p][c]=nq;
12                //可以直接复制下面的代码。
13                return nq; } }
14     int p=last,np=++tot;
15     len[np]=len[p]+1;
16     for(;p && !t[p][c];p=fail[p])t[p][c]=np;
17     if(!p)fail[np]=1;
18     else {
19         int q=t[p][c];
20         if(len[q]==len[p]+1)fail[np]=q;
21         else { int nq=++tot;

```

```

22 |         fail[nq]=fail[q];fail[q]=nq;
23 |         len[nq]=len[p]+1;memcpy(t[nq],t[q],sizeof(t[q]));
24 |         for(;p && t[p][c]==q;p=fail[p])t[p][c]=nq;
25 |         fail[np]=nq; } }
26 |     return np; } }sam;
27 | // scanf("%s",st+1);int slen=strlen(st+1);
28 | // int last=1;
29 | // for(int j=1;j<=slen;j++)last=sam.insert(st[j]-'a',last);

```

4.12 回文自动机

1 的子树是长为偶数的串, 0 的子树是长为奇数的. 1 代表空串, 0 没有意义.

```

1  constexpr int ALPHA_SIZE = 26;
2  struct PAM
3  {
4      struct Node{
5          array<int,ALPHA_SIZE>next;//可以换成 map
6          int dep;//回文树上深度 / * 当前 * 以这个点结尾的回文后缀个数
7          int len;//回文长度
8          int cnt;//作为最长回文后缀的次数, 调用 calc_cnt 后变成该回文子串数量
9          int fail;//指向最长回文真后缀
10         int trans;//长度小于等于自身一半的最长回文后缀
11         Node():next{},dep{},len{},cnt{1},fail{},trans{}{}
12     };
13
14     static constexpr int odd_root = 0;
15     static constexpr int even_root = 1;
16     //odd root -> 0
17     //even root -> 1
18
19     vector<Node>t;
20     int suff;
21     string s;
22
23     PAM()
24     {
25         init();
26     }
27
28     PAM(string &s)
29     {
30         init();
31         for(auto ch : s){
32             add(ch);
33         }
34     }
35
36     void init()
37     {
38         t.assign(2,Node());
39         t[0].len = -1;
40         suff = 1;
41         s.clear();
42     }
43
44     int newNode()
45     {
46         t.emplace_back();

```



```
47     return t.size() - 1;
48 }
49
50 constexpr int num(const char& c) noexcept
51 {
52     return c - 'a';
53 };
54
55 bool add(char c)
56 {
57     s += c;
58     int x = num(c);
59     int cur = get_fail(suff);
60     if(t[cur].next[x]){//exist
61         suff = t[cur].next[x];
62         t[suff].cnt++;
63         return false;
64     }
65
66     int p = newNode();
67     suff = p;//new longest palindrome suffix
68     t[p].len = t[cur].len + 2;
69     t[cur].next[x] = p;
70     if(t[p].len == 1){//trans form odd_root
71         t[p].fail = even_root;//even root
72         t[p].trans = even_root;
73         t[p].dep = 1;
74         return true;
75     }
76     int f = get_fail(t[cur].fail);// find new fail begin at lps(cur)
77     t[p].fail = t[f].next[x];
78     int tf = get_trans(t[cur].trans, t[p].len);
79     t[p].trans = t[tf].next[x];
80     t[p].dep = t[t[p].fail].dep + 1;
81     return true;
82 }
83
84 int get_fail(int p)
85 {
86     // if p == odd_root -> len = -1, ok
87     int len = s.length() - 1;
88     while(len - t[p].len - 1 < 0 || s[len] != s[len - t[p].len - 1])p =
        ↳ t[p].fail;
89     return p;
90 }
91
92 int get_trans(int p, int plen)
93 {
94     int len = s.length() - 1;
95     //(t[p].len + 2) * 2 < plen
96     while(t[p].len * 2 > plen - 4 || len - t[p].len - 1 < 0 || s[len] != s[len -
        ↳ t[p].len - 1])p = t[p].fail;
97     return p;
98 }
99
100 void calc_cnt()
101 {
102     for(int i = t.size() - 1; i > 1; i--){
103         t[t[i].fail].cnt += t[i].cnt;
104     }
```

```
105     }
106 };
```

```
1 struct PAM {
2     static constexpr int ALPHABET_SIZE = 26;
3     struct Node {
4         int len;
5         int link;
6         int cnt;
7         std::array<int, ALPHABET_SIZE> next;
8         Node() : len{}, link{}, cnt{}, next{} {}
9     };
10    std::vector<Node> t;
11    int suff;
12    std::string s;
13    PAM() { init(); }
14    void init() {
15        t.assign(2, Node());
16        t[0].len = -1; // 0 为奇根, 1 为偶根
17        suff = 1;
18        s.clear();
19    }
20    int newNode() {
21        t.emplace_back();
22        return t.size() - 1;
23    }
24    bool add(char c) {
25        int pos = s.size();
26        s += c;
27        int let = c - 'a';
28        int cur = suff, curlen = 0;
29        while (true) {
30            curlen = t[cur].len;
31            if (pos - 1 - curlen >= 0 && s[pos - 1 - curlen] == s[pos]) {
32                break;
33            }
34            cur = t[cur].link;
35        }
36        if (t[cur].next[let]) {
37            suff = t[cur].next[let];
38            return false;
39        }
40        int num = newNode();
41        suff = num;
42        t[num].len = t[cur].len + 2;
43        t[cur].next[let] = num;
44        if (t[num].len == 1) {
45            t[num].link = 1;
46            t[num].cnt = 1;
47            return true;
48        }
49        while (true) {
50            cur = t[cur].link;
51            curlen = t[cur].len;
52            if (pos - 1 - curlen >= 0 && s[pos - 1 - curlen] == s[pos]) {
53                t[num].link = t[cur].next[let];
54                break;
55            }
56        }
57    }
58 }
```

```
57     t[num].cnt = 1 + t[t[num].link].cnt;
58     return true;
59 }
60 int next(int p, int x) { return t[p].next[x]; }
61 int link(int p) { return t[p].link; }
62 int len(int p) { return t[p].len; }
63 int size() { return t.size(); }
64 };
```

4.13 回文自动机 + dp

使用 slink 指针优化 dp.

```
1 //使用 slink 指针来优化转移为回文子串的 dp
2 constexpr int ALPHA_SIZE = 26;
3 struct PAM
4 {
5     struct Node{
6         array<int,ALPHA_SIZE>next;
7         int dep;
8         int len;
9         int cnt;
10        int fail;//lps
11        int diff;//与 lps 的长度之差
12        int slink;//指向第一个 diff 不等于自身的回文后缀
13        Node():next{},dep{},len{},cnt{1},fail{},diff{},slink{}{}
14    };
15
16    static constexpr int odd_root = 0;
17    static constexpr int even_root = 1;
18    //odd root -> 0
19    //even root -> 1
20
21    vector<Node>t;
22    int suff;
23    string s;
24
25    PAM()
26    {
27        init();
28    }
29
30    PAM(string &s)
31    {
32        init();
33        for(auto ch : s){
34            add(ch);
35        }
36    }
37
38    void init()
39    {
40        t.assign(2,Node());
41        t[0].len = -1;
42        t[1].diff = 1;
43        suff = 1;
44        s.clear();
45    }
46
```

```

47     int newNode()
48     {
49         t.emplace_back();
50         return t.size() - 1;
51     }
52
53     constexpr int num(const char& c) noexcept
54     {
55         return c - 'a';
56     };
57
58     bool add(char c)
59     {
60         s += c;
61         int x = num(c);
62         int cur = get_fail(suff);
63         if(t[cur].next[x]){//exist
64             suff = t[cur].next[x];
65             t[suff].cnt++;
66             return false;
67         }
68
69         int p = newNode();
70         suff = p;//new longest palindrome suffix
71         t[p].len = t[cur].len + 2;
72         t[cur].next[x] = p;
73         if(t[p].len == 1){//trans form odd_root
74             t[p].fail = even_root;//even root
75             t[p].dep = 1;
76             t[p].diff = 1;
77             t[p].slink = 1;//even_root
78             return true;
79         }
80         int f = get_fail(t[cur].fail);// find new fail begin at lps(cur)
81         t[p].fail = t[f].next[x];
82         t[p].dep = t[t[p].fail].dep + 1;
83         t[p].diff = t[p].len - t[t[p].fail].len;
84         if(t[p].diff == t[t[p].fail].diff)t[p].slink = t[t[p].fail].slink;
85         else t[p].slink = t[p].fail;
86         return true;
87     }
88
89     int get_fail(int p)
90     {
91         // if p == odd_root -> len = -1, ok
92         int len = s.length() - 1;
93         while(len - t[p].len - 1 < 0 || s[len] != s[len - t[p].len - 1])p =
            ↳ t[p].fail;
94         return p;
95     }
96
97 };

```

4.14 Runs

```

1 struct Runs{
2     int l,r,p;
3 };vector<Runs> run;
4 bool operator==(Runs x,Runs y){return x.l==y.l && x.r==y.r;}

```

```

5 int gl(int x,int y); // 求 S[1,x],S[1,y] 的最长公共后缀
6 int gr(int x,int y); // 求 S[x,n],S[y,n] 的最长公共前缀
7 //上面两个可以用 二分 + Hash 或者后缀数组实现。
8 bool getcmp(int x,int y){//S[x,n]<S[y,n]
9     | int len=gr(x,y);
10    | return st[x+len]<st[y+len];}
11 int ly[N];
12 void lyndon(bool type){//后缀排序法求 Lyndon
13     | stack<PII> stk;stk.push({n,n});ly[n]=n;
14     | for(int i=n-1;i>=1;i--){
15     |     | int now=i;
16     |     | while(!stk.empty() && getcmp(i,stk.top().first)!=type)
17     |     |     | now=stk.top().second,stk.pop();
18     |     | ly[i]=now;
19     |     | stk.push({i,now});
20     | } }
21 void getrun(){
22     | for(int l=1;l<=n;l++){
23     |     | int r=ly[l],ll=l,rr=r;
24     |     | if(l!=1)ll-=gl(l-1,r);
25     |     | if(r!=n)rr+=gr(l,r+1);
26     |     | if(rr-ll+1>=2*(r-l+1))run.push_back({ll,rr,r-l+1}); } }
27 void solve(){
28     | st[n+1] = '\0'; run.clear();
29     | init();//Hash 或者 SA 的启动
30     | for(int op=0;op<=1;op++){//0 正常字典序,1 反序
31     |     | lyndon(op); getrun(); }
32     | sort(run.begin(),run.end(),[](Runs x, Runs y){
33     |     | return x == y ? x.p < y.p : (x.l != y.l ? x.l < y.l : x.r < y.r);});
34     | run.erase(unique(run.begin(),run.end()),run.end()); }

```

学习 command_block 的 blog 的 runs

```

1 string s;
2 cin>>s;
3 int n = s.length();
4
5 constexpr int P1 = 1000000007u;
6 constexpr int P2 = 1000000009u;
7 constexpr int b1 = 131;
8 constexpr int b2 = 13331;
9
10 // 0-based prefixes on s[0..n-1]; sentinel s[n] exists but is NOT hashed.
11 vector<int> p1(n), h1(n), p2(n), h2(n);
12 p1[0] = 1;
13 h1[0] = s[0];
14 p2[0] = 1;
15 h2[0] = s[0];
16 for (int i = 1; i < n; ++i) {
17     p1[i] = 11l * p1[i-1] * b1 % P1;
18     p2[i] = 11l * p2[i-1] * b2 % P2;
19     h1[i] = (11l * h1[i-1] * b1 + s[i]) % P1;
20     h2[i] = (11l * h2[i-1] * b2 + s[i]) % P2;
21 }
22
23 // get hash of 0-based substring [l..r]; allow l>r/negative => 0
24 auto get = [&](int l, int r)->pair<int,int>
25 {
26     int hs1 = h1[r], hs2 = h2[r];

```

```

27     if(1)hs1 -= 1ll * h1[l - 1] * p1[r - 1 + 1] % P1, hs2 -= 1ll * h2[l - 1] *
    ↪ p2[r - 1 + 1] % P2;
28     if(hs1 < 0)hs1 += P1;
29     if(hs2 < 0)hs2 += P2;
30     return make_pair(hs1,hs2);
31 };
32
33 auto lcp = [&](int x, int y)->int //return len(lcp(x, y))
34 {
35     int l = 0, r = min(n - x, n - y) + 1;
36     while(r - l > 1){
37         int m = l + r >> 1;
38         if(get(x, x + m - 1) == get(y, y + m - 1)){
39             l = m;
40         }
41         else{
42             r = m;
43         }
44     }
45     return l;
46 };
47
48 auto lcs = [&](int x, int y)->int //return len(lcs(x, y))
49 {
50     int l = 0, r = min(x + 1, y + 1) + 1;
51     while(r - l > 1){
52         int m = l + r >> 1;
53         if(get(x - m + 1, x) == get(y - m + 1, y)){
54             l = m;
55         }
56         else{
57             r = m;
58         }
59     }
60     return l;
61 };
62
63 auto get_lyndon_suffix = [&](bool order)->vector<int>
64 {
65     vector<int>lyn(n);
66     stack<int>stk;
67     for(int i = n - 1; i >= 0; i--){
68         lyn[i] = i;
69         while(!stk.empty()){
70             int j = stk.top();
71             int len = lcp(i, j);
72             if(order == 0){
73                 if(s[i + len] > s[j + len]){//no
74                     lyn[i] = j - 1;
75                     break;
76                 }
77             }
78             else{
79                 lyn[i] = lyn[j];
80                 stk.pop();
81             }
82         }
83         else{
84             if(s[i + len] < s[j + len]){//no
85                 lyn[i] = j - 1;
86                 break;

```

```

86         }
87         else{
88             lyn[i] = lyn[j];
89             stk.pop();
90         }
91     }
92 }
93     stk.push(i);
94 }
95     return lyn;
96 };
97
98     auto lyn0 = get_lyndon_suffix(0);
99     auto lyn1 = get_lyndon_suffix(1);
100
101     vector<array<int,3>>runs;
102     for(int i = 0;i < n;i++){
103         int j,l,r,p;
104         j = lyn0[i];//period is j - i + 1, compare i, j + 1 ([i,j] is cyc)
105         l = i - 1 - lcs(i - 1, j) + 1, r = j + 1 + lcp(i, j + 1) - 1,p = j - i + 1;
106         if(r - l + 1 >= 2 * (j - i + 1)){
107             runs.push_back({l, r, p});
108             //runs.push_back({l,r,j - i + 1});
109         }
110
111         j = lyn1[i];
112         l = i - 1 - lcs(i - 1, j) + 1, r = j + 1 + lcp(i, j + 1) - 1,p = j - i + 1;
113         if(r - l + 1 >= 2 * (j - i + 1)){
114             runs.push_back({l, r, p});
115             //runs.push_back({l,r,j - i + 1});
116         }
117     }
118
119     sort(runs.begin(),runs.end());
120     runs.erase(unique(runs.begin(),runs.end()),runs.end());
121
122     vector<array<int,3>>nruns;
123     for(int i = 0;i < runs.size();i++){
124         if(i == 0 || runs[i][0] != runs[i - 1][0] || runs[i][1] != runs[i - 1]
125             ↪ [1])nruns.push_back(runs[i]);
126     }
127     runs = move(nruns);

```

4.15 字符串 Hash

```

1 static constexpr u128 inv = []() {
2     | u128 ret = P;
3     | for (int i = 0; i < 6; i++) ret *= 2 - ret * P;
4     | return ret; }();
5 constexpr u128 chk = u128(-1) / P;
6 bool check(i128 a, i128 b) {
7     | if (a < b) swap(a, b);
8     | return (a - b) * inv <= chk; }

```

使用二分 hash 求 lcp/lcs

```

1 class Hash
2 {

```

```
3 static constexpr int P1 = 1000000007;
4 static constexpr int P2 = 1000000009;
5
6 static constexpr int b1 = 131;
7 static constexpr int b2 = 13331;
8
9 int n;
10 int rb1, rb2;
11 bool use_random_base;
12
13 // int rp1, rp2;
14 // bool use_random_mod;
15
16 vector<int>h1,p1,h2,p2;
17
18 public:
19
20 Hash(string& s)//default
21 {
22     n = s.length();
23     h1.resize(n);
24     p1.resize(n);
25     h2.resize(n);
26     p2.resize(n);
27
28     p1[0] = 1;
29     h1[0] = s[0];
30     p2[0] = 1;
31     h2[0] = s[0];
32     for (int i = 1; i < n; ++i) {
33         p1[i] = 111 * p1[i-1] * b1 % P1;
34         p2[i] = 111 * p2[i-1] * b2 % P2;
35         h1[i] = (111 * h1[i-1] * b1 + s[i]) % P1;
36         h2[i] = (111 * h2[i-1] * b2 + s[i]) % P2;
37     }
38 }
39
40 Hash(string& s, bool is_random)// random base
41 {
42     mt19937 g(chrono::steady_clock::now().time_since_epoch().count());
43     rb1 = g() % b1 + b1;
44     rb2 = g() % b2 + b2;
45     use_random_base = 1;
46
47     n = s.length();
48     h1.resize(n);
49     p1.resize(n);
50     h2.resize(n);
51     p2.resize(n);
52
53     p1[0] = 1;
54     h1[0] = s[0];
55     p2[0] = 1;
56     h2[0] = s[0];
57     for (int i = 1; i < n; ++i) {
58         p1[i] = 111 * p1[i-1] * rb1 % P1;
59         p2[i] = 111 * p2[i-1] * rb2 % P2;
60         h1[i] = (111 * h1[i-1] * rb1 + s[i]) % P1;
61         h2[i] = (111 * h2[i-1] * rb2 + s[i]) % P2;
62     }
}
```



```
63     }
64
65     Hash(string& s, int rb1_, int rb2_)// diy
66     {
67         rb1 = rb1_;
68         rb2 = rb2_;
69         use_random_base = 1;
70
71         n = s.length();
72         h1.resize(n);
73         p1.resize(n);
74         h2.resize(n);
75         p2.resize(n);
76
77         p1[0] = 1;
78         h1[0] = s[0];
79         p2[0] = 1;
80         h2[0] = s[0];
81         for (int i = 1; i < n; ++i) {
82             p1[i] = 111 * p1[i-1] * rb1 % P1;
83             p2[i] = 111 * p2[i-1] * rb2 % P2;
84             h1[i] = (111 * h1[i-1] * rb1 + s[i]) % P1;
85             h2[i] = (111 * h2[i-1] * rb2 + s[i]) % P2;
86         }
87     }
88
89     pair<int,int> get(int l, int r)
90     {
91         int hs1 = h1[r], hs2 = h2[r];
92         if(1)hs1 -= 111 * h1[l - 1] * p1[r - l + 1] % P1, hs2 -= 111 * h2[l - 1] *
          ↪ p2[r - l + 1] % P2;
93         if(hs1 < 0)hs1 += P1;
94         if(hs2 < 0)hs2 += P2;
95         return make_pair(hs1,hs2);
96     }
97
98     int lcp(int x, int y)
99     {
100         int l = 0, r = min(n - x, n - y) + 1;
101         while(r - l > 1){
102             int m = l + r >> 1;
103             if(get(x, x + m - 1) == get(y, y + m - 1)){
104                 l = m;
105             }
106             else{
107                 r = m;
108             }
109         }
110         return l;
111     }
112
113     int lcs(int x, int y)
114     {
115         int l = 0, r = min(x + 1, y + 1) + 1;
116         while(r - l > 1){
117             int m = l + r >> 1;
118             if(get(x - m + 1, x) == get(y - m + 1, y)){
119                 l = m;
120             }
121             else{
```

```

122         r = m;
123     }
124 }
125 return l;
126 }
127 };

```

4.16 String Conclusions

双回文串

如果 $s = x_1x_2 = y_1y_2 = z_1z_2$, $|x_1| < |y_1| < |z_1|$, x_2, y_2, z_2 是回文串, 则 x_1 和 z_2 也是回文串.

Border 和周期

如果 r 是 S 的一个 border, 则 $|S| - r$ 是 S 的一个周期.

如果 p 和 q 都是 S 的周期, 且满足 $p + q \leq |S| + \gcd(p, q)$, 则 $\gcd(p, q)$ 也是一个周期.

字符串匹配与 Border

若字符串 S, T 满足 $2|S| \geq |T|$, 则 S 在 T 中所有匹配位置成等差数列.

若 S 的匹配次数大于 2, 则等差数列的周期恰好等于 S 的最小周期.

Border 的结构

字符串 S 的所有不小于 $|S|/2$ 的 border 长度组成一个等差数列.

字符串 S 的所有 border 按长度排序后可分成 $O(\log |S|)$ 段, 每段是一个等差数列.

回文串 Border

回文串长度为 t 的后缀是一个回文后缀, 等价于 t 是该串的前缀. 因此回文后缀的长度也可以划分成 $O(\log |S|)$ 段.

子串最小后缀

设 $s[p..n]$ 是 $s[i..n]$, $(l \leq i \leq r)$ 中最小者, 则 $\text{minsuf}(l, r)$ 等于 $s[p..r]$ 的最短非空 border. $\text{minsuf}(l, r) = \min\{s[p..r], \text{minsuf}(r - 2^k + 1, r)\}$, $(2^k < r - l + 1 \leq 2^{k+1})$.

子串最大后缀

从左往右扫, 用 set 维护后缀的字典序递减的单调队列, 并在对应时刻添加“小于事件”点以便在之后修改队列; 查询直接在 set 里 lower_bound.

ZJJ: SAM 处理手法

1. 基本子串结构: CLB 搞的那玩意。
2. 正反串 SAM 的基本联系: 一个子串出现的位置将会在两个 SAM 中同时得到映照。
3. SAM 上转成数点问题。
4. 线段树合并维护 endpos 集合。
5. 树剖保证到根的链上只涉及 $\log$ 次修改和查询。(区间 border)
6. LCT 保证到根的链只修改均摊 $\log$ 个不同的颜色段。(区间本质不同子串数量)

ZJJ: 字符串常见错误

1. 字符串算法变式记得判匹配位置超出字符串的情况, 例如多组数据下的双端插入回文串, 后缀数组多组。
2. 警惕 char 运算中 'a' 和 '\a' 的区别。
3. char kmp[]

5. Math 数学

5.1 Long Long $O(1)$ 乘, Barrett

```

1 LL modmul(LL a, LL b, LL M) { // skip2004, M < 63bit
2   | LL ret = a * b - M * LL(1.L * a / M * b + 0.5);
3   | return ret < 0 ? ret + M : ret; }
4 ULL modmul(ULL a, ULL b, LL M) { // orz@CF, M in 63 bit
5   | ULL c = (long double)a * b / M;
6   | LL ret = LL(a * b - c * M) % LL(M); // must be signed
7   | return ret < 0 ? ret + M : ret; }
8 // use int128 instead if M > 63 bit
9 struct DIV {
10  | ULL p, ip;
11  | void init (ULL _p) { p = _p; ip = -1llu / p; }
12  | int mod (ULL x) { // x < 2 ^ 64
13    |   | ULL q = ULL(((u128)ip * x) >> 64);
14    |   | ULL r = x - q * p;
15    |   | return int(r >= p ? r - p : r);
16  } }; // speedup only when mod is not const

```

5.2 exgcd, 逆元

假设我们已经找到了一组解 (p_0, q_0) 满足 $ap_0 + bq_0 = \gcd(a, b)$, 那么其他的解都满足

$$p = p_0 + \frac{b}{\gcd(p, q)} \times t \quad q = q_0 - \frac{a}{\gcd(p, q)} \times t$$

其中 t 为任意整数.

```

1 /**
2  * @brief 求解 ax + by = c, 返回 x 的最小非负整数解和对应的 y
3  * @return 返回一个 pair<ll, ll>。如果方程有解, 返回 {x, y}, 其中 x
4  * 是最小非负整数解; 如果无解, 返回 {-1, -1}。
5  */
6 std::pair<ll, ll> exgcd(ll a, ll b, ll c) {
7   assert(a || b);
8   // 迭代基: 如果 b=0, 方程为 ax=c, 解为 x=c/a, y=0 (y 可为任意值, 取 0)
9   ll d = std::gcd(a, b);
10  if (c % d) return {-1, -1}; // c 不是 gcd(a,b) 的倍数, 无整数解
11  if (!b) return {c / a, 0};
12  // p, q 保存原始的 a, b 值
13  ll x = 1, x1 = 0, p = a, q = b, k;
14
15  // 用于计算最后 x 的模数
16  ll mod = std::abs(b / d);
17
18  // 循环结束后, 求得的 x 是满足 a*x + b*y = gcd(a,b) 的一个解
19  while (b) {
20    k = a / b;
21    x -= k * x1;
22    a -= k * b;
23    std::swap(x, x1);
24    std::swap(a, b);
25  }
26
27  // 将 ax + by = d 的解 x 扩展为 ax + by = c 的解
28  // 通解为 x' = x * (c/d) + k * (b/d), 我们要求最小非负整数解

```

```

29 // 所以先对 b/d 取模
30 x = x * (c / d) % mod;
31
32 // 保证 x 是最小非负整数
33 if (x < 0) x += mod;
34
35 // 根据求得的 x, 计算出对应的 y
36 return {x, (c - p * x) / q};
37 }

```

递推逆元: $\text{inv}(i) \equiv (P - P/i) \cdot \text{inv}(P \bmod i)$

5.3 同余方程

$ax \equiv b \pmod{c}$

```

1 std::pair<ll, ll> exgcd(ll a, ll b, ll c) {
2     assert(a || b);
3     ll d = std::gcd(a, b);
4     if (c % d) return {-1, -1};
5     if (!b) return {c / a, 0};
6     ll x = 1, x1 = 0, p = a, q = b, k;
7     ll mod = std::abs(b / d);
8     while (b) {
9         k = a / b;
10        x -= k * x1;
11        a -= k * b;
12        std::swap(x, x1);
13        std::swap(a, b);
14    }
15    x = x * (c / d) % mod;
16    if (x < 0) x += mod;
17    return {x, mod};
18 }
19
20
21 //a*x==b(mod c)
22 pair<ll,ll> cal(ll a,ll b,ll c) //0<=r<p / (-1,-1)
23 {
24     auto [x,mod]=exgcd(a,c,b);
25     return {x,mod};
26 }

```

5.4 CRT 中国剩余定理

```

1 namespace CRT {
2 pair<ll, ll> exgcd(ll a, ll b, ll c) {
3     assert(a || b);
4     if (!b) return {c / a, 0};
5     ll d = gcd(a, b);
6     if (c % d) return {-1, -1};
7     ll x = 1, x1 = 0, p = a, q = b, k;
8     b = abs(b);
9     while (b) {
10        k = a / b;
11        x -= k * x1;
12        a -= k * b;
13        swap(x, x1);
14        swap(a, b);

```

```

15     }
16     b = abs(q / d);
17     x = x * (c / d) % b;
18     if (x < 0) x += b;
19     return {x, (c - p * x) / q};
20 }
21 struct Q {
22     ll p, r; // 0<=r<p
23     Q operator+(const Q &o) const {
24         if (p == 0 || o.p == 0) return {0, 0};
25         auto [x, y] = exgcd(p, -o.p, r - o.r);
26         if (x == -1 && y == -1) return {0, 0};
27         ll q = lcm(p, o.p);
28         return {q, ((r - x * p) % q + q) % q};
29     }
30 };
31 }; // namespace CRT
32 using CRT::Q;

```

5.5 Miller Rabin, Pollard Rho

```

1 mt19937 rng(123);
2 #define rand() LL(rng() & LLONG_MAX)
3 const int BASE[] = {2, 7, 61}; //int(7,3e9)
4 //{2,325,9375,28178,450775,9780504,1795265022}LL(37)
5 struct miller_rabin {
6     bool check (const LL &M, const LL &base) {
7         LL a = M - 1;
8         while (~a & 1) a >>= 1;
9         LL w = power (base, a, M); // power should use mul
10        for (; a != M - 1 && w != 1 && w != M - 1; a <= 1)
11            w = mul (w, w, M);
12        return w == M - 1 || (a & 1) == 1; }
13 bool solve (const LL &a) { //O((3 or 7) · log n · mul)
14     if (a < 4) return a > 1;
15     if (~a & 1) return false;
16     for (int i = 0; i < sizeof(BASE)/4 && BASE[i] < a; ++i)
17         if (!check (a, BASE[i])) return false;
18     return true; } };
19 miller_rabin is_prime;
20 LL get_factor (LL a, LL seed) { //O(n1/4 · log n · mul)
21     LL x = rand () % (a - 1) + 1, y = x;
22     for (int head = 1, tail = 2; ; ) {
23         x = mul (x, x, a); x = (x + seed) % a;
24         if (x == y) return a;
25         LL ans = gcd (abs (x - y), a);
26         if (ans > 1 && ans < a) return ans;
27         if (++head == tail) { y = x; tail <= 1; } } }
28 void factor (LL a, vector<LL> &d) {
29     if (a <= 1) return;
30     if (is_prime.solve (a)) d.push_back (a);
31     else {
32         LL f = a;
33         for (; f >= a; f = get_factor (a, rand() % (a - 1) + 1));
34         factor (a / f, d);
35         factor (f, d); } }

```

5.6 线性基

```
1 struct LB {
2     const int BASE = 16;
3     vector<int> d, p;
4     int cnt, flag;
5     LB() {
6         d.assign(BASE, 0);
7         p.assign(BASE, 0);
8         cnt = flag = 0;
9     }
10    bool insert(int val, int pos) {
11        for (int i = BASE - 1; i >= 0; i--) {
12            if (val & (1ll << i)) {
13                if (!d[i]) {
14                    d[i] = val;
15                    p[i] = pos;
16                    return true;
17                }
18                if (pos > p[i]) swap(pos, p[i]), swap(val, d[i]);
19                val ^= d[i];
20            }
21        }
22        flag = 1; // 可以异或出 0
23        return false;
24    }
25    bool check(ll val) { // 判断 val 是否能被异或得到
26        for (int i = BASE - 1; i >= 0; i--) {
27            if (val & (1ll << i)) {
28                if (!d[i]) {
29                    return false;
30                }
31                val ^= d[i];
32            }
33        }
34        return true;
35    }
36    ll ask_max(ll res) {
37        for (int i = BASE - 1; i >= 0; i--) {
38            if ((res ^ d[i]) > res) res ^= d[i];
39        }
40        return res;
41    }
42    ll ask_min() {
43        if (flag) return 0; // 特判 0
44        for (int i = 0; i <= BASE - 1; i++) {
45            if (d[i]) return d[i];
46        }
47    }
48    void rebuild() { // guass 消元 保证只有该处有 1
49        for (int i = BASE - 1; i >= 0; i--) {
50            for (int j = i - 1; j >= 0; j--) {
51                if (d[i] & (1ll << j)) d[i] ^= d[j];
52            }
53        }
54        for (int i = 0; i <= BASE - 1; i++) {
55            if (d[i]) p[cnt++] = d[i];
56        }
57    }
58    ll kth(ll k) { // 查询能被异或得到的第 k 小值, 如不存在则返回 -1
```

```

59     if (flag) k--; // 特判 0, 如果不需要 0, 直接删去
60     if (!k) return 0;
61     ll res = 0;
62     if (k >= (1ll << cnt)) return -1;
63     for (int i = BASE - 1; i >= 0; i--) {
64         if (k & (1ll << i)) res ^= p[i];
65     }
66     return res;
67 }
68 };

```

5.7 扩展卢卡斯

```

1  int l, a[33], p[33], P[33];
2  U fac(int k, LL n) { // 求 n! mod pk^tk, 返回值 U{ 不包含 pk 的值, pk 出现的次数 }
3      if (!n) return U{1, 0}; LL x = n / p[k], y = n / P[k], ans = 1, int i;
4      if (y) { // 求出循环节的答案
5          for (i = 2; i < P[k]; i++) if (i % p[k]) ans = ans * i % P[k];
6          ans = Pw(ans, y, P[k]);
7      } for (i = y * P[k]; i <= n; i++) if (i % p[k]) ans = ans * i % M; // 求零散部分
8      U z = fac(k, x); return U{ans * z.x % M, x + z.z};
9  } LL get(int k, LL n, LL m) { // 求 C(n, m) mod pk^tk
10     U a = fac(k, n), b = fac(k, m), c = fac(k, n - m); // 分三部分求解
11     return Pw(p[k], a.z - b.z - c.z, P[k]) * a.x % P[k] *
        inv(b.x, P[k]) % P[k] * inv(c.x, P[k]) % P[k];
12 } LL CRT() { // CRT 合并答案
13     LL d, w, y, x, ans = 0;
14     for (i = 1; i <= M / P[i]; i++) exgcd(w, P[i], x, y), ans = (ans + w * x % M * a[i]) % M;
15     return (ans + M) % M;
16 } LL C(LL n, LL m) { // 求 C(n, m)
17     fr(i, 1, l) a[i] = get(i, n, m);
18     return CRT();
19 } LL exLucas(LL n, LL m, int M) {
20     int jj = M, i; // 求 C(n, m) mod M, M = prod(pi^ki), O(pi^kilg^2n)
21     for (i = 2; i * i <= jj; i++) if (jj % i == 0)
22         for (p[++l] = i, P[l] = 1; jj % i == 0; P[l] *= i) jj /= i;
23     if (jj > 1) l++, p[l] = P[l] = jj;
24     return C(n, m);

```

5.8 阶乘取模

```

1  // n! mod p^q Time : O(pq^2 * log^2 n / log p)
2  // Output : {a, b} means a * p^b
3  using Val = unsigned long long; // Val 需要 mod p^q 意义下 + *
4  typedef vector<Val> poly;
5  poly polymul(const poly &a, const poly &b) {
6      int n = (int) a.size(); poly c(n, Val(0));
7      for (int i = 0; i < n; ++i) {
8          for (int j = 0; i + j < n; ++j) {
9              c[i + j] = c[i + j] + a[i] * b[j]; } }
10     return c; } Val choo[70][70];
11 poly polyshift(const poly &a, Val delta) {
12     int n = (int) a.size(); poly res(n, Val(0));
13     for (int i = 0; i < n; ++i) { Val d = 1;
14         for (int j = 0; j <= i; ++j) {
15             res[i - j] = res[i - j] + a[i] * choo[i][j] * d;
16             d = d * delta; } } return res; }

```

```

17 void prepare(int q) {
18     for (int i = 0; i < q; ++ i) { choo[i][0] = Val(1);
19         for (int j = 1; j <= i; ++ j)
20             choo[i][j]=choo[i-1][j-1]+choo[i-1][j]; } }
21 pair<Val, LL> fact(LL n, LL p, LL q) { Val ans = 1;
22     for (int r = 1; r < p; ++ r) {
23         poly x (q, Val(0)), res (q, Val(0));
24         res[0] = 1; LL _res = 0; x[0] = r; LL _x = 0;
25         if (q > 1) x[1] = p, _x = 1; LL m = (n - r + p) / p;
26         while (m) { if (m & 1) {
27             res=polymul(res,polyshift(x,_res)); _res+=_x; }
28             m >>= 1; x = polymul(x, polyshift(x, _x)); _x+=_x; }
29         ans = ans * res[0]; }
30     LL cnt = n / p; if (n >= p) { auto tmp=fact(n / p, p, q);
31         ans = ans * tmp.first; cnt += tmp.second; }
32     return {ans, cnt}; }

```

5.9 类欧几里得直线下格点统计

```

1 //  $\sum_{i=0}^{n-1} \lfloor \frac{a+bi}{m} \rfloor$ ,  $n, m, a, b > 0$ 
2 LL solve(LL n, LL a, LL b, LL m){
3     if (b == 0) return n * (a / m);
4     if (a >= m) return n * (a / m) + solve(n, a % m, b, m);
5     if (b >= m) return (n-1)*n/2*(b/m) + solve(n,a,b%m,m);
6     return solve((a + b * n) / m, (a + b * n) % m, m, b); }

```

5.10 万能欧几里德

```

1 Val work(LL P, LL R, LL Q, LL n, Val VU, Val VR) {
2     //  $(Px+R)/Q$ ,  $1 \leq x \leq i$ , 经过整点先 U 再 R
3     if (!(((i128)n * P + R) / Q)) return ksm(VR, n);
4     if (P >= Q) return work(P%Q,R,Q,n, VU, ksm(VU, P/Q) * VR);
5     Val res; swap(VU,VR);
6     res = ksm(VU, (Q-R-1)/P)*VR;
7     LL m = ((i128)n * P + R) / Q;
8     res = res * work(Q, (Q-R-1)%P, P, m-1, VU, VR);
9     return res * ksm(VU, n - ((i128)m*Q - R - 1) / P); }

```

5.11 平方剩余

```

1 //  $x^2 \equiv a \pmod p$ ,  $0 \leq a < p$ , 返回 true or false 代表是否存在解
2 // p 必须是质数, 若是多个单次质数的乘积, 可以分别求解再用 CRT 合并
3 // 复杂度为  $O(\log n)$ 
4 void multiply(ll &c, ll &d, ll a, ll b, ll w) {
5     int cc = (a * c + b * d % MOD * w) % MOD;
6     int dd = (a * d + b * c) % MOD; c = cc, d = dd; }
7 bool solve(int n, int &x) {
8     if (n==0) return x=0,true; if (MOD==2) return x=1,true;
9     if (power(n, MOD / 2, MOD) == MOD - 1) return false;
10    ll c = 1, d = 0, b = 1, a, w;
11    // finding a such that  $a^2 - n$  is not a square
12    do { a = rand() % MOD; w = (a * a - n + MOD) % MOD;
13        if (w == 0) return x = a, true;
14    } while (power(w, MOD / 2, MOD) != MOD - 1);
15    for (int times = (MOD + 1) / 2; times; times >>= 1) {
16        if (times & 1) multiply(c, d, a, b, w);

```



```

17 | multiply(a, b, a, b, w); }
18 | // x = (a + sqrt(w)) ^ ((p + 1) / 2)
19 | return x = c, true; }

```

5.12 线性同余不等式

```

1 // Find the minimal non-negative solutions for  $l \leq d \cdot x \bmod m \leq r$ 
2 //  $0 \leq d, l, r < m; l \leq r, O(\log n)$ 
3 LL cal(LL m, LL d, LL l, LL r) {
4 | if (l==0) return 0; if (d==0) return MXL; // 无解
5 | if (d * 2 > m) return cal(m, m - d, m - r, m - l);
6 | if ((l - 1) / d < r / d) return (l - 1) / d + 1;
7 | LL k = cal(d, (-m % d + d) % d, l % d, r % d);
8 | return k==MXL ? MXL : (k*m + l - 1)/d+1; } // 无解 2
9 // return all x satisfying  $l1 \leq x \leq r1$  and  $l2 \leq (x * mul + add) \% LIM \leq r2$ 
10 // here LIM =  $2^{32}$  so we use UI instead of "%".
11 //  $O(\log p + \#solutions)$ 
12 struct Jump { UI val, step;
13 | Jump(UI val, UI step) : val(val), step(step) { }
14 | Jump operator + (const Jump & b) const {
15 | | return Jump(val + b.val, step + b.step); }
16 | Jump operator - (const Jump & b) const {
17 | | return Jump(val - b.val, step + b.step); } };
18 inline Jump operator * (UI x, const Jump & a) {
19 | return Jump(x * a.val, x * a.step); }
20 vector<UI> solve(UI l1, UI r1, UI l2, UI r2, pair<UI,UI> muladd) {
21 | UI mul = muladd.first, add = muladd.second, w = r2 - l2;
22 | Jump up(mul, 1), dn(-mul, 1); UI s(l1 * mul + add);
23 | Jump lo(r2 - s, 0), hi(s - l2, 0);
24 | function<void(Jump&, Jump&)> sub=[&](Jump& a, Jump& b){
25 | | if (a.val > w) {
26 | | | UI t(((LL)a.val-max(0LL, w+1LL-b.val)) / b.val);
27 | | | a = a - t * b; } };
28 | sub(lo, up), sub(hi, dn);
29 | while (up.val > w || dn.val > w) {
30 | | sub(up, dn); sub(lo, up);
31 | | sub(dn, up); sub(hi, dn); }
32 | assert(up.val + dn.val > w); vector<UI> res;
33 | Jump bg(s + mul * min(lo.step, hi.step), min(lo.step, hi.step));
34 | while (bg.step <= r1 - l1) {
35 | | if (l2 <= bg.val && bg.val <= r2)
36 | | | res.push_back(bg.step + l1);
37 | | if (l2 <= bg.val-dn.val && bg.val-dn.val <= r2) {
38 | | | bg = bg - dn;
39 | | } else bg = bg + up; }
40 | return res; }

```

5.13 原根

定义 使得 $a^x \bmod m = 1$ 的最小的 x , 记作 $\delta_m(a)$. 若 $a \equiv g^s \bmod m$, 其中 g 为 m 的一个原根. 则虽然 s 随 g 的不同取值有所不同, 但是必然满足 $\delta_m(a) = \gcd(s, \varphi(m))$.

性质 $\delta_m(a^k) = \frac{\delta_m(a)}{\gcd(\delta_m(a), k)}$

k 次剩余 给定方程 $x^k \equiv a \bmod m$, 求所有解. 若 k 与 $\varphi(m)$ 互质, 则可以直接求出 k 对 $\varphi(m)$ 的逆元. 否则, 将 k 拆成两部分, $k = uv$, 其中 $u \perp \varphi(m)$, $v | \varphi(m)$, 先求 $x^v \equiv a \bmod m$, 则 $ans = x^{u^{-1}}$. 下面讨论 $k | \varphi(m)$ 的问题. 任取一原根 g , 对两侧取离散对数, 设 $x = g^s$, $a = g^t$, 其中 t 可以用 BSGS 求出, 则问题转化为求出所有的 s 满足 $ks \equiv t \bmod \varphi(m)$, exgcd 即可求解, 显然有解的条件是 $k | \delta_m(a)$.

5.14 多项式

```

1 using ll = long long;
2 const int mod = 998244353;
3
4 int qmi(ll a, ll k) {
5     ll res = 1;
6     while (k) {
7         if (k & 1) res = res * a % mod;
8         k >>= 1;
9         a = a * a % mod;
10    }
11    return res;
12 }
13
14 namespace NTT {
15     const int g = 3;
16     vector<int> Omega(int L) {
17         int wn = qmi(g, mod / L);
18         vector<int> w(L);
19         w[L >> 1] = 1;
20         for (int i = L / 2 + 1; i <= L - 1; i++) w[i] = 1ll * w[i - 1] * wn % mod;
21         for (int i = L / 2 - 1; i; i--) w[i] = w[i << 1];
22         return w;
23     }
24     auto W = Omega(1 << 21); // NOLINT
25     void DIF(int* a, int n) {
26         for (int k = n >> 1; k; k >>= 1)
27             for (int i = 0, y; i < n; i += k << 1)
28                 for (int j = 0; j < k; j++)
29                     y = a[i + j + k],
30                     a[i + j + k] = 1ll * (a[i + j] - y + mod) * W[k + j] % mod,
31                     a[i + j] = (y + a[i + j]) % mod;
32     }
33     void IDIT(int* a, int n) {
34         for (int k = 1; k < n; k <= 1)
35             for (int i = 0, x, y; i < n; i += k << 1)
36                 for (int j = 0; j < k; j++)
37                     x = a[i + j], y = (1ll * a[i + j + k] * W[k + j]) % mod,
38                     a[i + j + k] = x - y < 0 ? x - y + mod : x - y,
39                     a[i + j] = (y + a[i + j]) % mod;
40         int Inv = mod - (mod - 1) / n;
41         for (int i = 0; i < n; i++) a[i] = 1ll * a[i] * Inv % mod;
42         reverse(a + 1, a + n);
43     }
44     // 用普通数组实现的 NTT 蝶形变化
45 } // namespace NTT
46 namespace Polynomial {
47     using Poly = std::vector<int>;
48
49     // mul/div int
50     Poly& operator*=(Poly& a, int b) {
51         for (auto& x : a) x = 1ll * x * b % mod;
52         return a;
53     }
54     Poly operator*(Poly a, int b) { return a *= b; }
55     Poly operator*(int a, Poly b) { return b * a; }
56     Poly& operator/=(Poly& a, int b) { return a *= qmi(b, mod - 2); }
57     Poly operator/(Poly a, int b) { return a /= b; }
58 }

```

```

59 // Poly add/sub
60 Poly& operator+=(Poly& a, Poly b) {
61     a.resize(max(a.size(), b.size()));
62     for (int i = 0; i < b.size(); i++) a[i] = (a[i] + b[i]) % mod;
63     return a;
64 }
65 Poly operator+(Poly a, Poly b) { return a += b; }
66 Poly& operator-=(Poly& a, Poly b) {
67     a.resize(max(a.size(), b.size()));
68     for (int i = 0; i < b.size(); i++) a[i] = (a[i] - b[i] + mod) % mod;
69     return a;
70 }
71 Poly operator-(Poly a, Poly b) { return a -= b; }
72
73 // Poly mul
74 void DFT(Poly& a) { NTT::DIF(a.data(), a.size()); }
75 void IDFT(Poly& a) { NTT::IDIT(a.data(), a.size()); }
76 int norm(int n) {
77     return 1 << (32 - __builtin_clz(n - 1));
78 } // 返回大于等于 n 的最小 2 的整数次幂
79 void norm(Poly& a) {
80     if (!a.empty()) a.resize(norm(a.size()), 0);
81 } // 剩余的用 0 填
82 Poly& dot(Poly& a, Poly& b) {
83     for (int i = 0; i < a.size(); i++) a[i] = 1ll * a[i] * b[i] % mod;
84     return a;
85 }
86 Poly operator*(Poly a, Poly b) {
87     int n = a.size() + b.size() - 1, L = norm(n);
88     if (a.size() <= 8 || b.size() <= 8) {
89         Poly c(n);
90         for (int i = 0; i < a.size(); i++)
91             for (int j = 0; j < b.size(); j++)
92                 c[i + j] = (c[i + j] + 1ll * a[i] * b[j]) % mod;
93         return c;
94     }
95     a.resize(L), b.resize(L);
96     DFT(a), DFT(b), dot(a, b), IDFT(a);
97     return a.resize(n), a;
98 }
99
100 // Poly inv 利用牛顿迭代递归实现的乘法逆
101 Poly Inv2k(Poly a) { // a.size() = 2^k
102     int n = a.size(), m = n >> 1;
103     if (n == 1) return {qmi(a[0], mod - 2)};
104     Poly b = Inv2k(Poly(a.begin(), a.begin() + m)), c = b;
105     b.resize(n), DFT(a), DFT(b), dot(a, b), IDFT(a);
106     for (int i = 0; i < n; i++) a[i] = i < m ? 0 : mod - a[i];
107     DFT(a), dot(a, b), IDFT(a);
108     return move(c.begin(), c.end(), a.begin()), a;
109 }
110 Poly Inv(Poly a) {
111     int n = a.size();
112     norm(a), a = Inv2k(a);
113     return a.resize(n), a;
114 }
115
116 // Poly div/mod
117 Poly operator/(Poly a, Poly b) {
118     int k = a.size() - b.size() + 1;

```

```

119     if (k < 0) return {0};
120     reverse(a.begin(), a.end());
121     reverse(b.begin(), b.end());
122     b.resize(k), a = a * Inv(b);
123     a.resize(k), reverse(a.begin(), a.end());
124     return a;
125 }
126 pair<Poly, Poly> operator%(Poly a, const Poly& b) {
127     Poly c = a / b;
128     a -= b * c, a.resize(b.size() - 1);
129     return {c, a};
130 }
131
132 // 求导和积分
133 Poly deriv(Poly a) {
134     for (int i = 1; i < a.size(); i++) a[i - 1] = 11l * i * a[i] % mod;
135     return a.pop_back(), a;
136 }
137
138 // 此处如果预处理逆元的话大概可以优化
139 Poly integ(Poly a) {
140     a.push_back(0);
141     for (int i = a.size() - 1; i; i--)
142         a[i] = 11l * a[i - 1] * qmi(i, mod - 2) % mod;
143     return a[0] = 0, a;
144 }
145
146 // Poly 要求 a[0] = 1
147 Poly Ln(Poly a) {
148     int n = a.size();
149     a = deriv(a) * Inv(a);
150     return a.resize(n - 1), integ(a);
151 }
152
153 // Poly exp
154 Poly Exp(Poly a) {
155     int n = a.size(), k = norm(n);
156     Poly b = {1}, c, d;
157     a.resize(k);
158     for (int L = 2; L <= k; L <= 1) {
159         d = b, b.resize(L), c = Ln(b), c.resize(L);
160         for (int i = 0; i < L; i++)
161             c[i] = a[i] - c[i] + (a[i] < c[i] ? mod : 0);
162         c[0] = (c[0] + 1) % mod;
163         DFT(b), DFT(c), dot(b, c), IDFT(b);
164         move(d.begin(), d.end(), b.begin());
165     }
166     return b.resize(n), b;
167 }
168
169 Poly Pow(Poly& a, int b) { return Exp(Ln(a) * b); } // a[0] = 1, 多项式快速幂
170
171 Poly Sqrt(Poly a) {
172     int n = a.size(), k = norm(n);
173
174     Poly b = {(int)(new Cipolla)->sqrt(a[0], mod)}, c;
175     a.resize(k * 2, 0);
176     for (int L = 2; L <= k; L <= 1) {
177         b.resize(2 * L, 0), c = Poly(a.begin(), a.begin() + L) * Inv(b);
178         for (int i = 0; i <= 2 * L; i++)

```

```

179         b[i] = 111 * (b[i] + c[i]) * (mod + 1 >> 1) % mod;
180     }
181     return b.resize(n), b;
182 }
183
184 } // namespace Polynomial
185 using namespace Polynomial;

```

5.15 FFT

```

1 using cp = complex<double>; const double PI = acos(-1.0);
2 vector<cp> omega[25]; // 单位根
3 // n 是 DFT 的最大长度, 例如如果最多有两个长为 m 的多项式相乘,
4 // 或者求逆的长度为 m, 那么 n 需要 >= 2m
5 void fft_init(int n) { // n = 2^k
6     for (int k = 2, d = 0; k <= n; k *= 2, d++) {
7         omega[d].resize(k + 1);
8         for (int i = 0; i <= k; i++) // polar 是用模和辐角求复数
9             omega[d][i] = polar(1.0, 2 * PI * i / k); } }
10 void fft(cp* a, int n, int t) {
11     for (int i = 1, j = 0; i < n - 1; i++) {
12         int k = n; do j ^= (k >>= 1); while (j < k);
13         if (i < j) swap(a[i], a[j]); }
14     for (int k = 1, d = 0; k < n; k *= 2, d++)
15         for (int i = 0; i < n; i += k * 2)
16             for (int j = 0; j < k; j++) {
17                 cp w = omega[d][t > 0 ? j : k * 2 - j];
18                 cp u = a[i + j], v = w * a[i + j + k];
19                 a[i + j] = u + v; a[i + j + k] = u - v; }
20     if (t < 0) for (int i = 0; i < n; i++) a[i] /= n; }

```

5.16 NTT

```

1 vector<int> omega[25]; // 单位根
2 // n 是 DFT 的最大长度, 例如如果最多有两个长为 m 的多项式相乘,
3 // 或者求逆的长度为 m, 那么 n 需要 >= 2m
4 void ntt_init(int n) { // n = 2^k
5     for (int k = 2, d = 0; k <= n; k *= 2, d++) {
6         omega[d].resize(k + 1);
7         int wn = qpow(3, (p - 1) / k), tmp = 1;
8         for (int i = 0; i <= k; i++) { omega[d][i] = tmp;
9             tmp = (LL)tmp * wn % p; } } }
10 // 传入的数必须是 [0, p) 范围内, 不能有负的
11 // 否则把 d == 16 改成 d % 8 == 0 之类, 多取几次模
12 void ntt(int *c, int n, int tp) {
13     static ULL a[N];
14     for (int i = 0; i < n; i++) a[i] = c[i];
15     for (int i = 1, j = 0; i < n - 1; i++) {
16         int k = n; do j ^= (k >>= 1); while (j < k);
17         if (i < j) swap(a[i], a[j]); }
18     for (int k = 1, d = 0; k < n; k *= 2, d++) {
19         if (d == 16) for (int i = 0; i < n; i++) a[i] %= p;
20         for (int i = 0; i < n; i += k * 2)
21             for (int j = 0; j < k; j++) {
22                 int w = omega[d][tp > 0 ? j : k * 2 - j];
23                 ULL u = a[i + j], v = w * a[i + j + k] % p;
24                 a[i + j] = u + v;
25                 a[i + j + k] = u - v + p; } }

```

```

26 | if (tp>0) {for (int i = 0; i < n; i++) c[i] = a[i] % p;}
27 | else { int inv = qpow(n, p - 2);
28 |     for (int i = 0; i < n; i++) c[i] = a[i] * inv % p;}}

```

5.17 MTT 任意模数卷积

```

1 void dft(cp* a, cp* b, int n) { static cp c[MAXN];
2   for (int i = 0; i < n; i++)
3     | c[i] = cp(a[i].real(), b[i].real());
4   fft(c, n, 1);
5   for (int i = 0; i < n; i++) { int j = (n - i) & (n - 1);
6     | a[i] = (c[i] + conj(c[j])) * 0.5;
7     | b[i] = (c[i] - conj(c[j])) * -0.5i; } }
8 void idft(cp* a, cp* b, int n) { static cp c[MAXN];
9   for (int i = 0; i < n; i++) c[i] = a[i] + 1i * b[i];
10  fft(c, n, -1);
11  for (int i = 0; i < n; i++) {
12    | a[i] = c[i].real(); b[i] = c[i].imag(); } }
13 vector<int> multiply(const vector<int>& u,
14   | const vector<int>& v, int mod) { // 任意模数卷积
15   static cp a[2][MAXN], b[2][MAXN], c[3][MAXN];
16   int base = ceil(sqrt(mod));
17   int n = (int)u.size(), m = (int)v.size();
18   int fft_n = 1; while (fft_n < n + m - 1) fft_n *= 2;
19   for (int i = 0; i < 2; i++) {
20     | fill(a[i], a[i] + fft_n, 0);
21     | fill(b[i], b[i] + fft_n, 0); }
22   for (int i = 0; i < 3; i++)
23     | fill(c[i], c[i] + fft_n, 0);
24   for (int i = 0; i < n; i++) { // 一定要取模!
25     | a[0][i] = (u[i] % mod) % base;
26     | a[1][i] = (u[i] % mod) / base; }
27   for (int i = 0; i < m; i++) { // 一定要取模!
28     | b[0][i] = (v[i] % mod) % base;
29     | b[1][i] = (v[i] % mod) / base; }
30   dft(a[0], a[1], fft_n); dft(b[0], b[1], fft_n);
31   for (int i = 0; i < fft_n; i++) {
32     | c[0][i] = a[0][i] * b[0][i];
33     | c[1][i] = a[0][i] * b[1][i] + a[1][i] * b[0][i];
34     | c[2][i] = a[1][i] * b[1][i]; }
35   fft(c[1], fft_n, -1); idft(c[0], c[2], fft_n);
36   int base2 = base * base % mod;
37   vector<int> ans(n + m - 1);
38   for (int i = 0; i < n + m - 1; i++)
39     | ans[i] = ((LL)(c[0][i].real() + 0.5) +
40     | (LL)(c[1][i].real() + 0.5) % mod * base +
41     | (LL)(c[2][i].real() + 0.5) % mod * base2) % mod;
42   return ans; }

```

5.18 多项式运算

5.18.1 多项式求逆 开根 ln exp

```

1 using poly = vector<int>; // 用到 poly 的部分补成 2 ^ k
2 poly poly_calc(const poly& u, const poly& v, // 长度要相同
3   | function<int(int, int)> op) { // 返回长度是两倍
4   static int a[MAXN], b[MAXN], c[MAXN];
5   int n = (int)u.size();

```

```

6 | memcpy(a, u.data(), sizeof(int) * n);
7 | fill(a + n, a + n * 2, 0);
8 | memcpy(b, v.data(), sizeof(int) * n);
9 | fill(b + n, b + n * 2, 0);
10 | ntt(a, n * 2, 1); ntt(b, n * 2, 1);
11 | for (int i = 0; i < n * 2; i++) c[i] = op(a[i], b[i]);
12 | ntt(c, n * 2, -1); return poly(c, c + n * 2); }
13 poly poly_mul(const poly& u, const poly& v) { // 乘法
14 | return poly_calc(u, v, [](int a, int b)
15 | | { return (LL)a * b % p; }); } // 返回长度是两倍
16 poly poly_inv(const poly& a) { // 求逆, 返回长度不变
17 | poly c{qpow(a[0], p - 2)}; // 常数项一般都是 1
18 | for (int k = 2; k <= (int)a.size(); k *= 2) {
19 | | c.resize(k); poly b(a.begin(), a.begin() + k);
20 | | c = poly_calc(b, c, [](int bi, int ci) {
21 | | | return ((2 - (LL)bi * ci) % p + p) * ci % p; });
22 | | memset(c.data() + k, 0, sizeof(int) * k); }
23 | c.resize(a.size()); return c; }
24 poly poly_sqrt(const poly& a) { // 开根, 返回长度不变
25 | poly c{1}; // 常数项不是 1 的话要写二次剩余
26 | for (int k = 2; k <= (int)a.size(); k *= 2) {
27 | | c.resize(k); poly b(a.begin(), a.begin() + k);
28 | | b = poly_mul(b, poly_inv(c));
29 | | for (int i = 0; i < k; i++) // inv_2 是 2 的逆元
30 | | | c[i] = (LL)(c[i] + b[i]) * inv_2 % p; }
31 | c.resize(a.size()); return c; }
32 poly poly_derivative(const poly& a) { poly c(a.size());
33 | for (int i = 1; i < (int)a.size(); i++) // 求导
34 | | c[i - 1] = (LL)a[i] * i % p; return c; }
35 poly poly_integrate(const poly& a) { poly c(a.size());
36 | for (int i = 1; i < (int)a.size(); i++) // 不定积分
37 | | c[i] = (LL)a[i - 1] * inv[i] % p; return c; }
38 poly poly_ln(const poly& a) { // ln, 常数项非 0, 返回长度不变
39 | auto c = poly_mul(poly_derivative(a), poly_inv(a));
40 | c.resize(a.size()); return poly_integrate(c); }
41 // exp, 常数项必须是 0, 返回长度不变
42 // 常数很大并且总代码很长, 一般可以改用分治 FFT
43 // 依据: 设  $G(x) = \exp F(x)$ , 则  $g_i = \frac{1}{i} \sum_{k=1}^{i-1} g_{i-k} k f_k$ 
44 poly poly_exp(const poly& a) { poly c{1};
45 | for (int k = 2; k <= (int)a.size(); k *= 2) {
46 | | c.resize(k); auto b = poly_ln(c);
47 | | for (int i = 0; i < k; i++)
48 | | | b[i] = (a[i] - b[i] + p) % p;
49 | | (++b[0]) %= p; c = poly_mul(b, c);
50 | | memset(c.data() + k, 0, sizeof(int) * k); }
51 | c.resize(a.size()); return c; }

```

5.18.2 多项式除法 取模

需要抄求逆。

```

1 poly poly_auto_mul(poly a, poly b) { // 自动判断长度的乘法
2 | int res_len = (int)a.size() + (int)b.size() - 1;
3 | int ntt_n = 1; while (ntt_n < res_len) ntt_n *= 2;
4 | a.resize(ntt_n); b.resize(ntt_n);
5 | ntt(a.data(), ntt_n, 1); ntt(b.data(), ntt_n, 1);
6 | for (int i = 0; i < ntt_n; i++)
7 | | a[i] = (LL)a[i] * b[i] % p;
8 | ntt(a.data(), ntt_n, -1); a.resize(res_len); return a; }
9 // 多项式除法, a 和 b 长度可以任意

```

```

10 // 商的长度是 n - m + 1, 余数的长度是 m - 1
11 poly poly_div(const poly& a, const poly& b) {
12     | int n = (int)a.size(), m = (int)b.size();
13     | if (n < m) return {};
14     | int ntt_n = 1; while (ntt_n < n - m + 1) ntt_n *= 2;
15     | poly f(ntt_n), g(ntt_n);
16     | for (int i = 0; i < n - m + 1; i++) f[i] = a[n - i - 1];
17     | for (int i = 0; i < m && i < n - m + 1; i++)
18     |     | g[i] = b[m - i - 1];
19     | auto g_inv = poly_inv(g);
20     | fill(g_inv.begin() + n - m + 1, g_inv.end(), 0);
21     | auto c = poly_mul(f, g_inv); c.resize(n - m + 1);
22     | reverse(c.begin(), c.end()); return c; }
23 // 多项式取模, a 和 b 长度可以任意, 返回 (余数, 商)
24 pair<poly, poly> poly_mod(const poly& a, const poly& b) {
25     | int n = (int)a.size(), m = (int)b.size();
26     | if (n < m) return {a, {}};
27     | auto d = poly_div(a, b); auto c = poly_auto_mul(b, d);
28     | poly r(m - 1);
29     | for (int i = 0; i < m - 1; i++)
30     |     | r[i] = (a[i] - c[i] + p) % p;
31     | return {r, d}; }

```

5.18.3 多点求值

需要抄取模。

```

1 struct poly_eval { poly f; vector<int> x; // 函数和询问点
2     | vector<poly> gs; vector<int> ans; // gs 是预处理数组
3     | poly_eval(poly f, vector<int> x) : f(f), x(x) {}
4     | void pretreat(int l, int r, int o) { poly& g = gs[o];
5     |     | if (l == r) { g = poly{p - x[l], 1}; return; }
6     |     | int mid = (l + r) / 2; pretreat(l, mid, o * 2);
7     |     | pretreat(mid + 1, r, o * 2 + 1);
8     |     | if (o > 1)
9     |     |     | g = poly_auto_mul(gs[o * 2], gs[o * 2 + 1]); }
10    | void solve(int l, int r, int o, const poly& f) {
11    |     | if (l == r) { ans[l] = f[0]; return; }
12    |     | int mid = (l + r) / 2;
13    |     | solve(l, mid, o * 2, poly_mod(f, gs[o * 2]).first);
14    |     | solve(mid + 1, r, o * 2 + 1,
15    |     |     | poly_mod(f, gs[o * 2 + 1]).first); }
16    | vector<int> operator() () { // 包装好的接口
17    |     | int n = (int)f.size(), m = (int)x.size();
18    |     | if (m <= n) x.resize(m = n + 1);
19    |     | else if (n < m - 1) f.resize(n = m - 1);
20    |     | int bit_ceil = 1; while (bit_ceil < m) bit_ceil *= 2;
21    |     | ntt_init(bit_ceil * 2); // 注意这里 ntt_init 过了
22    |     | gs.resize(2 * bit_ceil + 1); pretreat(0, m - 1, 1);
23    |     | ans.resize(m); solve(0, m - 1, 1, f); return ans; } };

```

5.18.4 插值

牛顿插值 实现时可以用 k 次差分替代右边的式子, 也可以卷积。

$$f(n) = \sum_{i=0}^k \binom{n}{i} r_i \iff r_i = \sum_{j=0}^i (-1)^{i-j} \binom{i}{j} f(j)$$

拉格朗日插值

$$f(x) = \sum_i f(x_i) \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

5.19 线性递推

 $O(k^2 \log n)$

```

1 // Complexity: init  $O(n^2 \log)$  query  $O(n^2 \log k)$ 
2 // Requirement: const LOG const MOD
3 // Example: In: {1, 3} {2, 1}  $a_n = 2a_{n-1} + a_{n-2}$ 
4 //           Out: calc(3) = 7
5 typedef vector<int> poly;
6 struct LinearRec {
7     | int n; poly first, trans; vector<poly> bin;
8     poly add(poly &a, poly &b) {
9         | poly res(n * 2 + 1, 0);
10        | // 不要每次新开 vector, 可以使用矩阵乘法优化
11        | for (int i = 0; i <= n; ++i) {
12            | | for (int j = 0; j <= n; ++j) {
13                | | | (res[i+j]+=(LL)a[i] * b[j] % MOD) %= MOD;
14            | | for (int i = 2 * n; i > n; --i) {
15                | | | for (int j = 0; j < n; ++j) {
16                    | | | | (res[i-1-j]+=(LL)res[i]*trans[j]%MOD) %=MOD;}
17                | | | res[i] = 0; }
18            | res.erase(res.begin() + n + 1, res.end());
19        | return res; }
20    LinearRec(poly &first, poly &trans): first(first), trans(trans) {
21        | n = first.size(); poly a(n + 1, 0); a[1] = 1;
22        | bin.push_back(a); for (int i = 1; i < LOG; ++i)
23            | | bin.push_back(add(bin[i - 1], bin[i - 1])); }
24    int calc(int k) { poly a(n + 1, 0); a[0] = 1;
25        | for (int i = 0; i < LOG; ++i)
26            | | if (k >> i & 1) a = add(a, bin[i]);
27        | int ret = 0; for (int i = 0; i < n; ++i)
28            | | | if ((ret += (LL)a[i + 1] * first[i] % MOD) >= MOD)
29                | | | ret -= MOD;
30        | return ret; }

```

 $O(k \log k \log n)$ - Bostan-Mori

```

1 int bostan_mori(int k, poly a, poly b) {
2     | int n = (int)a.size(); while (k) { poly c = b;
3         | | for (int i = 1; i < n; i += 2) c[i] = (p - c[i]) % p;
4         | | a = poly_mul(a, c); b = poly_mul(b, c);
5         | | for (int i = 0; i < n; i++) {
6             | | | a[i] = a[i * 2 + k % 2]; b[i] = b[i * 2]; }
7         | | a.resize(n); b.resize(n); k /= 2; }
8     | return (LL)a[0] * qpow(b[0], p - 2) % p; }
9 //  $a_n = \sum_{i=1}^m f_i a_{n-i}$  ( $f_0 = 0$ ), f.size() = a.size() + 1
10 int linear_recurrance(int n, poly f, poly a) {
11     | int m = (int)a.size(), ntt_n = 1;
12     | while (ntt_n <= m) ntt_n *= 2; ntt_init(ntt_n * 2);
13     | f.resize(ntt_n); a.resize(ntt_n); f[0] = 1;
14     | for (int i = 1; i <= m; i++) f[i] = (p - f[i]) % p;
15     | a = poly_mul(a, f); a.resize(ntt_n);
16     | fill(a.data() + m, a.data() + ntt_n, 0);
17     | return bostan_mori(n, a, f); }

```

$O(k \log k \log n)$ - 多项式取模

需要抄前面的多项式取模。预处理 $O(k \log k \log n)$ ，固定 n 和系数只改变初始值的话，询问一次 $O(k)$ 。注意只询问一次的话不如 Bostan-Mori 快。

```

1 poly poly_power_mod(LL k, const poly& m) { //  $x^k \bmod m$ 
2   | poly ans{1}, a{0, 1}; while (k) { if (k & 1)
3   |   |   | ans = poly_mod(poly_auto_mul(ans, a), m).first;
4   |   |   | a = poly_mod(poly_auto_mul(a, a), m).first; k /= 2; }
5   |   |   | return ans; }
6 //  $a_n = \sum_{i=1}^m c_i a_{n-i} \quad (c_0 = 0)$ 
7 struct linear_recurrence { poly f; // f 是预处理结果
8   | linear_recurrence(const poly& c, LL n) {
9   |   | assert(c[0] == 0); // c[0] 是没有用的
10  |   | int m = (int)c.size() - 1;
11  |   | int ntt_n = 1; while (ntt_n < m * 2) ntt_n *= 2;
12  |   | ntt_init(ntt_n); // 图省事就直接 ntt_init(1 << 18)
13  |   | poly t(m + 1); t[m] = 1;
14  |   | for (int i = 0; i < m; i++) t[i] = (p - c[m - i]) % p;
15  |   | f = poly_power_mod(n, t); }
16  | int operator()(const vector<int>& a) { // 0~m-1 项初始值
17  |   | assert(a.size() == f.size()); int ans = 0;
18  |   | for (int i = 0; i < (int)a.size(); i++)
19  |   |   | ans = (ans + (LL)f[i] * a[i]) % p;
20  |   | return ans; } };

```

5.20 Berlekamp-Massey 最小多项式

如果要求出一个次数为 k 的递推式，则输入的数列需要至少有 $2k$ 项。

返回的内容满足 $\sum_{j=0}^{m-1} a_{i-j} c_j = 0$ ，并且 $c_0 = 1$ 。

如果不加最后的处理的话，代码返回的结果会变成 $a_i = \sum_{j=0}^{m-1} c_{j-1} a_{i-j}$ ，有时候这样会方便接着跑递推，需要的话就删掉最后的处理。

```

1 vector<int> berlekamp_massey(const vector<int> &a) {
2   | vector<int> v, last; // v is the answer, 0-based
3   | int k = -1, delta = 0;
4   | for (int i = 0; i < (int)a.size(); i++) { int tmp = 0;
5   |   | for (int j = 0; j < (int)v.size(); j++)
6   |   |   | tmp = (tmp + (LL)a[i - j - 1] * v[j]) % p;
7   |   | if (a[i] == tmp) continue;
8   |   | if (k < 0) { k = i; delta = (a[i] - tmp + p) % p;
9   |   |   | v = vector<int>(i + 1); continue; }
10  |   | vector<int> u = v;
11  |   | int val = (LL)(a[i] - tmp + p) *
12  |   |   | qpow(delta, p - 2) % p;
13  |   | if (v.size() < last.size() + i - k)
14  |   |   | v.resize(last.size() + i - k);
15  |   | (v[i - k - 1] += val) %= p;
16  |   | for (int j = 0; j < (int)last.size(); j++) {
17  |   |   | v[i - k + j] = (v[i - k + j] -
18  |   |   |   | (LL)val * last[j]) % p;
19  |   |   | if (v[i - k + j] < 0) v[i - k + j] += p; }
20  |   | if ((int)u.size() - i < (int)last.size() - k) {
21  |   |   | last = u; k = i; delta = a[i] - tmp;
22  |   |   | if (delta < 0) delta += p; } }
23  | for (auto &x : v) x = (p - x) % p;
24  | v.insert(v.begin(), 1); // 一般是需要最小递推式的，处理一下
25  | return v; } //  $\forall i, \sum_{j=0}^m a_{i-j} v_j = 0$ 

```

如果要求向量序列的递推式，就把每位乘一个随机权值（或者说是乘一个随机行向量 v^T ）变成求数列递推式即可。如果是矩阵序列的话就随机一个行向量 u^T 和列向量 v ，然后把矩阵变成 $u^T A v$ 的数列。

优化矩阵快速幂 DP 假设 f_i 有 n 维，先暴力求出 $f_{0 \sim 2n-1}$ ，然后跑 Berlekamp-Massey，最后调用快速线性递推即可。

求矩阵最小多项式 矩阵 A 的最小多项式是次数最小的并且 $f(A) = 0$ 的多项式 f 。实际上最小多项式就是 $\{A^i\}$ 的最小递推式，所以直接调用 Berlekamp-Massey 就好了，显然它的次数不超过 n 。

瓶颈在于求出 A^i ，实际上我们只要处理 $A^i v$ 就行了，每次对向量做递推。

求稀疏矩阵的行列式 如果能求出特征多项式，则常数项乘上 $(-1)^n$ 就是行列式，但是最小多项式不一定就是特征多项式。

把 A 乘上一个随机对角阵 B ，则 AB 的最小多项式有很大概率就是特征多项式，最后再除掉 $\det B$ 就行了。

求稀疏矩阵的秩 设 A 是一个 $n \times m$ 的矩阵，首先随机一个 $n \times n$ 的对角阵 P 和一个 $m \times m$ 的对角阵 Q ，然后计算 $QAP A^T Q$ 的最小多项式即可。

实际上不用计算这个矩阵，因为求最小多项式时要用它乘一个向量，我们依次把这几个矩阵乘到向量里就行了。答案就是最小多项式除掉所有 x 因子后剩下的次数。

解稀疏方程组 $Ax = b$ ，其中 A 是一个 $n \times n$ 的满秩稀疏矩阵， b 和 x 是 $1 \times n$ 的列向量， A, b 已知，需要解出 x 。

做法：显然 $x = A^{-1}b$ 。如果我们能求出 $\{A^i b\} (i \geq 0)$ 的最小递推式 $\{r_0 \dots r_{m-1}\} (m \leq n)$ ，那么就有结论

$$A^{-1}b = -\frac{1}{r_{m-1}} \sum_{i=0}^{m-2} A^i b r_{m-2-i}$$

因为 A 是稀疏矩阵，直接按定义递推出 $b \dots A^{2n-1}b$ 即可。

```

1 vector<int> solve_sparse_equations(const vector<tuple<int, int, int> > &A, const
  ↳ vector<int> &b) {
2   | int n = (int)b.size(); // 0-based
3   | vector<vector<int> > f({b});
4   | for (int i = 1; i < 2 * n; i++) {
5   |   | vector<int> v(n); auto &u = f.back();
6   |   | for (auto [x, y, z] : A) // [x, y, value]
7   |   |   | v[x] = (v[x] + (long long)u[y] * z) % p;
8   |   | f.push_back(v); }
9   | vector<int> w(n); mt19937 gen;
10  | for (auto &x : w)
11  |   | x = uniform_int_distribution<int>(1, p - 1)(gen);
12  | vector<int> a(2 * n);
13  | for (int i = 0; i < 2 * n; i++)
14  |   | for (int j = 0; j < n; j++)
15  |   |   | a[i] = (a[i] + (long long)f[i][j] * w[j]) % p;
16  | auto c = berlekamp_massey(a); int m = (int)c.size();
17  | vector<int> ans(n);
18  | for (int i = 0; i < m - 1; i++)
19  |   | for (int j = 0; j < n; j++)
20  |   |   | ans[j] = (ans[j] +
21  |   |   |   | (long long)c[m - 2 - i] * f[i][j]) % p;
22  | int inv = qpow(p - c[m - 1], p - 2);
23  | for (int i = 0; i < n; i++)
24  |   | ans[i] = (long long)ans[i] * inv % p;
25  | return ans; }

```

5.21 FWT

```
1 /*
```

```

2 And:  $\begin{pmatrix} 1,1 \\ 0,1 \end{pmatrix} \begin{pmatrix} 1,-1 \\ 0,1 \end{pmatrix}$  Or:  $\begin{pmatrix} 1,0 \\ 1,1 \end{pmatrix} \begin{pmatrix} 1,0 \\ -1,1 \end{pmatrix}$  Xor:  $\begin{pmatrix} 1,1 \\ 1,-1 \end{pmatrix} \begin{pmatrix} 0.5,0.5 \\ 0.5,-0.5 \end{pmatrix}$ 
3 IFFT 的矩阵是 FFT 的逆, 对于任意运算  $\oplus$ , 满足 FFT 的矩阵需要:
4  $C[i][j] \times C[i][k] = C[i][j \oplus k]$ 
5 对于不存在 FFT 矩阵的运算: 通过映射 01 变成另外一个可行的运算。*/
6 const LL XOR[2][2] = {{1, 1}, {1, M-1}};
7 const LL i2 = (M+1)/2, iXOR[2][2] = {{i2, i2}, {i2, M-i2}};
8 void FWT(LL f[], const LL C[2][2], int n) {
9     for (int t = 1; t < n; t <= 1) {
10         for (int l = 0; l < n; l += t + t) {
11             for (int i = 0; i < t; i++) {
12                 LL x = f[l + i], y = f[l + t + i];
13                 f[l + i] = (C[0][0] * x + C[0][1] * y) % M;
14                 f[l + t + i] = (C[1][0] * x + C[1][1] * y) % M;
15             } } } }

```

5.22 K 进制 FWT

```

1 // n : power of k, omega[i] : (primitive kth root) ^ i
2 void fwt(int* a, int k, int type) {
3     static int tmp[K];
4     for (int i = 1; i < n; i *= k)
5         for (int j = 0, len = i * k; j < n; j += len)
6             for (int low = 0; low < i; low++) {
7                 for (int t = 0; t < k; t++)
8                     tmp[t] = a[j + t * i + low];
9                 for (int t = 0; t < k; t++){
10                     int x = j + t * i + low;
11                     a[x] = 0;
12                     for (int y = 0; y < k; y++)
13                         a[x] = int(a[x] + 1ll * tmp[y] * omega[(k + type) * t * y % k] % MOD);
14                 }
15             }
16     if (type == -1)
17         for (int i = 0, invn = inv(n); i < n; i++)
18             a[i] = int(1ll * a[i] * invn % MOD); }

```

5.23 Simplex 单纯形

```

1 const LD eps = 1e-9, INF = 1e9; const int N = 105;
2 namespace Simplex {
3     int n, m, id[N], tp[N]; LD a[N][N];
4     void pivot(int r, int c) {
5         | swap(id[r + n], id[c]);
6         | LD t = -a[r][c]; a[r][c] = -1;
7         | for (int i = 0; i <= n; i++) a[r][i] /= t;
8         | for (int i = 0; i <= m; i++) if (a[i][c] && r != i) {
9             | | t = a[i][c]; a[i][c] = 0;
10            | | for (int j = 0; j <= n; j++) a[i][j] += t*a[r][j];}}
11 bool solve() {
12     | for (int i = 1; i <= n; i++) id[i] = i;
13     | for ( ; ; ) {
14         | | int i = 0, j = 0; LD w = -eps;
15         | | for (int k = 1; k <= m; k++)
16         | | | if (a[k][0] < w || (a[k][0] < -eps && rand() & 1))
17         | | | | w = a[i = k][0];
18         | | if (!i) break;
19         | | for (int k = 1; k <= n; k++)
20         | | | if (a[i][k] > eps) {j = k; break;}

```

```

21 | | if (!j) { printf("Infeasible"); return 0;}
22 | | pivot(i, j);}
23 | for ( ; ; ) {
24 | | int i = 0, j = 0; LD w = eps, t;
25 | | for (int k = 1; k <= n; k++)
26 | | | if (a[0][k] > w) w = a[0][j = k];
27 | | if (!j) break;
28 | | w = INF;
29 | | for (int k = 1; k <= m; k++)
30 | | | if (a[k][j] < -eps && (t = -a[k][0]/a[k][j]) < w)
31 | | | | w = t, i = k;
32 | | if (!i) { printf("Unbounded"); return 0;}
33 | | pivot(i, j);}
34 | return 1;}
35 LD ans() {return a[0][0];}
36 void output() {
37 | for (int i = n + 1; i <= n + m; i++) tp[id[i]] = i - n;
38 | for (int i = 1; i <= n; i++) printf("%.9lf ", tp[i] ? a[tp[i]][0] : 0);}
39 }using namespace Simplex;
40 int main() { int K; read(n); read(m); read(K);
41 for (int i = 1; i <= n; i++) {LD x; scanf("%lf", &x); a[0][i] = x;}
42 for (int i = 1; i <= m; i++) {LD x;
43 | for (int j = 1; j <= n; j++) scanf("%lf", &x), a[i][j] = -x;
44 | scanf("%lf", &x); a[i][0] = x;}
45 if (solve()) { printf("%.9lf\n", (LD)ans()); if (K) output();}}
46 // 标准型: maximize  $c^T x$ , subject to  $Ax \leq b$  and  $x \geq 0$ 
47 // 对偶型: minimize  $b^T y$ , subject to  $A^T x \geq c$  and  $y \geq 0$ 

```

5.24 高斯消元最小范数解

```

1 typedef vector<LD> vec; /* sum a[i][0..d] = 0 */
2 pair<vec, vector<vec>> gauss(vector<vec> &a, int n, int d) {
3 | vector<int> pivot(d, -1);
4 | for (int i = 0, o = 0; i < d; i++) {
5 | | int j = o; while (j < n && abs(a[j][i]) < eps) j++;
6 | | if (j == n) continue;
7 | | swap(a[j], a[o]); LD w = a[o][i];
8 | | for (int k = 0; k <= d; k++) a[o][k] /= w;
9 | | for (int x = 0; x < n; x++)
10 | | | if (x != o && abs(a[x][i]) > eps) {
11 | | | | w = a[x][i];
12 | | | | for (int k = 0; k <= d; k++)
13 | | | | | a[x][k] -= a[o][k] * w; }
14 | | pivot[i] = o++;
15 | } vec x0(d); vector<vec> t; int free = 0;
16 | for (int i = 0; i < d; i++)
17 | | if (pivot[i] != -1) x0[i] = -a[pivot[i]][d];
18 | | else free++;
19 | for (int i = 0; i < d; i++) if (pivot[i] == -1) {
20 | | vec x(d); x[i] = -1;
21 | | for (int j = 0; j < d; j++)
22 | | | if (pivot[j] != -1) x[j] = a[pivot[j]][i];
23 | | t.push_back(x);
24 | } if (t.size()) {
25 | | vector<vec> f;
26 | | for (int u = 0; u < free; u++) {
27 | | | vec x(free + 1);
28 | | | for (int i = 0; i < free; i++)
29 | | | | for (int j = 0; j < d; j++)

```

```

30 |         |         | x[i] += t[u][j] * t[i][j];
31 |         |         | for (int j = 0; j < d; j++)
32 |         |         | x[free] += t[u][j] * x0[j];
33 |         |         | f.push_back(x);
34 |         |     }
35 |         |     auto [k, tt] = gauss(f, free, free);
36 |         |     assert (tt.size() == 0);
37 |         |     for (int x = 0; x < free; x++)
38 |         |         for (int i = 0; i < d; i++)
39 |         |             x0[i] += k[x] * t[x][i];
40 |     } return {x0, t}; }

```

5.25 Pell 方程

```

1 //  $x^2 - n * y^2 = 1$  最小正整数根, n 为完全平方数时无解
2 //  $x_{k+1} = x_0 x_k + n y_0 y_k$ 
3 //  $y_{k+1} = x_0 y_k + y_0 x_k$ 
4 pair<LL, LL> pell(LL n) {
5     | static LL p[N], q[N], g[N], h[N], a[N];
6     | p[1] = q[0] = h[1] = 1; p[0] = q[1] = g[1] = 0;
7     | a[2] = (LL)(floor(sqrt1(n) + 1e-7L));
8     | for(int i = 2; ; i++) {
9     |     | g[i] = -g[i - 1] + a[i] * h[i - 1];
10    |     | h[i] = (n - g[i] * g[i]) / h[i - 1];
11    |     | a[i + 1] = (g[i] + a[2]) / h[i];
12    |     | p[i] = a[i] * p[i - 1] + p[i - 2];
13    |     | q[i] = a[i] * q[i - 1] + q[i - 2];
14    |     | if(p[i] * p[i] - n * q[i] * q[i] == 1)
15    |         | return {p[i], q[i]}; }

```

5.26 解一元三次方程

```

1 double a(p[3]), b(p[2]), c(p[1]), d(p[0]);
2 double k(b / a), m(c / a), n(d / a);
3 double p(-k * k / 3. + m);
4 double q(2. * k * k * k / 27 - k * m / 3. + n);
5 Complex omega[3] = {Complex(1, 0), Complex(-0.5, 0.5 * sqrt(3)), Complex(-0.5, -0.5
    ↳ * sqrt(3))};
6 Complex r1, r2; double delta(q * q / 4 + p * p * p / 27);
7 if (delta > 0) {
8     | r1 = cubrt(-q / 2. + sqrt(delta));
9     | r2 = cubrt(-q / 2. - sqrt(delta));
10 } else {
11     | r1 = pow(-q / 2. + pow(Complex(delta), 0.5), 1. / 3);
12     | r2 = pow(-q / 2. - pow(Complex(delta), 0.5), 1. / 3); }
13 for(int _ (0); _ < 3; _++) {
14     | Complex x = -k/3. + r1*omega[_] + r2*omega[_* 2 % 3]; }

```

5.27 自适应 Simpson

```

1 // Adaptive Simpson's method : LD simpson::solve (LD (*f) (LD), LD l, LD r, LD eps)
2 ↳: integrates f over (l, r) with error eps.
3 struct simpson {
4     LD area (LD (*f) (LD), LD l, LD r) {
5     | LD m = l + (r - l) / 2;
6     | return (f(l) + 4 * f(m) + f(r)) * (r - l) / 6;

```

```
6 }
7 LD solve (LD (*f) (LD), LD l, LD r, LD eps, LD a) {
8   | LD m = l + (r - l) / 2;
9   | LD left = area (f, l, m), right = area (f, m, r);
10  | if (abs (left + right - a) <= 15 * eps) // TLE: || eps < EPS ** 2
11  |   | return left + right + (left + right - a) / 15.0;
12  | return solve (f, l, m, eps / 2, left) + solve (f, m, r, eps / 2, right);
13 }
14 LD solve (LD (*f) (LD), LD l, LD r, LD eps) {
15   | return solve (f, l, r, eps, area (f, l, r));
16 };
```

6. Tricks

6.1 线性预处理左边第 k 大

```

1  int n, l[N][60], r[N][60], a[N], k;
2  /*
3  l[i][j] 表示 i 左边第 j 个比 a[i] 大的数。
4  r[i][j] 同理
5  */
6  vector<int> stk[60], tmp;
7  void init() {
8      for (int i = 1; i <= n; i++) l[i][0] = r[i][0] = i;
9      for (int i = n; i; i--) {
10         for (int j = k; j; --j) {
11             tmp.clear();
12             while (!stk[j].empty() && a[i] > a[stk[j].back()]) {
13                 l[stk[j].back()][j] = i;
14                 tmp.push_back(stk[j].back());
15                 stk[j].pop_back();
16             }
17             if (j + 1 <= k)
18                 stk[j + 1].insert(stk[j + 1].end(), tmp.rbegin(), tmp.rend());
19         }
20         stk[1].push_back(i);
21     }
22     for (int i = 1; i <= k; i++) stk[i].clear();
23     for (int i = 1; i <= n; i++) {
24         for (int j = k; j; --j) {
25             tmp.clear();
26             while (!stk[j].empty() && a[i] > a[stk[j].back()]) {
27                 r[stk[j].back()][j] = i;
28                 tmp.push_back(stk[j].back());
29                 stk[j].pop_back();
30             }
31             if (j + 1 <= k)
32                 stk[j + 1].insert(stk[j + 1].end(), tmp.rbegin(), tmp.rend());
33         }
34         stk[1].push_back(i);
35     }
36 }

```

6.2 二维支配前驱查询

假设每个元素有个三个属性, $x[i]$, $y[i]$, i , 问题即找最大的 j 使得 $x[i] > x[j]$ 且 $y[i] > y[j]$ 且 $j < i$ 。使用线段树上二分可以在 $O(n \log n)$ 的时间内对每个点求解这个前驱: 首先按照 x 从小到大排序, 以 i 为线段树的下标, 动态插入 $y[i]$, 维护已经扫描过的 i 对应 $y[i]$ 区间最小值, 然后在线段树上二分。

6.3 区间众数

离线做法 使用莫队, 同时维护 $cnt[x]$ 表示当前区间内 x 的出现次数, 以及 $cntt[y]$ 表示满足 $cnt[x] = y$ 的 x 种数。

6.4 区间 mex 和 mex 区间

离线做法: 从前往后扫描, 用 p_x 表示 x 最后一场出现的位置, 没出现则为 0, 线段树节点维护 $\min_{i=1}^r p[i]$ 对于每个询问, 在线段树上二分找第一个没有出现的值即可。

在线做法只需把线段树建成前缀主席树即可。

关于 mex 区间，首先，极小 mex 区间只有 $O(n)$ 个。

定义极小 mex 区间指区间 $[l, r]$ 使得它不存在子区间 $[l', r']$ 满足

$$\text{mex}(a_{[l,r]}) = \text{mex}(a_{[l',r']})$$

证明：假设 $[l, r]$ 是极小 mex 区间。 $l = r$ 的情况只有 $O(n)$ 个，其余情况 $a_l \neq a_r$ 。

分成 $a_l > a_r$ 和 $a_l < a_r$ 的区间，先考虑前者。

由于是极小 mex 区间，故删除 l 或 r 都会使得 mex 变化，因此

$$\text{mex}(a_{[l,r]}) > a_l > a_r$$

因此一个 l 只会对应一个 r 。

假设存在 $r' > r$ 且 $[l, r']$ 是极小 mex 区间，并且 $a_l > a_{r'}$ ，那么

$$\text{mex}(a_{[l,r]}) > a_l > a_{r'}$$

也就是说 $a_{r'}$ 在 $[l, r]$ 中已经出现过，删除 r' 不会使得 mex 变化，矛盾。

求出这些极小的区间可以考虑从前往后枚举左端点 x ，使用珂朵莉树维护连续段 (l, r, v) 表示对任意的 $l \leq y \leq r$ ，有 $\text{mex}(a_{[x,y]}) = v$ 。记 next_x 为最小的 $i > x$ 满足 $a[i] = a[x]$ ，当我们删除左端点 x 时，只有右端点 y 满足 $\text{mex}(a_{[x,y]}) \geq a[x] \wedge y < \text{next}_x$ 的区间 mex 值会发生改变。

分析可知每次删除左端点，珂朵莉树只会新增一个节点（我们规定只修改了右端点的节点不算新建）和删除若干节点。

当我们删除节点 (l, r, v) 时，取出所有的区间 $[x, l]$ 就可以取到所有的极小 mex 区间，虽然这些区间不一定满足极小的定义，但被淘汰的区间一定不是极小的。

关于极大 mex 区间，一种做法就是新建一个节点时，取出 $[x, r]$ ，另一种做法就是考虑由所有极小 mex 区间去扩展，找到左右两侧第一个为 mex 的数即可。

时间复杂度均为 $O(n \log n)$ 。

```

1
2 struct Node {
3     int l, r, v;
4     bool operator<(const Node& a) const { return l < a.l; }
5 };
6
7 vector<pair<int, int>> seg[N]; // 按照 mex 值存放极小区间。
8 void solve() {
9     set<int> s;
10    for (int i = 1; i <= n; i++) {
11        s.insert(a[i]);
12        while (s.count(cur)) cur++;
13        b[i] = cur;
14    }
15
16    set<Node> odt;
17    for (int i = 1; i <= n; i++) {
18        int j = i;
19        while (j <= n && b[j] == b[i]) j++;
20        odt.insert({i, j - 1, b[i]});
21        i = j;
22    }
23    for (int i = 1; i <= n; i++) {

```

```

24     auto it = odt.lower_bound({lst[i], 0, 0});
25     assert(it != odt.begin());
26     it = prev(it);
27     int x = n + 1, y = 0;
28     while (it->v >= a[i]) {
29         auto [l, r, v] = *it;
30
31         seg[v].push_back({i, l});
32         x = min(x, l), y = max(r, y);
33         if (r >= lst[i]) odt.insert({lst[i], r, v});
34
35         if (it == odt.begin()) {
36             odt.erase(it);
37             break;
38         }
39
40         /*
41         C++ 17 后可以 odt.erase(it--);
42         */
43
44         auto nit = prev(it);
45         odt.erase(it);
46         it = nit;
47     }
48
49     y = min(y, lst[i] - 1);
50
51     if (x <= y) odt.insert({x, y, a[i]});
52     seg[a[i] == 0].push_back({i, i});
53     it = odt.begin();
54     int r = it->r, v = it->v;
55
56     odt.erase(it);
57     if (r > i) odt.insert({i + 1, r, v});
58 }
59 }

```

6.5 平面不相交矩形建树

假设每个矩形满足: $x_1 < x_2$, $y_1 < y_2$ 。

我们考虑在 x 这个维度从小到大扫描每个矩形的边界, 当扫描到 x_1 时, 加入对应矩形的贡献。

同时计算这个矩形的父亲。当扫描到 x_2 时, 减去对应矩形的贡献。

加入贡献的方法:

维护一个 $\text{set}<\text{pair}<\text{int}, \text{int}>\text{s}$, 插入 (y_1, id) 和 (y_2, id) 两个二元组, 也就是插入两条线。

计算父亲的方法:

在 s 中的找到第一条在当前矩形上方的线。则这条线对应的矩形要么是它的父亲, 要么有着相同的父亲。

正确性是因为 s 中那些不是它兄弟或父亲的线, 一定会被它的兄弟包裹。

使用线段树的方法:

使用一颗区间修改, 单点查询的线段树维护当前 x 值时 y 对应的矩形 (不存在就设置为虚根的下标)。按 x 的从小到大排序矩形的边, 遍历, 遇到左侧边时查询对应 y 范围内的任意 y 值属于的矩形, 将其设置为父亲, 再区间修改自己的 y 范围为自己的下标。遇到右侧边时区间修改自己的 y 范围为父亲的下标 (即还原操作)。

遍历完后, 森林里的所有树指向设置好的虚根。

6.6 无穷字符串匹配/比较

1. 周期引理

字符串的周期: S 有周期 d 当且仅当 $\forall i < |S|, S[i] = S[i \bmod d]$ 成立

字符串的完整周期: S 有完整周期 d 当且仅当 d 是 S 的周期且满足 $d \mid |S|$

字符串 S 有完整周期 p, q , 那么有完整周期 $\gcd(p, q)$

****Fine-Wilf 定理:****

字符串 S 有周期 p, q , 且 $|S| \geq p + q - \gcd(p, q)$ 那么有周期 $\gcd(p, q)$

一个简单的证明:

周期 p 将 S 划分成 p 个等价类, 还要添加 $p - \gcd(p, q)$ 个等价关系使得 S 可以划分为 $\gcd(p, q)$ 个等价类, 而这可以通过在周期 q 后添加 $p - \gcd(p, q)$ 个字符实现, 故需要 $|S| \geq p + q - \gcd(p, q)$

边界实例: 斐波那契字符串

定义 $f_1 = a, f_2 = b, f_n = f_{n-1} + f_{n-2}$ (注意连接顺序)

那么 $g_n = f_n[1 : |f_n| - 2]$ 即为边界的实例 (如 $g_7 = babbababbab$, 有周期 5, 8)

2. 判断无穷字符串相等:

如果字符串 $a^\infty = b^\infty$, 那么设字符串 c 是字符串 a 和 b 的本原字符串, 那么 $a = c^{k_1}, b = c^{k_2}$, 所以我们只需求出来 a 和 b 的最小周期字符串来比较即可。

也可以用下面所描述的算法。

3. 无穷字符串匹配:

如果 $a^\infty = b^\infty$, 那么就完全匹配, 否则我们可以找第一个失配点, 即 $\text{lcp}(a^\infty, b^\infty)$ 的长度。

根据 Fine-Wilf 定理, 我们只需要检查 a^∞, b^∞ 的前 $|a| + |b| - \gcd(|a|, |b|)$ 个字符。

如果相等, 根据 Fine-Wilf 定理, 我们可以得到 a^∞, b^∞ 的一个共同的周期 $c = \gcd(|a|, |b|)$, 那么 a^∞, b^∞ 其实都可以表示成 c^∞ , 即 $a^\infty = b^\infty$ 。

否则, 我们通过暴力匹配找到了第一个失配点, 而暴力匹配的不会超过 $|a| + |b| - \gcd(|a|, |b|)$

对于多个无穷字符串 s_i^∞ 的匹配/建立字典树问题, 我们可以把它们先转化成本原字符串, 然后按照字符串长度从大到小插入到 trie, 但是为了保证出现失配点但不浪费, 我们插入的应该是 $s_i^\infty[1 : \max(2|s_i|, \text{maxlen})]$, 其中 maxlen 是字典树中 **当前子树** 最深节点的深度, 所以插入的深度实际上是动态决定的, 而至少为长度的 2 倍则是为了让第一个失配点必须出现 ($2|s_i| \geq |s_i| + |s_j| - \gcd(|s_i|, |s_j|)$)。这保证了我们插入操作是正确的且是最少的。

这里我们给出一个对应的字典树代码

```

1 struct trie {
2     struct Node {
3         array<int, 3> next;
4         int cnt;
5         int len;
6         int maxlen;
7         Node() : next{}, cnt{}, len{}, maxlen{} {};
8     };
9     vector<Node> t;
10
11     trie() {
12         t.assign(2, Node());
13         t[0].next.fill(1);
14         t[0].cnt = 0;
15         t[0].len = 0;
16         t[0].maxlen = 0;
17     }
18
19     int newNode() {

```

```

20     t.emplace_back();
21     return t.size() - 1;
22 }
23
24 int num(char c) { return c - 'a'; }
25
26 void add(const string& s, int weight) {
27     int p = 1;
28     int len = s.length();
29     int maxlen = 0;
30     string x;
31     for (int i = 0; i < len * 2 || i < t[p].maxlen; i++) {
32         if (next(p, s[i % len]) == 0) {
33             next(p, s[i % len]) = newNode();
34             t[next(p, s[i % len])].len = t[p].len + 1;
35         }
36         x += s[i % len];
37         p = next(p, s[i % len]);
38         t[p].cnt += weight;
39         maxlen++;
40     }
41     p = 1;
42     for (int i = 0; i < maxlen; i++) {
43         p = next(p, s[i % len]);
44         t[p].maxlen = max(t[p].maxlen, maxlen);
45     }
46
47     return;
48 }
49
50 int& next(int p, char c) { return t[p].next[num(c)]; }
51 };

```

6.7 曼哈顿和切比雪夫

对于任意两点 $P_1(x_1, y_1)$ 和 $P_2(x_2, y_2)$, 以及它们在 $(u, v) = (x - y, x + y)$ 坐标系下的对应点 $P'_1(u_1, v_1)$ 和 $P'_2(u_2, v_2)$:

1. 原坐标系的曼哈顿距离等于新坐标系的切比雪夫距离:

$$d_M(P_1, P_2) = d_C(P'_1, P'_2)$$

1. 原坐标系的切比雪夫距离等于新坐标系曼哈顿距离的一半:

$$d_C(P_1, P_2) = \frac{1}{2} d_M(P'_1, P'_2)$$

6.8 接受有效日期的最小状态自动机

0 代表没有转移边

根节点: 1 号节点

闰根和平根: 12, 13 节点

终止节点 (输入一个合法序列后转移到的节点): 26

```

1 {2, 3, 4, 3, 4, 3, 4, 3, 4, 3};
2 {5, 7, 7, 7, 6, 7, 7, 7, 6, 7};
3 {7, 7, 6, 7, 7, 7, 6, 7, 7, 7};
4 {6, 7, 7, 7, 6, 7, 7, 7, 6, 7};

```

```
5 {8, 10, 11, 10, 11, 10, 11, 10, 11, 10};
6 {11, 10, 11, 10, 11, 10, 11, 10, 11, 10};
7 {9, 10, 11, 10, 11, 10, 11, 10, 11, 10};
8 {0, 13, 13, 13, 12, 13, 13, 13, 12, 13};
9 {13, 13, 13, 13, 12, 13, 13, 13, 12, 13};
10 {13, 13, 12, 13, 13, 13, 12, 13, 13, 13};
11 {12, 13, 13, 13, 12, 13, 13, 13, 12, 13};
12 {14, 16, 0, 0, 0, 0, 0, 0, 0, 0};
13 {15, 16, 0, 0, 0, 0, 0, 0, 0, 0};
14 {0, 18, 17, 18, 19, 18, 19, 18, 18, 19};
15 {0, 18, 20, 18, 19, 18, 19, 18, 18, 19};
16 {18, 19, 18, 0, 0, 0, 0, 0, 0, 0};
17 {21, 25, 25, 0, 0, 0, 0, 0, 0, 0};
18 {21, 25, 25, 23, 0, 0, 0, 0, 0, 0};
19 {21, 25, 25, 22, 0, 0, 0, 0, 0, 0};
20 {21, 25, 24, 0, 0, 0, 0, 0, 0, 0};
21 {0, 26, 26, 26, 26, 26, 26, 26, 26, 26};
22 {26, 0, 0, 0, 0, 0, 0, 0, 0, 0};
23 {26, 26, 0, 0, 0, 0, 0, 0, 0, 0};
24 {26, 26, 26, 26, 26, 26, 26, 26, 26, 0};
25 {26, 26, 26, 26, 26, 26, 26, 26, 26, 26};
26 {0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
```

7. Appendix

7.1 Formulas 公式表

7.1.1 Mobius Inversion

$$F(n) = \sum_{d|n} f(d) \Rightarrow f(n) = \sum_{d|n} \mu(d) F\left(\frac{n}{d}\right)$$

$$F(n) = \sum_{n|d} f(d) \Rightarrow f(n) = \sum_{n|d} \mu\left(\frac{d}{n}\right) F(d)$$

$$[n = 1] = \sum_{d|n} \mu(d)$$

7.1.2 杜教筛

$$S_\varphi(n) = \frac{n(n+1)}{2} - \sum_{d=2}^n S_\varphi\left(\left\lfloor \frac{n}{d} \right\rfloor\right)$$

$$S_\mu(n) = 1 - \sum_{d=2}^n S_\mu\left(\left\lfloor \frac{n}{d} \right\rfloor\right)$$

7.1.3 降幂公式

$$a^k \equiv a^{k \bmod \varphi(p) + \varphi(p)}, \quad k \geq \varphi(p)$$

7.1.4 其他常用公式

$$\sum_{\substack{1 \leq i \leq n \\ (i,n)=1}} i = n \frac{\varphi(n) + [n=1]}{2}$$

$$\sum_{i=1}^n \sum_{\substack{1 \leq j \leq i \\ (i,j)=d}} 1 = S_\varphi\left(\left\lfloor \frac{n}{d} \right\rfloor\right)$$

$$\sum_{\substack{1 \leq i \leq n \\ 1 \leq j \leq m \\ (i,j)=d}} 1 = \sum_{k=1}^{\infty} \mu(k) \left\lfloor \frac{n}{dk} \right\rfloor \left\lfloor \frac{m}{dk} \right\rfloor$$

$$\sum_{i=1}^n f(i) \sum_{j=1}^{\lfloor n/i \rfloor} g(j) = \sum_{i=1}^n g(i) \sum_{j=1}^{\lfloor n/i \rfloor} f(j)$$

7.1.5 单位根反演

$$\sum_{i=0}^{\lfloor \frac{n}{k} \rfloor} C_n^{ik}$$

引理

$$\frac{1}{k} \sum_{i=0}^{k-1} \omega_k^{in} = [k \mid n]$$

反演

$$\begin{aligned}
 Ans &= \sum_{i=0}^n C_n^i [k \mid i] \\
 &= \sum_{i=0}^n C_n^i \left(\frac{1}{k} \sum_{j=0}^{k-1} \omega_k^{ij} \right) \\
 &= \frac{1}{k} \sum_{i=0}^n C_n^i \sum_{j=0}^{k-1} \omega_k^{ij} \\
 &= \frac{1}{k} \sum_{j=0}^{k-1} \left(\sum_{i=0}^n C_n^i (\omega_k^j)^i \right) \\
 &= \frac{1}{k} \sum_{j=0}^{k-1} (1 + \omega_k^j)^n
 \end{aligned}$$

另, 如果要求的是 $[n \% k = t]$, 其实就是 $[k \mid (n - t)]$. 同理推式子即可.

7.1.6 Arithmetic Function

$$\begin{aligned}
 (p-1)! &\equiv -1 \pmod{p} \\
 a > 1, m, n > 0 &\Rightarrow \gcd(a^m - 1, a^n - 1) = a^{\gcd(m, n)} - 1 \\
 \mu^2(n) &= \sum_{d^2 \mid n} \mu(d) \\
 a > b, \gcd(a, b) = 1 &\Rightarrow \gcd(a^m - b^m, a^n - b^n) = a^{\gcd(m, n)} - b^{\gcd(m, n)} \\
 \prod_{\substack{k=1 \\ \gcd(k, m)=1}}^m k &\equiv \begin{cases} -1 & \pmod{m}, m = 4, p^q, 2p^q \\ 1 & \pmod{m}, \text{otherwise} \end{cases} \\
 \sigma_k(n) &= \sum_{d \mid n} d^k = \prod_{i=1}^{\omega(n)} \frac{p_i^{(a_i+1)k} - 1}{p_i^k - 1} \\
 J_k(n) &= n^k \prod_{p \mid n} \left(1 - \frac{1}{p^k} \right)
 \end{aligned}$$

$J_k(n)$ is the number of k -tuples of positive integers all less than or equal to n that form a coprime $(k+1)$ -tuple together with n .

$$\begin{aligned}
 \sum_{\delta \mid n} J_k(\delta) &= n^k \\
 \sum_{\substack{1 \leq i \leq n \\ 1 \leq j \leq n \\ \gcd(i, j)=1}} ij &= \sum_{i=1}^n i^2 \varphi(i) \\
 \sum_{\delta \mid n} \delta^s J_r(\delta) J_s\left(\frac{n}{\delta}\right) &= J_{r+s}(n) \\
 \sum_{\delta \mid n} \varphi(\delta) d\left(\frac{n}{\delta}\right) &= \sigma(n), \quad \sum_{\delta \mid n} |\mu(\delta)| = 2^{\omega(n)} \\
 \sum_{\delta \mid n} 2^{\omega(\delta)} &= d(n^2), \quad \sum_{\delta \mid n} d(\delta^2) = d^2(n) \\
 \sum_{\delta \mid n} d\left(\frac{n}{\delta}\right) 2^{\omega(\delta)} &= d^2(n), \quad \sum_{\delta \mid n} \frac{\mu(\delta)}{\delta} = \frac{\varphi(n)}{n} \\
 \sum_{\delta \mid n} \frac{\mu(\delta)}{\varphi(\delta)} &= d(n), \quad \sum_{\delta \mid n} \frac{\mu^2(\delta)}{\varphi(\delta)} = \frac{n}{\varphi(n)}
 \end{aligned}$$

$$\begin{aligned}
 & n|\varphi(a^n - 1) \\
 & \sum_{\substack{1 \leq k \leq n \\ \gcd(k, n) = 1}} f(\gcd(k - 1, n)) = \varphi(n) \sum_{d|n} \frac{(\mu * f)(d)}{\varphi(d)} \\
 & \varphi(\text{lcm}(m, n))\varphi(\gcd(m, n)) = \varphi(m)\varphi(n) \\
 & \sum_{\delta|n} d^3(\delta) = \left(\sum_{\delta|n} d(\delta) \right)^2 \\
 & d(uv) = \sum_{\delta|\gcd(u, v)} \mu(\delta) d\left(\frac{u}{\delta}\right) d\left(\frac{v}{\delta}\right) \\
 & \sigma_k(u)\sigma_k(v) = \sum_{\delta|\gcd(u, v)} \delta^k \sigma_k\left(\frac{uv}{\delta^2}\right) \\
 & \mu(n) = \sum_{\substack{k=1 \\ \gcd(k, n)=1}}^n \cos\left(2\pi \frac{k}{n}\right) \\
 & \varphi(n) = \sum_{\substack{k=1 \\ \gcd(k, n)=1}}^n 1 = \sum_{k=1}^n \gcd(k, n) \cos\left(2\pi \frac{k}{n}\right) \\
 & \begin{cases} S(n) = \sum_{k=1}^n (f * g)(k) \\ \sum_{k=1}^n S\left(\left\lfloor \frac{n}{k} \right\rfloor\right) = \sum_{i=1}^n f(i) \sum_{j=1}^{\lfloor n/i \rfloor} (g * 1)(j) \end{cases} \\
 & \begin{cases} S(n) = \sum_{k=1}^n (f \cdot g)(k), g \text{ is completely multiplicative} \\ \sum_{k=1}^n S\left(\left\lfloor \frac{n}{k} \right\rfloor\right) g(k) = \sum_{k=1}^n (f * 1)(k) g(k) \end{cases}
 \end{aligned}$$

7.1.7 组合数

C \	0	1	2	3	4	5	6	7	8	9	10
0	1										
1	1	1									
2	1	2	1								
3	1	3	3	1							
4	1	4	6	4	1						
5	1	5	10	10	5	1					
6	1	6	15	20	15	6	1				
7	1	7	21	35	35	21	7	1			
8	1	8	28	56	70	56	28	8	1		
9	1	9	36	84	126	126	84	36	9	1	
10	1	10	45	120	210	252	210	120	45	10	1

$$\begin{aligned}
 & \binom{n}{k} \equiv [n \& k = k] \pmod{2} \\
 & \sum_{k=0}^n \binom{k}{m} = \binom{n+1}{m+1} \\
 & \sqrt{1+z} = 1 + \sum_{k=1}^{\infty} \frac{(-1)^{k-1}}{k \cdot 2^{2k-1}} \binom{2k-2}{k-1} z^k \\
 & \sum_{k=0}^r \binom{r-k}{m} \binom{s+k}{n} = \binom{r+s+1}{m+n+1} \\
 & C_{n,m} = \binom{n+m}{m} - \binom{n+m}{m-1}, n \geq m \\
 & \binom{n}{k} = (-1)^k \binom{k-n-1}{k}, \sum_{k \leq n} \binom{r+k}{k} = \binom{r+n+1}{n}
 \end{aligned}$$

$$\binom{n_1 + \cdots + n_p}{m} = \sum_{k_1 + \cdots + k_p = m} \binom{n_1}{k_1} \cdots \binom{n_p}{k_p}$$

$$\sum_{i=0}^k \binom{n}{i} \binom{m}{k-i} = \binom{n+m}{k}$$

7.1.8 Fibonacci Numbers, Lucas Numbers

$$F(z) = \frac{z}{1 - z - z^2}$$

$$\hat{\phi} = \frac{1 - \sqrt{5}}{2}$$

$$\sum_{k=1}^n f_k = f_{n+2} - 1, \quad \sum_{k=1}^n f_k^2 = f_n f_{n+1}$$

$$\sum_{k=0}^n f_k f_{n-k} = \frac{1}{5}(n-1)f_n + \frac{2}{5}n f_{n-1}$$

$$\frac{f_{2n}}{f_n} = f_{n-1} + f_{n+1}$$

$$f_1 + 2f_2 + 3f_3 + \cdots + n f_n = n f_{n+2} - f_{n+3} + 2$$

$$\gcd(f_m, f_n) = f_{\gcd(m, n)}$$

$$f_n^2 + (-1)^n = f_{n+1} f_{n-1}$$

$$f_{n+k} = f_n f_{k+1} + f_{n-1} f_k$$

$$f_{2n+1} = f_n^2 + f_{n+1}^2$$

$$(-1)^k f_{n-k} = f_n f_{k-1} - f_{n-1} f_k$$

```

1 def fib(n): # F(n), F(n + 1)
2     if not n: return (0, 1)
3     a, b = fib(n >> 1)
4     c = a * (2 * b - a)
5     d = a * a + b * b
6     if n & 1:
7         return (d, c + d)
8     else:
9         return (c, d)

```

$$\text{Modulo } f_n, f_{mn+r} \equiv \begin{cases} f_r, & m \bmod 4 = 0; \\ (-1)^{r+1} f_{n-r}, & m \bmod 4 = 1; \\ (-1)^n f_r, & m \bmod 4 = 2; \\ (-1)^{r+1+n} f_{n-r}, & m \bmod 4 = 3. \end{cases}$$

$$L_0 = 2, L_1 = 1, L_n = L_{n-1} + L_{n-2} = \left(\frac{1 + \sqrt{5}}{2} \right)^n + \left(\frac{1 - \sqrt{5}}{2} \right)^n$$

$$L(x) = \frac{2 - x}{1 - x - x^2}$$

除了 $n = 0, 4, 8, 16$, 若 L_n 是素数, 则 n 是素数.

$$\phi = \frac{1 + \sqrt{5}}{2}, \quad \hat{\phi} = \frac{1 - \sqrt{5}}{2}$$

$$F_n = \frac{\phi^n - \hat{\phi}^n}{\sqrt{5}}, \quad L_n = \phi^n + \hat{\phi}^n$$

$$\frac{L_n + F_n \sqrt{5}}{2} = \left(\frac{1 + \sqrt{5}}{2} \right)^n$$

7.1.9 Sum of Powers

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}, \quad \sum_{i=1}^n i^3 = \left(\frac{n(n+1)}{2} \right)^2$$

$$\sum_{i=1}^n i^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

$$\sum_{i=1}^n i^5 = \frac{n^2(n+1)^2(2n^2+2n-1)}{12}$$

7.1.10 Catalan Numbers 1, 1, 2, 5, 14, 42, 132, 429, 1430...

$$c_0 = 1, c_n = \sum_{i=0}^{n-1} c_i c_{n-1-i} = c_{n-1} \frac{4n-2}{n+1} = \frac{\binom{2n}{n}}{n+1} = \binom{2n}{n} - \binom{2n}{n-1}$$

$$c(x) = \frac{1 - \sqrt{1-4x}}{2x}$$

Usage: n 对括号序列; n 个点满二叉树; $n \times n$ 的方格左下到右上不过对角线方案数; 凸 $n+2$ 边形三角形分割数; n 个数的出栈方案数; $2n$ 个顶点连接, 线段两两不交的方案数.

类卡特兰数 从 $(1, 1)$ 出发走到 (n, m) , 只能向右或者向上走, 不能越过 $y = x$ 这条线 (即保证 $x \geq y$), 合法方案数是 $\binom{n+m-2}{n-1} - \binom{n+m-2}{n-2}$.

7.1.11 Motzkin Numbers 1, 1, 2, 4, 9, 21, 51, 127, 323, 835...

圆上 n 点间画不相交弦的方案数. 选 n 个数 $k_1, k_2, \dots, k_n \in \{-1, 0, 1\}$, 保证 $\sum_i^a k_i (1 \leq a \leq n)$ 非负且所有数总和为 0 的方案数.

$$M_{n+1} = M_n + \sum_{i=1}^{n-1} M_i M_{n-1-i} = \frac{(2n+3)M_n + 3nM_{n-1}}{n+3}$$

$$M_n = \sum_{k=0}^{\lfloor n/2 \rfloor} \binom{n}{2k} \text{Catalan}(k)$$

$$M(X) = \frac{1 - x - \sqrt{1 - 2x - 3x^2}}{2x^2}$$

7.1.12 Derangement **错排数** 0, 1, 2, 9, 44, 265, 1854, 14833...

$$D_1 = 0, D_2 = 1, D_n = n! \left(\frac{1}{0!} - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \dots + \frac{(-1)^n}{n!} \right)$$

$$D_n = (n-1)(D_{n-1} + D_{n-2})$$

7.1.13 Bell Numbers 1, 1, 2, 5, 15, 52, 203, 877, 4140 ...

n 个元素集合划分的方案数.

$$B_n = \sum_{k=1}^n \left\{ \begin{matrix} n \\ k \end{matrix} \right\}, \quad B_{n+1} = \sum_{k=0}^n \binom{n}{k} B_k$$

$$B_{p^m+n} \equiv mB_n + B_{n+1} \pmod{p}$$

$$B(x) = \sum_{n=0}^{\infty} \frac{B_n}{n!} x^n = e^{e^x-1}$$

7.1.14 Stirling Numbers

第一类 n 个元素集合分作 k 个**非空轮换**方案数.

$$\begin{bmatrix} n+1 \\ k \end{bmatrix} = n \begin{bmatrix} n \\ k \end{bmatrix} + \begin{bmatrix} n \\ k-1 \end{bmatrix}$$

$$s(n, k) = (-1)^{n-k} \begin{bmatrix} n \\ k \end{bmatrix}$$

n\k	0	1	2	3	4	5	6
0	1						
1	0	1					
2	0	1	1				
3	0	2	3	1			
4	0	6	11	6	1		
5	0	24	50	35	10	1	
6	0	120	274	225	85	15	1
7	0	720	1764	1624	735	175	21

$$\begin{aligned}\left[\begin{matrix} n+1 \\ 2 \end{matrix} \right] &= n!H_n \text{ (see 7.1.16)} \\ x^n &= \sum_k \left[\begin{matrix} n \\ k \end{matrix} \right] (-1)^{n-k} x^k \\ x^{\bar{n}} &= \sum_k \left[\begin{matrix} n \\ k \end{matrix} \right] x^k\end{aligned}$$

For fixed k , EGF:

$$\sum_{n=k}^{\infty} \left[\begin{matrix} n \\ k \end{matrix} \right] \frac{x^n}{n!} = \frac{1}{k!} \left(\ln \frac{1}{1-x} \right)^k$$

第二类 把 n 个元素集合分作 k 个**非空子集**方案数.

$$\begin{aligned}\left\{ \begin{matrix} n+1 \\ k \end{matrix} \right\} &= k \left\{ \begin{matrix} n \\ k \end{matrix} \right\} + \left\{ \begin{matrix} n \\ k-1 \end{matrix} \right\} \\ m! \left\{ \begin{matrix} n \\ m \end{matrix} \right\} &= \sum_k \binom{m}{k} k^n (-1)^{m-k}\end{aligned}$$

n\k	0	1	2	3	4	5	6
0	1						
1	0	1					
2	0	1	1				
3	0	1	3	1			
4	0	1	7	6	1		
5	0	1	15	25	10	1	
6	0	1	31	90	65	15	1
7	0	1	63	301	350	140	21

$$\begin{aligned}x^n &= \sum_k \left\{ \begin{matrix} n \\ k \end{matrix} \right\} x^k \\ &= \sum_k \left\{ \begin{matrix} n \\ k \end{matrix} \right\} (-1)^{n-k} x^{\bar{k}}\end{aligned}$$

For fixed k , EGF and OGF:

$$\begin{aligned}\sum_{n=k}^{\infty} \left\{ \begin{matrix} n \\ k \end{matrix} \right\} \frac{x^n}{n!} &= \frac{(e^x - 1)^k}{k!} \\ \sum_{n=k}^{\infty} \left\{ \begin{matrix} n \\ k \end{matrix} \right\} x^n &= \frac{x^k}{\prod_{i=1}^k (1 - ix)}\end{aligned}$$

7.1.15 Eulerian Numbers

n\k	0	1	2	3	4	5	6
1	1						
2	1	1					
3	1	4	1				
4	1	11	11	1			
5	1	26	66	26	1		
6	1	57	302	302	57	1	
7	1	120	1191	2416	1191	120	1

$$\begin{aligned}\left\langle \begin{matrix} n \\ k \end{matrix} \right\rangle &= (k+1) \left\langle \begin{matrix} n-1 \\ k \end{matrix} \right\rangle + (n-k) \left\langle \begin{matrix} n-1 \\ k-1 \end{matrix} \right\rangle \\ x^n &= \sum_k \left\langle \begin{matrix} n \\ k \end{matrix} \right\rangle \binom{x+k}{n}\end{aligned}$$

$$\left\langle \begin{matrix} n \\ m \end{matrix} \right\rangle = \sum_{k=0}^m \binom{n+1}{k} (m+1-k)^n (-1)^k$$

7.1.16 Harmonic Numbers, 1, 3/2, 11/6, 25/12, 137/60...

$$H_n = \sum_{k=1}^n \frac{1}{k}, \sum_{k=1}^n H_k = (n+1)H_n - n$$

$$\sum_{k=1}^n kH_k = \frac{n(n+1)}{2}H_n - \frac{n(n-1)}{4}$$

$$\sum_{k=1}^n \binom{k}{m} H_k = \binom{n+1}{m+1} \left(H_{n+1} - \frac{1}{m+1} \right)$$

7.1.17 卡迈克尔函数

卡迈克尔函数表示模 m 剩余系下最大的阶, 即 $\lambda(m) = \max_{a \perp m} \delta_m(a)$.

容易看出, 若 $\lambda(m) = \varphi(m)$, 则 m 存在原根.

该函数可由下述方法计算: 分解质因数 $m = p_1^{\alpha_1} p_2^{\alpha_2} \dots p_t^{\alpha_t}$. 则 $\lambda(m) = \text{lcm}(\lambda(p_1^{\alpha_1}), \lambda(p_2^{\alpha_2}), \dots, \lambda(p_t^{\alpha_t}))$. 其中对奇质数 p , $\lambda(p^\alpha) = (p-1)p^{\alpha-1}$. 对 2 的幂, $\lambda(2^k) = 2^{k-2}$, s.t. $k \geq 3$. $\lambda(4) = \lambda(2) = 2$.

7.1.18 求拆分数

```

1 def penta(k):
2     return k*(3*k-1)//2
3 def compute_partition(goal):
4     p = [1]
5     for n in range(1,goal+1):
6         p.append(0)
7         for k in range(1,n+1):
8             c = (-1)**(k+1)
9             for t in [penta(k), penta(-k)]:
10                 if (n-t) >= 0:
11                     p[n] = p[n] + c*p[n-t]
12     return p
    
```

$\Phi(x) = \prod_{n=1}^{\infty} (1-x^n) = \sum_{k=-\infty}^{\infty} (-1)^k x^{k(3k-1)/2}$ 记 $p(n)$ 表示 n 的拆分数, $f(n, k)$ 表示将 n 拆分且每种数字使用次数必须小于等于 k 的拆分数. 则

$$P(x)\Phi(x) = 1, F_k(x) = P(x)\Phi(x^{k+1})$$

暴力拆开卷积, 可以得到将 1, -1, 2, -2... 带入五边形数 $k(3k-1)/2$ 中, 由于小于 n 的五边形数只有 $O(\sqrt{n})$ 个, 可以 $O(n\sqrt{n})$ 计算答案:

$$p(n) = p(n-1) + p(n-2) - p(n-5) - p(n-7) + \dots$$

$$f(n, k) = p(n) - p(n-k-1) - p(n-2(k+1)) + \dots$$

7.1.19 Bernoulli Numbers 1, 1/2, 1/6, 0, -1/30, 0, 1/42 ...

$$B(x) = \sum_{i \geq 0} \frac{B_i x^i}{i!} = \frac{x}{e^x - 1}$$

$$B_n = [n=0] - \sum_{k=0}^{n-1} \binom{n}{k} \frac{B_k}{n-k+1}, \sum_{i=0}^n \binom{n+1}{i} B_i = 0$$

$$S_n(m) = \sum_{i=0}^{m-1} i^n = \frac{1}{n+1} \sum_{k=0}^n \binom{n+1}{k} B_k m^{n+1-k}$$

$B_0 = 1, B_1 = -\frac{1}{2}, B_4 = -\frac{1}{30}, B_6 = \frac{1}{42}, B_8 = -\frac{1}{30}, \dots$ (除了 $B_1 = -\frac{1}{2}$ 以外, 伯努利数的奇数项都是 0.) 自然数幂次和关于次数的 EGF:

$$F(x) = \sum_{k=0}^{\infty} \frac{\sum_{i=0}^n i^k}{k!} x^k = \sum_{i=0}^n e^{ix} = \frac{e^{(n+1)x} - 1}{e^x - 1}$$

7.1.20 k-th MAX-MIN 反演

$$k\text{-th max}(S) = \sum_{\emptyset \neq T \subseteq S} (-1)^{|T|-k} \binom{|T|-1}{k-1} \min(T)$$

代入 $k = 1$ 即为 MAX-MIN 反演:

$$\max(S) = \sum_{\emptyset \neq T \subseteq S} (-1)^{|T|-1} \min(T)$$

7.1.21 伍德伯里矩阵等式

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1}$$

该等式可以动态维护矩阵的逆, 令 $C = [1]$, U, V 分别为 $1 \times n$ 和 $n \times 1$ 的向量, 这样可以构造出 UCV 为只有某行或者某列不为 0 的矩阵, 一次修改复杂度为 $O(n^2)$.

7.1.22 Sum of Squares

$r_k(n)$ 表示用 k 个平方数组成 n 的方案数. 假设:

$$n = 2^{a_0} p_1^{2a_1} \cdots p_r^{2a_r} q_1^{b_1} \cdots q_s^{b_s}$$

其中 $p_i \equiv 3 \pmod{4}$, $q_i \equiv 1 \pmod{4}$, 那么

$$r_2(n) = \begin{cases} 0 & \text{if any } a_i \text{ is a half-integer} \\ 4 \prod_{i=1}^s (b_i + 1) & \text{if all } a_i \text{ are integers} \end{cases}$$

$r_3(n) > 0$ 当且仅当 n 不满足 $4^a(8b+7)$ 的形式 (a, b 为整数).

7.1.23 枚举勾股数 Pythagorean Triple

枚举 $x^2 + y^2 = z^2$ 的三元组: 可令 $x = m^2 - n^2, y = 2mn, z = m^2 + n^2$, 枚举 m 和 n 即可 $O(\sqrt{N})$ 枚举勾股数. 判断素勾股数方法: m, n 至少一个为偶数并且 m, n 互质, 那么 x, y, z 就是素勾股数.

7.1.24 四面体体积 Tetrahedron Volume

If U, V, W, u, v, w are lengths of edges of the tetrahedron (first three form a triangle; u opposite to U and so on)

$$V = \frac{\sqrt{4u^2v^2w^2 - \sum_{cyc} u^2(v^2 + w^2 - U^2)^2 + (v^2 + w^2 - U^2)(w^2 + u^2 - V^2)(u^2 + v^2 - W^2)}}{12}$$

7.1.25 杨氏矩阵与钩子公式

满足: 格子 (i, j) 没有元素, 则它右边和上边相邻格子也没有元素; 格子 (i, j) 有元素 $a[i][j]$, 则它右边和上边相邻格子要么没有元素, 要么有元素且比 $a[i][j]$ 大.

计数: $F_1 = 1, F_2 = 2, F_n = F_{n-1} + (n-1)F_{n-2}, F(x) = e^{x + \frac{x^2}{2}}$

钩子公式: 对于给定形状 λ , 不同杨氏矩阵的个数为:

$$d_\lambda = \frac{n!}{\prod_{(i,j) \in \lambda} h_\lambda(i,j)}$$

$h_\lambda(i, j)$ 表示该格子右边和上边的格子数量加 1 (自身也算 1).

7.1.26 常见博弈游戏

Nim-K 游戏 n 堆石子轮流拿, 每次最多可以拿 k 堆石子, 谁走最后一步输. 结论: 把每一堆石子的 sg 值 (即石子数量) 二进制分解, 先手必败当且仅当每一位二进制位上 1 的个数是 $(k+1)$ 的倍数.

Anti-Nim 游戏 n 堆石子轮流拿, 谁走最后一步输. 结论: 先手胜当且仅当 1. 所有堆石子数都为 1 且游戏的 SG 值为 0 (即有偶数个孤单堆-每堆只有 1 个石子数) 2. 存在某堆石子数大于 1 且游戏的 SG 值不为 0.

斐波那契博弈 有一堆物品, 两人轮流取物品, 先手最少取一个, 至多无上限, 但不能把物品取完, 之后每次取的物品数不能超过上一次取的物品数的二倍且至少为一件, 取走最后一件物品的人获胜. 结论: 先手胜当且仅当物品数 n 不是斐波那契数.

威佐夫博弈 有两堆石子, 博弈双方每次可以取一堆石子中的任意个, 不能不取, 或者取两堆石子中的相同个. 先取完者赢. 结论: 求出两堆石子 A 和 B 的差值 C , 如果 $\lfloor C \cdot \frac{\sqrt{5}+1}{2} \rfloor = \min(A, B)$ 那么后手赢, 否则先手赢.

约瑟夫环 n 个人 $0, \dots, n-1$, 令 $f(i, m)$ 为 i 个人报 m 的胜利者, 则 $f(1, m) = 0, f(i, m) = (f(i-1, m) + m) \bmod i$.

阶梯 Nim 在一个阶梯上, 每次选一个台阶上任意个石子移到下一个台阶上, 不可移动者输. 结论: SG 值等于奇数层台阶上石子数的异或和. 对于树形结构也适用, 奇数层节点上所有石子数异或起来即可.

图上博弈 给定无向图, 先手从某点开始走, 只能走相邻且未走过的点, 无法移动者输. 对该图求最大匹配, 若某个点不一定在最大匹配中则先手必败, 否则先手必胜.

最大最小定理求纳什均衡点 在二人零和博弈中, 可以用以下方式求出一个纳什均衡点: 在博弈双方中**任选**一方, 求混合策略 p 使得对方选择任意一个**纯策略**时, 己方的最小收益最大 (等价于对方的最大收益最小). 据此可以求出双方在此局面下的最优期望得分, 分别等于己方最大的最小收益和对方最小的最大收益. 一般而言, 可以得到形如

$$\max_p \min_i \sum_{p_j \in p, p_j \geq 0, \sum p_j = 1} p_j w_{i,j}$$

的形式. 当 $\sum p_j w_{i,j}$ 可以表示成只与 i 有关的函数 $f(i)$ 时, 可以令初始时 $p_i = 0$, 不断调整 $\sum p_j w_{i,j}$ 最小的那个 i 的概率 p_i , 直至无法调整或者 $\sum p_j = 1$ 为止.

7.1.27 概率相关

$$D(X) = E((X - E(X))^2) = E(X^2) - (E(X))^2, D(X + Y) = D(X) + D(Y),$$

$$D(aX) = a^2 D(X), E[X] = \sum_{i=1}^{\infty} P(X \geq i) \text{ (for non-negative integer values), } m \text{ 个数的方差: } s^2 = \frac{\sum_{i=1}^m x_i^2}{m} - \bar{x}^2$$

7.1.28 邻接矩阵行列式的意义

在无向图中取若干个环, 一种取法权值就是边权的乘积, 对行列式的贡献是 $(-1)^{\text{even}}$, 其中 even 是偶环的个数.

7.1.29 Others (某些近似数值公式在这里)

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$$

$$\cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{(2n)!} = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \dots$$

$$\ln(1+x) = \sum_{n=1}^{\infty} \frac{(-1)^{n+1} x^n}{n} = x - \frac{x^2}{2} + \frac{x^3}{3} - \dots \quad (\text{for } -1 < x \leq 1)$$

$$\arctan(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{2n+1} = x - \frac{x^3}{3} + \frac{x^5}{5} - \dots \quad (\text{for } -1 \leq x \leq 1)$$

$$(1+x)^k = \sum_{n=0}^{\infty} \binom{k}{n} x^n = 1 + kx + \frac{k(k-1)}{2!} x^2 + \dots$$

$$\arcsin(x) = x + \sum_{n=1}^{\infty} \frac{(2n-1)!!}{(2n)!!} \frac{x^{2n+1}}{2n+1}$$

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots, \quad \frac{1}{\pi} = \frac{2\sqrt{2}}{9801} \sum_{k=0}^{\infty} \frac{(4k)!(1103 + 26390k)}{(k!)^4 396^{4k}}$$

$$\sum_{i=1}^n \frac{1}{i} \approx \ln\left(n + \frac{1}{2}\right) + \gamma + \frac{1}{24(n+0.5)^2} \quad (\gamma \approx 0.57721566)$$

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \frac{1}{12n} + \frac{1}{288n^2} + O\left(\frac{1}{n^3}\right)\right)$$

$$\max\{a, b, c\} - \min\{a, b, c\} = \frac{1}{2} (|a-b| + |b-c| + |c-a|)$$

$$\sum_{k=0}^n kc^k = \frac{nc^{n+2} - (n+1)c^{n+1} + c}{(c-1)^2}$$

$$(a+b)(b+c)(c+a) = (a+b+c)(ab+bc+ca) - abc$$

$$a^3 + b^3 = (a+b)(a^2 - ab + b^2), a^3 - b^3 = (a-b)(a^2 + ab + b^2)$$

$n \bmod 2 = 1 :$

$$a^n + b^n = (a+b)(a^{n-1} - a^{n-2}b + \dots - ab^{n-2} + b^{n-1})$$

划分问题: n 个 $k-1$ 维超平面最多把 k 维空间分为 $\sum_{i=0}^k \binom{n}{i}$ 份.

7.2 Calculus, Integration Table 导数积分表

$(\frac{u}{v})' = \frac{u'v - uv'}{v^2}$	$(\arctan x)' = \frac{1}{1+x^2}$	$(\operatorname{arcsinh} x)' = \frac{1}{\sqrt{1+x^2}}$
$(a^x)' = (\ln a)a^x$	$(\operatorname{arccot} x)' = -\frac{1}{1+x^2}$	$(\operatorname{arccosh} x)' = \frac{1}{\sqrt{x^2-1}}$
$(\tan x)' = \sec^2 x$	$(\operatorname{arccsc} x)' = -\frac{1}{x\sqrt{1-x^2}}$	$(\operatorname{arctanh} x)' = \frac{1}{1-x^2}$
$(\cot x)' = \csc^2 x$	$(\operatorname{arcsec} x)' = \frac{1}{x\sqrt{1-x^2}}$	$(\operatorname{arccoth} x)' = \frac{1}{x^2-1}$
$(\sec x)' = \tan x \sec x$	$(\tanh x)' = \operatorname{sech}^2 x$	$(\operatorname{arcsch} x)' = -\frac{1}{ x \sqrt{1+x^2}}$
$(\csc x)' = -\cot x \csc x$	$(\coth x)' = -\operatorname{csch}^2 x$	$(\operatorname{arcsech} x)' = -\frac{1}{x\sqrt{1-x^2}}$
$(\arcsin x)' = \frac{1}{\sqrt{1-x^2}}$	$(\operatorname{sech} x)' = -\operatorname{sech} x \tanh x$	
$(\arccos x)' = -\frac{1}{\sqrt{1-x^2}}$	$(\operatorname{csch} x)' = -\operatorname{csch} x \coth x$	

· Integrals involving $ax^2 + bx + c$ ($a > 0$)

$$\int \frac{dx}{ax^2 + bx + c} = \begin{cases} \frac{2}{\sqrt{4ac-b^2}} \arctan \frac{2ax+b}{\sqrt{4ac-b^2}} + C & (b^2 < 4ac) \\ \frac{1}{\sqrt{b^2-4ac}} \ln \left| \frac{2ax+b-\sqrt{b^2-4ac}}{2ax+b+\sqrt{b^2-4ac}} \right| + C & (b^2 > 4ac) \end{cases}$$

$$\int \frac{x}{ax^2 + bx + c} dx = \frac{1}{2a} \ln |ax^2 + bx + c| - \frac{b}{2a} \int \frac{dx}{ax^2 + bx + c}$$

· Integrals involving $\sqrt{\pm ax^2 + bx + c}$ ($a > 0$)

$$\int \frac{dx}{\sqrt{ax^2 + bx + c}} = \frac{1}{\sqrt{a}} \ln \left| 2ax + b + 2\sqrt{a}\sqrt{ax^2 + bx + c} \right| + C$$

$$\int \sqrt{ax^2 + bx + c} dx = \frac{2ax+b}{4a} \sqrt{ax^2 + bx + c} + \frac{4ac-b^2}{8a\sqrt{a}} \ln \left| 2ax + b + 2\sqrt{a}\sqrt{ax^2 + bx + c} \right| + C$$

$$\int \frac{x}{\sqrt{ax^2 + bx + c}} dx = \frac{1}{a} \sqrt{ax^2 + bx + c} - \frac{b}{2a\sqrt{a}} \ln \left| 2ax + b + 2\sqrt{a}\sqrt{ax^2 + bx + c} \right| + C$$

$$\int \frac{dx}{\sqrt{c+bx-ax^2}} = \frac{1}{\sqrt{a}} \arcsin \frac{2ax-b}{\sqrt{b^2+4ac}} + C$$

$$\int \sqrt{c+bx-ax^2} dx = \frac{2ax-b}{4a} \sqrt{c+bx-ax^2} + \frac{b^2+4ac}{8a\sqrt{a}} \arcsin \frac{2ax-b}{\sqrt{b^2+4ac}} + C$$

$$\int \frac{x}{\sqrt{c+bx-ax^2}} dx = -\frac{1}{a} \sqrt{c+bx-ax^2} + \frac{b}{2a\sqrt{a}} \arcsin \frac{2ax-b}{\sqrt{b^2+4ac}} + C$$

· Integrals involving $\sqrt{(x-a)(b-x)}$, ($a < b$)

$$\int \frac{dx}{\sqrt{(x-a)(b-x)}} = 2 \arcsin \sqrt{\frac{x-a}{b-a}} + C$$

$$\int \sqrt{(x-a)(b-x)} dx = \frac{2x-a-b}{4} \sqrt{(x-a)(b-x)} + \frac{(b-a)^2}{8} \arcsin \frac{2x-a-b}{b-a} + C$$

Trigonometric Integrals (Constant +C is omitted)

$\int \tan x dx = -\ln \cos x $	$\int \csc^2 x dx = -\cot x$
$\int \cot x dx = \ln \sin x $	$\int \sec x \tan x dx = \sec x$
$\int \sec x dx = \ln \sec x + \tan x $	$\int \csc x \cot x dx = -\csc x$
$\int \csc x dx = \ln \csc x - \cot x $	$\int \sin^2 x dx = \frac{x}{2} - \frac{1}{4} \sin 2x$
$\int \sec^2 x dx = \tan x$	$\int \cos^2 x dx = \frac{x}{2} + \frac{1}{4} \sin 2x$

$$\begin{aligned}
 \int \sin^n x dx &= -\frac{1}{n} \sin^{n-1} x \cos x + \frac{n-1}{n} \int \sin^{n-2} x dx \\
 \int \cos^n x dx &= \frac{1}{n} \cos^{n-1} x \sin x + \frac{n-1}{n} \int \cos^{n-2} x dx \\
 \int \frac{dx}{\sin^n x} &= -\frac{\cos x}{(n-1) \sin^{n-1} x} + \frac{n-2}{n-1} \int \frac{dx}{\sin^{n-2} x} \\
 \int \frac{dx}{\cos^n x} &= \frac{\sin x}{(n-1) \cos^{n-1} x} + \frac{n-2}{n-1} \int \frac{dx}{\cos^{n-2} x} \\
 \int \cos^m x \sin^n x dx &= \frac{\cos^{m-1} x \sin^{n+1} x}{m+n} + \frac{m-1}{m+n} \int \cos^{m-2} x \sin^n x dx \\
 &= -\frac{\cos^{m+1} x \sin^{n-1} x}{m+n} + \frac{n-1}{m+n} \int \cos^m x \sin^{n-2} x dx \\
 \int \frac{dx}{a+b \sin x} &= \begin{cases} \frac{2}{\sqrt{a^2-b^2}} \arctan \frac{a \tan \frac{x}{2} + b}{\sqrt{a^2-b^2}} & (a^2 > b^2) \\ \frac{1}{\sqrt{b^2-a^2}} \ln \left| \frac{a \tan \frac{x}{2} + b - \sqrt{b^2-a^2}}{a \tan \frac{x}{2} + b + \sqrt{b^2-a^2}} \right| & (a^2 < b^2) \end{cases} \\
 \int \frac{dx}{a+b \cos x} &= \begin{cases} \frac{2}{\sqrt{a^2-b^2}} \arctan \left(\sqrt{\frac{a-b}{a+b}} \tan \frac{x}{2} \right) & (a^2 > b^2) \\ \frac{1}{\sqrt{b^2-a^2}} \ln \left| \frac{\tan \frac{x}{2} \sqrt{b+a} + \sqrt{b-a}}{\tan \frac{x}{2} \sqrt{b+a} - \sqrt{b-a}} \right| & (a^2 < b^2) \end{cases} \\
 \int \frac{dx}{a^2 \cos^2 x + b^2 \sin^2 x} &= \frac{1}{ab} \arctan \left(\frac{b}{a} \tan x \right) \\
 \int \frac{dx}{a^2 \cos^2 x - b^2 \sin^2 x} &= \frac{1}{2ab} \ln \left| \frac{b \tan x + a}{b \tan x - a} \right| \\
 \int x \sin(ax) dx &= \frac{1}{a^2} \sin(ax) - \frac{1}{a} x \cos(ax) \\
 \int x^2 \sin(ax) dx &= -\frac{1}{a} x^2 \cos(ax) + \frac{2}{a^2} x \sin(ax) + \frac{2}{a^3} \cos(ax) \\
 \int x \cos(ax) dx &= \frac{1}{a^2} \cos(ax) + \frac{1}{a} x \sin(ax) \\
 \int x^2 \cos(ax) dx &= \frac{1}{a} x^2 \sin(ax) + \frac{2}{a^2} x \cos(ax) - \frac{2}{a^3} \sin(ax)
 \end{aligned}$$

Inverse Trigonometric Integrals ($a > 0$)

$$\begin{aligned}
 \int \arcsin \frac{x}{a} dx &= x \arcsin \frac{x}{a} + \sqrt{a^2 - x^2} + C \\
 \int x \arcsin \frac{x}{a} dx &= \left(\frac{x^2}{2} - \frac{a^2}{4} \right) \arcsin \frac{x}{a} + \frac{x}{4} \sqrt{a^2 - x^2} + C \\
 \int x^2 \arcsin \frac{x}{a} dx &= \frac{x^3}{3} \arcsin \frac{x}{a} + \frac{1}{9} (x^2 + 2a^2) \sqrt{a^2 - x^2} + C \\
 \int \arccos \frac{x}{a} dx &= x \arccos \frac{x}{a} - \sqrt{a^2 - x^2} + C \\
 \int x \arccos \frac{x}{a} dx &= \left(\frac{x^2}{2} - \frac{a^2}{4} \right) \arccos \frac{x}{a} - \frac{x}{4} \sqrt{a^2 - x^2} + C \\
 \int x^2 \arccos \frac{x}{a} dx &= \frac{x^3}{3} \arccos \frac{x}{a} - \frac{1}{9} (x^2 + 2a^2) \sqrt{a^2 - x^2} + C \\
 \int \arctan \frac{x}{a} dx &= x \arctan \frac{x}{a} - \frac{a}{2} \ln(a^2 + x^2) + C \\
 \int x \arctan \frac{x}{a} dx &= \frac{1}{2} (x^2 + a^2) \arctan \frac{x}{a} - \frac{ax}{2} + C
 \end{aligned}$$

$$\int x^2 \arctan \frac{x}{a} dx = \frac{x^3}{3} \arctan \frac{x}{a} - \frac{ax^2}{6} + \frac{a^3}{6} \ln(a^2 + x^2) + C$$

Exponential Integrals

$$\begin{aligned} \int a^x dx &= \frac{a^x}{\ln a} + C \\ \int e^{ax} dx &= \frac{1}{a} e^{ax} + C \\ \int x e^{ax} dx &= \frac{e^{ax}}{a^2} (ax - 1) + C \\ \int x^n e^{ax} dx &= \frac{1}{a} x^n e^{ax} - \frac{n}{a} \int x^{n-1} e^{ax} dx \\ \int x a^x dx &= \frac{x a^x}{\ln a} - \frac{a^x}{(\ln a)^2} + C \\ \int x^n a^x dx &= \frac{x^n a^x}{\ln a} - \frac{n}{\ln a} \int x^{n-1} a^x dx \\ \int e^{ax} \sin(bx) dx &= \frac{e^{ax}}{a^2 + b^2} (a \sin(bx) - b \cos(bx)) + C \\ \int e^{ax} \cos(bx) dx &= \frac{e^{ax}}{a^2 + b^2} (a \cos(bx) + b \sin(bx)) + C \\ \int e^{ax} \sin^n(bx) dx &= \frac{e^{ax} \sin^{n-1}(bx)}{a^2 + b^2 n^2} (a \sin(bx) - nb \cos(bx)) \\ &\quad + \frac{n(n-1)b^2}{a^2 + b^2 n^2} \int e^{ax} \sin^{n-2}(bx) dx \\ \int e^{ax} \cos^n(bx) dx &= \frac{e^{ax} \cos^{n-1}(bx)}{a^2 + b^2 n^2} (a \cos(bx) + nb \sin(bx)) \\ &\quad + \frac{n(n-1)b^2}{a^2 + b^2 n^2} \int e^{ax} \cos^{n-2}(bx) dx \end{aligned}$$

Logarithmic Integrals

$$\begin{aligned} \int \ln x dx &= x \ln x - x + C \\ \int \frac{dx}{x \ln x} &= \ln |\ln x| + C \\ \int x^n \ln x dx &= \frac{x^{n+1}}{n+1} \left(\ln x - \frac{1}{n+1} \right) + C \\ \int (\ln x)^n dx &= x (\ln x)^n - n \int (\ln x)^{n-1} dx \\ \int x^m (\ln x)^n dx &= \frac{x^{m+1} (\ln x)^n}{m+1} - \frac{n}{m+1} \int x^m (\ln x)^{n-1} dx \end{aligned}$$

7.3 Python Hint

```

1 def RandomAndList():
2     """
3     关于随机数和列表操作的常用技巧。
4     """
5     import random
6
7     # 生成一个符合正态分布（高斯分布）的随机浮点数。
8     # 第一个参数是均值（mu），第二个参数是标准差（sigma）。
9     # 类似于 C++ 中使用 <random> 库的 std::normal_distribution。
10    random.normalvariate(0.5, 0.1)
11
12    # 列表推导式：一种非常 Pythonic 的创建列表的方式。
13    # 这里创建了一个包含字符串 '0' 到 '8' 的列表。
14    # 注意：l 中的元素是 str 类型，不是 int 类型。

```

```
15 l = [str(i) for i in range(9)]
16
17 # sorted(l): 返回一个新的已排序的列表, 原列表 l 不变。由于元素是字符串, 所以是按字典序排序。
18 # min(l), max(l): 同样按字典序返回最小/最大元素。
19 # len(l): 返回列表长度, 相当于 C++ 中的 a.size()。
20 sorted(l), min(l), max(l), len(l)
21
22 # random.shuffle(l): 原地打乱列表 l 中的元素顺序。
23 # 相当于 C++ 中的 std::shuffle。
24 random.shuffle(l)
25
26 int_l = [i for i in range(9)]
27
28 # l.sort(): 原地排序列表。
29 # key=lambda x: x ^ 1: `key` 参数接受一个函数, 该函数为列表中的每个元素计算一个用于排序的
    ↳ “键”。
30 # `lambda x: ...` 是一个匿名函数, 这里表示对于每个元素 x, 使用 x ^ 1 的结果进行排序。
31 # reverse=True: 表示降序排序。
32 int_l.sort(key=lambda x: x ^ 1, reverse=True)
33
34 from functools import cmp_to_key
35 # 在 Python 3 中, sort 不再支持 cmp 函数 (即比较两个元素的函数)。
36 # cmp_to_key 可以将一个老式的 cmp 函数 (返回负、零、正) 转换成一个 key 函数。
37 # 这对于习惯了 C++ `sort` 中自定义比较器的选手很有帮助。
38 # `lambda x, y: (y^1) - (x^1)` 就是一个 cmp 函数。
39 int_l.sort(key=cmp_to_key(lambda x, y: (y ^ 1) - (x ^ 1)))
40
41
42 # itertools 模块: 包含了一系列用于高效迭代的工具, 功能非常强大。
43 from itertools import *
44
45 # product: 计算笛卡尔积。相当于 C++ 中的多层嵌套 for 循环。
46 # product('ABCD', repeat=2) 会生成 AA AB AC AD BA BB BC BD...
47 for i in product('ABCD', repeat=2):
48     pass
49
50 # permutations: 生成指定长度的全排列。
51 # permutations('ABCD', 2) 会生成 AB AC AD BA BC BD CA CB... (P(4, 2))
52 for i in permutations('ABCD', 2):
53     pass
54
55 # combinations: 生成指定长度的组合 (不考虑顺序)。
56 # combinations('ABCD', 2) 会生成 AB AC AD BC BD CD (C(4, 2))
57 for i in combinations('ABCD', 2):
58     pass
59
60 # combinations_with_replacement: 生成可重复选择的组合。
61 # combinations_with_replacement('ABCD', 2) 会生成 AA AB AC AD BB BC BD CC CD DD
62 for i in combinations_with_replacement('ABCD', 2):
63     pass
64
65 def FractionOperation():
66     """
67     分数 (有理数) 运算, 可以避免浮点数精度问题。
68     """
69     from fractions import Fraction
70
71     # 示例分数
72     a = Fraction(3, 4) # a = 3/4
73
74     # a.numerator: 获取分子
75     # a.denominator: 获取分母
76     # str(a): 将分数转换为字符串 "3/4"
77     print(a.numerator, a.denominator, str(a))
78
79     # 一个非常实用的功能: 将一个浮点数转换为最接近它的、分母不超过指定值的分数。
80     # 这在处理一些需要精确分数的几何或数论问题时非常有用。
81     a = Fraction(0.233).limit_denominator(1000) # 结果会是 7/30
```

```
82
83 def DecimalOperation():
84     """
85     高精度十进制数运算。这是 Python 在算法竞赛中替代 C++ 高精度写法的核心。
86     """
87     from decimal import Decimal, getcontext, FloatOperation
88
89     # 设置全局精度（有效数字位数）。
90     getcontext().prec = 100
91
92     # 设置舍入模式，默认是 ROUND_HALF_EVEN（四舍六入五成双）。
93     # 其他模式如 ROUND_FLOOR（向下舍入），ROUND_CEILING（向上舍入）等。
94     # import decimal # 需要先导入 decimal 才能使用下面的属性
95     # getcontext().rounding = getattr(decimal, 'ROUND_HALF_EVEN')
96
97     # 设置一个陷阱，当 Decimal 和 float 混合运算时会抛出异常。
98     # 这是一个好习惯，可以防止不经意间的精度损失。
99     getcontext().traps[FloatOperation] = True
100
101     # Decimal 的构造方式之一：(符号, (数字元组), 指数)
102     # (0, (1, 4, 1, 4), -3) 表示 +1.414
103     d = Decimal((0, (1, 4, 1, 4), -3))
104     print(d)
105
106     # 进行高精度计算，这里的数字远超标准 int 或 double 的范围。
107     a = Decimal(1 << 63) / Decimal(10**18)
108
109     # 使用 f-string 格式化输出，可以指定小数位数。
110     print(f"{a:.9f}")
111
112     # Decimal 对象支持常见的数学运算，并且保持精度。
113     print(a.sqrt(), a.ln(), a.log10(), a.exp(), a ** 2)
114
115 def ComplexOperation():
116     """
117     复数运算。Python 原生支持复数。
118     """
119     # Python 使用 'j' 来表示虚数单位。
120     a = 1 - 2j
121
122     # a.real: 获取实部 (float)
123     # a.imag: 获取虚部 (float)
124     # a.conjugate(): 获取共轭复数
125     print(a.real, a.imag, a.conjugate())
126
127 def FastIO():
128     """
129     在 Python 中实现快速 I/O 的一个常用模板。
130     原理类似于 C++ 的 `ios::sync_with_stdio(false); cin.tie(NULL);`
131     并且一次性读入所有数据，再一次性写出。
132     """
133     import sys, atexit, io
134
135     # 1. 在程序开始时，一次性读取所有标准输入到内存中。
136     _INPUT_LINES = sys.stdin.read().splitlines()
137
138     # 2. 创建一个输入行的迭代器，并重载 `input()` 函数。
139     # 之后每次调用 `input()` 都只是从内存中取下一行，而不是进行 I/O 操作。
140     input = iter(_INPUT_LINES).__next__
141
142     # 3. 创建一个内存中的输出缓冲区（类似于 C++ 的 stringstream）。
143     _OUTPUT_BUFFER = io.StringIO()
144
145     # 4. 将标准输出重定向到这个内存缓冲区。
146     # 之后所有的 `print()` 都会写入到这个缓冲区，而不是直接输出到屏幕。
147     sys.stdout = _OUTPUT_BUFFER
148
149     # 5. 注册一个程序退出时执行的函数。
```

```
150     @atexit.register
151     def write():
152         # 在程序结束时，将缓冲区中的所有内容一次性写入到真正的标准输出。
153         # 这种 "buffered write" 能极大地减少 I/O 次数，从而加速程序。
154         sys.__stdout__.write(_OUTPUT_BUFFER.getvalue())
155
156 # --- 主程序逻辑在这里开始 ---
157 # 如果使用了 FastIO，建议将所有代码都放在一个函数内调用，避免全局作用域问题。
158 def main():
159     # FastIO() # 如果需要，取消这行的注释
160     # 示例：
161     # line = input()
162     # print(f"You entered: {line}")
163     pass
164
165 if __name__ == '__main__':
166     main()
```

8. Miscellany

8.1 初始模板

```
1 #include <bits/stdc++.h>
2
3 #define ls p << 1
4 #define rs p << 1 | 1
5 #define lowbit(x) ((x) & -(x))
6 using namespace std;
7 using ll = long long;
8 constexpr ll INF = 1e18;
9 constexpr int mod = 1e9 + 7;
10 constexpr int N = 1e5 + 10;
11
12 void solve() {}
13
14 int main() {
15     ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
16     int t = 1;
17     cin >> t;
18     while (t--) solve();
19     return 0;
20 }
```

8.2 WIN 运行脚本

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(int argc, char* argv[]) {
5     string s = argv[1];
6     string cpp = s + ".cpp";
7     string exe = s + ".exe";
8     string out = s + ".out";
9     string err = s + ".err";
10
11     string cp =
12         "g++ -std=c++20 -O2 -Wl,--stack,536870912 " + cpp + " -o " + exe; // 设置栈为
13         ↳ 512 MB
14
15     if (system(cp.c_str()) != 0) {
16         cerr << "Compile Failed!" << endl;
17         return 1;
18     } else {
19         system("cls");
20         cerr << "Compile Success!" << endl;
21     }
22
23     string run = exe;
24     if (argc > 2) {
25         run += " < " + string(argv[2]);
26     }
27     run += " > " + out + " 2> " + err;
28
29     system(run.c_str());
30
31     cout << "\n--- Output ---\n";
32 }
```

```

31     system(("type " + out).c_str());
32     cout << "\n-----\n";
33
34     cout << "\n--- Error ---\n";
35     system(("type " + err).c_str());
36     cout << "\n-----\n";
37
38     return 0;
39 }

```

8.3 Zeller 日期公式

```

1 // weekday=(id+1)%7;{Sun=0,Mon=1,...}   getId(1, 1, 1) = 0
2 int getId(int y, int m, int d) {
3     | if (m < 3) { y--; m += 12; }
4     | return 365 * y + y / 4 - y / 100 + y / 400 + (153 * (m - 3) + 2) / 5 + d - 307;
5     | }
6 // y<0: 统一加 400 的倍数年
7 auto date(int id) {
8     | int x=id+1789995, n, i, j, y, m, d;
9     | n = 4 * x / 146097; x -= (146097 * n + 3) / 4;
10    | i = (4000 * (x + 1)) / 1461001; x -= 1461 * i / 4 - 31;
11    | j = 80 * x / 2447; d = x - 2447 * j / 80; x = j / 11;
12    | m = j + 2 - 12 * x; y = 100 * (n - 49) + i + x;
13    | return make_tuple(y, m, d); }

```

8.4 有理数二分: Stern-Brocot 树, Farey 序列

```

1 def build(a, b, c, d, level = 1):
2     x = a + c; y = b + d
3     build(a, b, x, y, level + 1)
4     build(x, y, c, d, level + 1)
5 build(0, 1, 1, 0) # 最简分数, Stern-Brocot
6 build(0, 1, 1, 1) # 最简真分数, Farey

```

8.5 黄金三分

```

1 constexpr LD R = (sqrt(5) - 1) / 2;
2 auto split = [](LD l, LD r) { return l + (r - l) * R; };
3 LD solve(LD a, LD c, auto f) {
4     | LD b = split(a, c), bv = f(b);
5     | for (int _ = T; _; _--) {
6     |     | LD x = split(a, b), xv = f(x);
7     |     | if (xv < bv) c = b, b = x, bv = xv; // 最小化, 注意符号
8     |     | else a = c, c = x;
9     | } return bv; }

```

8.6 DP 优化

8.6.1 四边形不等式

```

1 // a ≤ b ≤ c ≤ d: w(b, c) ≤ w(a, d), w(a, c) + w(b, d) ≤ w(a, d) + w(b, c)
2 for (int len = 2; len <= n; ++len) {
3     | for (int l = 1, r = len; r <= n; ++l, ++r) {
4     |     | f[l][r] = INF;
5     |     | for (int k = m[l][r - 1]; k <= m[l + 1][r]; ++k) {

```

```

6 |         |         | if (f[l][r] > f[l][k] + f[k + 1][r] + w(l, r)) {
7 |         |         |     f[l][r] = f[l][k] + f[k + 1][r] + w(l, r);
8 |         |         |     m[l][r] = k;
9 |     } } }

```

8.6.2 树形背包优化

限制: 必须取与根节点相连的一个连通块.

转化: 一个点的子树对应于 DFS 序中的一个区间. 则每个点的决策为, 取该点, 或者舍弃该点对应的区间. 从后往前 dp, 设 $f(i, v)$ 表示从后往前考虑到 i 号点, 总体积为 V 的最优价值, 设 i 号点对应的区间为 $[i, i + \text{size}_i - 1]$, 转移为 $f(i, v) = \max\{f(i + 1, V - v_i) + w_i, f(i + \text{size}_i, v)\}$.

如果要求任意连通块, 则点分治后转为指定根的连通块问题即可.

8.6.3 $O(n \cdot \max a_i)$ Subset Sum

```

1 int SubsetSum(vector<int> &a, int t) {
2     int B = *max_element(a.begin(), a.end());
3     int n = (int) a.size(), s = 0, i = 0;
4     while (i < n && s + a[i] <= t) s += a[i++];
5     if (i == n) return s;
6     vector<int> f(2 * B + 1, -1), pre (B + 1, -1);
7     f[s - (t - B)] = i;
8     for ( ; i < n; i++) {
9         s += a[i];
10        for (int d = 0; d <= B; d++) pre[d] = max(0, f[d + B]);
11        for (int d = B; d >= 0; d--)
12            f[d + a[i]] = max(f[d + a[i]], f[d]);
13        for (int d = 2 * B; d > B; --d)
14            for (int j = pre[d - B]; j < f[d]; j++)
15                f[d - a[j]] = max(f[d - a[j]], j); }
16    for (i = 0; i <= B; i++) if (f[B - i] >= 0) return t - i;}

```

8.7 O3, 读入优化

```
1 #pragma GCC optimize("Ofast,unroll-loops")
2
3 int read() {
4     char c = getchar();
5     int f = 1, x = 0;
6     while (c < 48 || c > 57) {
7         if (c == '-') f = -1;
8         c = getchar();
9     }
10    while (c >= 48 && c <= 57) {
11        x = 10 * x + c - 48;
12        c = getchar();
13    }
14    return x * f;
15 }
16 void write(int x) {
17     if (x < 0) putchar('-'), x = -x;
18     if (x > 9) write(x / 10);
19     putchar(x % 10 + 48);
20 }
```

8.8 builtin

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     unsigned int a = 99; // 32 位无符号整数
6     unsigned long long b = 1234567890123456789ULL; // 64 位无符号长整型
7
8     // --- 1. __builtin_clz / __builtin_clzll ---
9     // 计算前导零的个数
10
11    // 对于 a = 99 (二进制 ...01100011)
12    // 32 位表示: 00000000 00000000 00000000 01100011
13    // 最高位的 '1' 在第 7 位 (从右往左, 0 开始), 所以前面有 32 - 7 = 25 个零。
14    int clz_a = __builtin_clz(a); // 25
15
16    // 对于 b = 1234567890123456789ULL
17    // 64 位表示: 00010001 00101101 01001011 10000010 11010010 01010101 01100011
18    // 00010101 最高位的 '1' 在第 60 位, 所以前面有 64 - 61 = 3 个零。
19    int clz_b = __builtin_clzll(b); // 3
20    // 对 0 调用 clz 是未定义行为
21
22    // --- 2. __builtin_ctz / __builtin_ctzll ---
23    // 计算末尾零的个数
24
25    unsigned int c = 120;
26    // 32 位表示: 00000000 00000000 00000000 01111000
27    // 最低位的 '1' 在第 3 位, 所以后面有 3 个零。
28    int ctz_c = __builtin_ctz(c); // 3
29
30    unsigned long long d = 96ULL; // ...01100000
31    int ctz_d = __builtin_ctzll(d); // 5
32    // 对 0 调用 ctz 是未定义行为
33
34    // --- 3. __builtin_popcount / __builtin_popcountll ---
```



```

35 // 计算二进制中 '1' 的个数
36
37 // 对于 a = 99
38 // 32 位表示: 00000000 00000000 00000000 01100011, 共有 4 个'1'
39 int popcount_a = __builtin_popcount(a); // 4
40
41 // 对于 b = 1234567890123456789ULL
42 // 64 位表示: 00010001 00101101 01001011 10000010 11010010 01010101 01100011
43 // 00010101 共有 32 个'1'
44 int popcount_b = __builtin_popcountll(b); // 32
45
46 // --- 4. __builtin_parity / __builtin_parityll ---
47 // 检查 '1' 的个数是奇数还是偶数
48
49 unsigned int f = 103; // ...01100111, 有 5 个'1' (奇数)
50
51 int parity_f = __builtin_parity(f); // 1
52
53 // b 有 32 个'1' (偶数)
54 int parity_b = __builtin_parityll(b); // 0
55
56 return 0;
57 }

```

8.9 试机赛与纪律文件

- 检查身份证件: 护照、学生证、胸牌以及现场所需通行证。
- 确认什么东西能带进场。特别注意: 智能手表、金属 (钥匙) 等等。
- 测试鼠标、键盘、显示器和座椅。如果有问题, 立刻联系工作人员。
- 测试比赛提交方式。如果有 submit 命令, 确认如何使用。
- 设置自动保存, 自动备份。
- 测试编译器版本。C++20 cin >> (s + 1); C++17 auto [x, y]: a; C++14 [] (auto x, auto y); C++11 auto; bits/stdc++.h; pb_ds。

```

#include <ext/rope>
using namespace __gnu_cxx;
rope <int> R; R.insert(y, x); R[x]; R.erase(x, 1);
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
tree <int, null_type, less<int>, rb_tree_tag,
tree_order_statistics_node_update> s;
s.insert(1); s.find_by_order(0); s.order_of_key(5);

```

- 测试 __int128, __float128, long double
- 测试代码长度限制, 尝试触发 NO-OUTPUT, OUTPUT-LIMIT, RUN-ERROR。
- 测试 pragma 是否 CE。
- 测试 clar: 如果问不同类型的愚蠢的问题, 得到的回复是否不一样?
- 测试 -fsanitize address, undefined, define_GLIBCXX_DEBUG
- 测试 time 命令是否能显示内存占用。/usr/bin/time -v ./a.out
- 测试 clock() 是否能够正常工作; 测试本地性能与提交性能。

```

const int N = 1 << 20;
for (int T = 4; T; T--) {
    int a[N], b[N], c[N];
    for (int i = 0; i < N; i++) a[i] = i, b[i] = N - i;
    ntt_init(N); ntt(a, N, 1); ntt(b, N, 1);
    for (int i = 0; i < N; i++) c[i] = a[i] * b[i];
    ntt(c, N, -1); assert(c[0] == 0xad223d); }
// 机房: 190ms; (1s T = 21)
// QOJ: 340+-40ms; (1s T = 11.9)
// CF: 1050+-100ms; (1s T = 3.9)

import sys
sys.set_int_max_str_digits(0) # >= Python 3.11
T = 3
for _ in range(T):
    assert str(3 ** 100000 * 10 ** 100000)[:9] == "133497141"
# 机房: 590ms; (1s T = 5)
# QOJ: 201ms; (1s T = 16)

```

CF Python3: 1100ms; (1s T = 2.9) Pypy3: 310ms; (1s T = 12)

提交前检查:

- 保存, 编译, 测过样例, 测过边界 $n=0/1$
- 数组开到 $n/2n/m/n+32, LL/int128$
- 多测清空, 调用初始化, 清空时数组大小, 没读入完就 break
- 取模取到正值, 输入输出格式 (%Lf, %llu)
- 时间空间限制, 关闭流同步, 时间卡开 Ofast

提交后检查:

- sum 多组输入: 应只用了输入内容 (memset TL, 错下标 WA), 是否正确撤销
- 输入是否保证顺序, 保证是整数, 保证三点共线
- int/LL 溢出, INF/-1 大小, 浮点数 eps 和 = 0, __builtin_popcountll
- 类似 pair <LL, int> x = pair <int, int>(); 不会报警告
- 离散化与二分: lower_bound upper_bound +-1, begin(), end()
- 自定义排序: 排序方向, 比较函数为小于, 考虑坏点: 如叉积 (0, 0)
- 样例是否为对称/回文的? 考虑构造不对称的情况, 正序逆序
- 复制过的代码对应位置正确修改
- 检查变量重名, 变量复用
- 图: 边标号初始是否为 1, 单双向边, 反向边边权
- 几何: 共线, sqrt(-0.0), nan / inf, 重点, 除零 det/dot, 旋转方向, 求得的是否是所求

8.10 Constant Table 常数表

NTT 976224257 r=3 (20) 985661441 r=3 (22) 998244353 r=3 (23)

1004535809 r=3 (21) 1007681537 r=3 (20) 1012924417 r=5 (21)

1045430273 r=3 (20) 1051721729 r=6 (20) 1053818881 r=7 (20)

n	$\log_{10} n$	$n!$	$C(n, n/2)$	$\text{LCM}(1 \dots n)$	P_n	
2	0.30102999	2	2	2	2	
3	0.47712125	6	3	6	3	
4	0.60205999	24	6	12	5	
5	0.69897000	120	10	60	7	
6	0.77815125	720	20	60	11	
7	0.84509804	5040	35	420	15	
8	0.90308998	40320	70	840	22	
9	0.95424251	362880	126	2520	30	
10	1	3628800	252	2520	42	
11	1.04139269	39916800	462	27720	56	
12	1.07918125	479001600	924	27720	77	
15	1.17609126	1.31e12	6435	360360	176	
20	1.30103000	2.43e18	184756	232792560	627	
25	1.39794001	1.55e25	5200300	26771144400	1958	
30	1.47712125	2.65e32	155117520	1.444e14	5604	
P_n	37338 ₄₀	204226 ₅₀	966467 ₆₀	190569292 ₁₀₀	1e9 ₁₁₄	
$n \leq$	10	100	1e3	1e4	1e5	1e6
$\max \omega(n)$	2	3	4	5	6	7
$\max d(n)$	4	12	32	64	128	240
$\pi(n)$	4	25	168	1229	9592	78498
$n \leq$	1e7	1e8	1e9	1e10	1e11	1e12
$\max \omega(n)$	8	8	9	10	10	11
$\max d(n)$	448	768	1344	2304	4032	6720
$\pi(n)$	664579	5761455	5.08e7	4.55e8	4.12e9	3.7e10
$n \leq$	1e13	1e14	1e15	1e16	1e17	1e18
$\max \omega(n)$	12	12	13	13	14	15
$\max d(n)$	10752	17280	26880	41472	64512	103680
$\pi(n)$	Prime number theorem: $\pi(x) \sim x/\log(x)$					

Vimrc, Bashrc

```

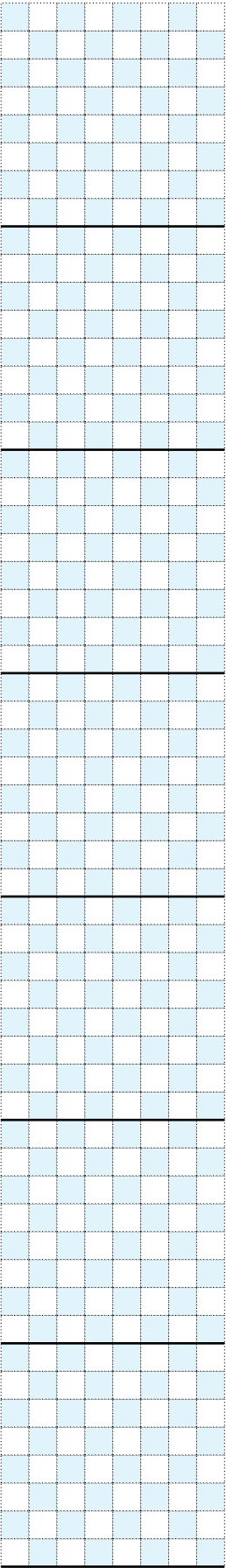
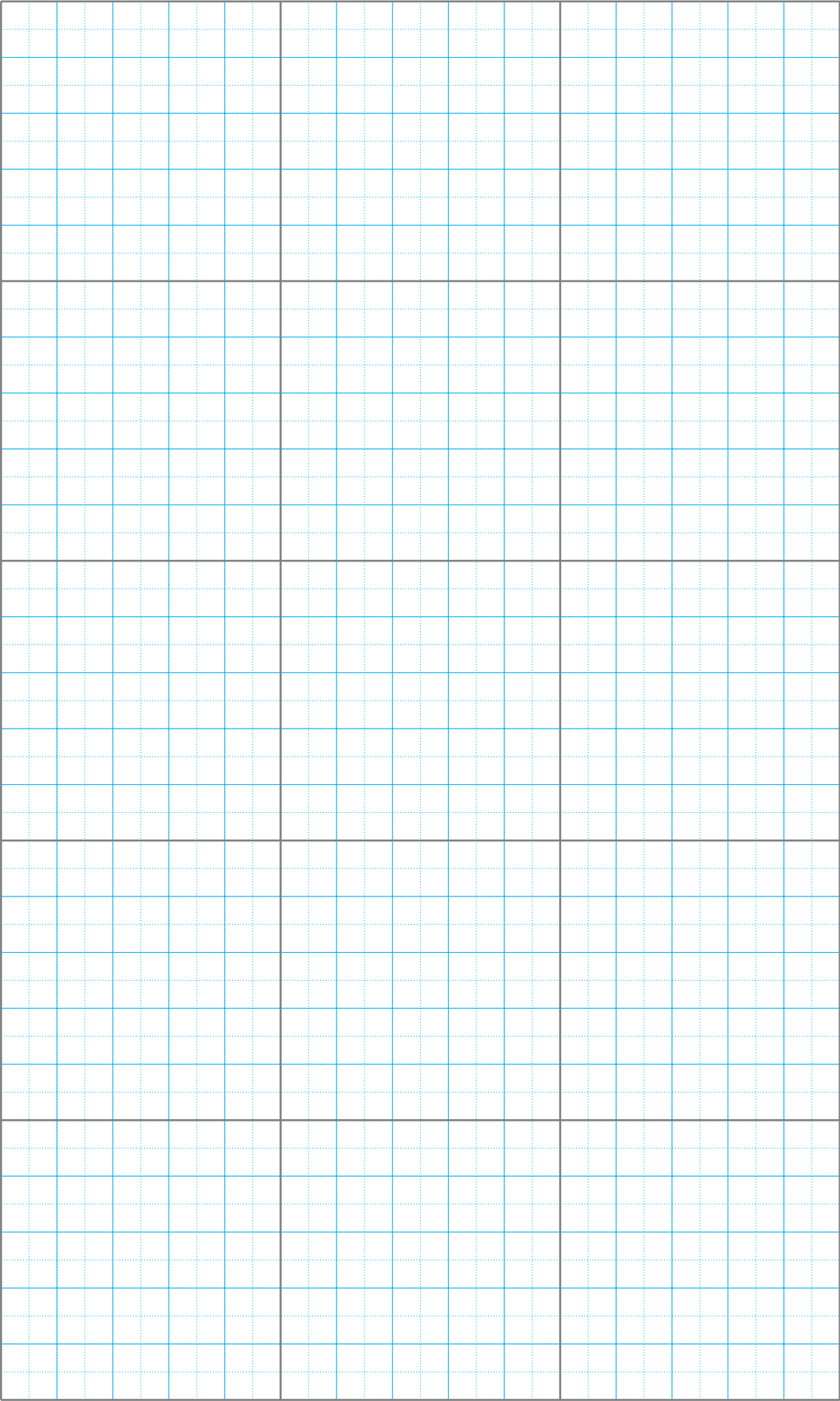
1 source $VIMRUNTIME/mswin.vim
2 behave mswin
3 set mouse=a ci ai si nu ts=4 sw=4 is hls backup undofile
4 color slate

```

```
5 map <F7> : ! make %<<CR>
6 map <F8> : ! time ./%< <CR>
```

```
1 export CXXFLAGS='-g -Wall -Wextra -Wconversion -Wshadow -std=gnu++20'
2 ulimit -s 1048576
```

VSCode: 打开自动保存



STL 积分/求和

1. $\int_0^1 t^{x-1}(1-t)^{y-1}dt = \text{beta}(x, y) = \frac{\Gamma(x)\Gamma(y)}{\Gamma(x+y)}$
2. $\int_0^{+\infty} t^{\text{num}-1}e^{-t}dt = \text{tgamma}(\text{num}) = e^{\text{lgamma}(\text{num})} = \Gamma(\text{num})$
3. $\int_0^{\text{phi}} \frac{d\theta}{\sqrt{1-k^2 \sin^2 \theta}} = \text{ellint}_1(k, \text{phi})$
4. $\int_0^{\text{phi}} \sqrt{1-k^2 \sin^2 \theta} d\theta = \text{ellint}_2(k, \text{phi})$
5. $\int_{\text{num}}^{+\infty} \frac{e^{-t}}{t} dt = -\text{expint}(-\text{num})$
6. $\sum_{n=1}^{+\infty} n^{-\text{num}} = \text{riemann_zeta}(\text{num})$
7. $\frac{2}{\sqrt{\pi}} \int_0^{\text{arg}} e^{-t^2} dt = \text{erf}(\text{arg})$